Overview of Kalman Filter Theory and Applications

Michael L. Carroll

June 7, 2007

©2007 by Michael L. Carroll

Course Overview

- Topic Overview
- Course Structure
- Detailed Topic List
- Instructor Background

Course Overview: Topic Overview

- Kalman Filtering
  - General Theory Overview w/o proofs
  - Practical Aspects of Application
  - Matlab and Simulink Examples
- Applications to GPS Control Segment

Course Overview: Course Structure

- 10 segments (based on 5-day engineering short course)
- 1 hour per segment
- Informational Course
  - Concept
  - Example
  - Exercise (optional)

Course Overview: Detailed Topic List

- Segments 1-2: The Basic Kalman Equations, Part I – Heuristic Overview

- Segments 3-4: The Basic Kalman Equations, Part II – State Dynamics, Probability, Statistics and Random Processes

- Segment 5-6: General Code Implementation Issues and Applications to Navigation

- Segments 7-10: Details of the GPS Control Segment Kalman Filter

Course Overview: Instructor Background

- Michael Carroll
- Masters Degree in Mathematics
- Software Integrator on GPS SE&I Contract

Course Overview: Instructor Background
Formerly ...

- Software Architect on C-130 Avionics Modernization Program at Boeing in Long Beach, CA

- Worked with Calibration and Alignment filters for ballistic missile inertial measurement units (IMU)

- Led avionics software development efforts on various USAF Special Operations Command programs at Lockheed Aircraft Service Company (LASC) in Ontario, CA

Course Overview: Instructor Background
Formerly ...

- GPS/INS Navigation Analyst at BEI Systron Donner

- Have taught formal Kalman filtering courses several times

Kalman Filter Overview

- Optimal state estimator of uncertain dynamic system using noisy measurements
- Essentially recursive least squares
- Rudolf E. Kalman
  Seminal paper in 1960: "A New Approach to Linear Filtering and Prediction Problems"

`http://www.cs.unc.edu/~welch/kalman/kalmanPaper.html`

Kalman Filter Overview

- Actually preceded by work at RAND Corp by Peter Swerling in 1950s
- Least squares goes back to Gauss, Lagrange late 1700s and early 1800s
- Rigorous mathematical foundation built on work of mathematician David Hilbert
- Kalman worked in the time domain using state space techniques

Kalman Filter Overview

- Used in early 1960s by Schmidt on Apollo program
- Kalman filters are now ubiquitous, especially in navigation

Kalman Filter Overview

- GPS uses:
  - All three segments have Kalman filters
  - CS: ECEST = Ephemeris and Clock Estimator (legacy term)
  - Estimates inertial positions and velocities of SVs, solar pressure parameters, SV and MS clock parameters, and troposphere state at each MS
  - In UE receivers, satellite nav software, and offline computations

– Used in aircraft Operational Flight Programs (OFPs) and Inertial Navigation Systems (INSs) to integrate GPS and other external navaids to optimally estimate INS error states.

– Paradigm shift in navigation:
Was: "INS/GPS"
Is: "GPS/INS".

- Kalman filtering, as a branch of applied mathematics, sits at the confluence of many mathematical subjects:
  differential equations, measure theory, probability, statistics, linear algebra, functional analysis (Hilbert space).

- But its essentials are very intuitive and can be understood with very little math background.

Some Notational Conventions

- Time-dependent scalar or vector quantities:
  $x(k)$ (discrete), $x(t)$ (continuous)

- Corresponding vector components:
  $x_i(k)$ or $x_i(t)$

- Time-dependent matrix quantities (capital letters):
  $P(k)$ (discrete), $P(t)$ (continuous)

- Corresponding matrix components (elements):
  $p_{ij}(k)$ or $p_{ij}(t)$

Day 1: The Basic Kalman Equations, Part I
Segments

- Understanding the Equations: Heuristic Overview
- Taking the Equations Apart
- Differential Equations, Difference Equations and Dynamic Systems
- State Space Concepts

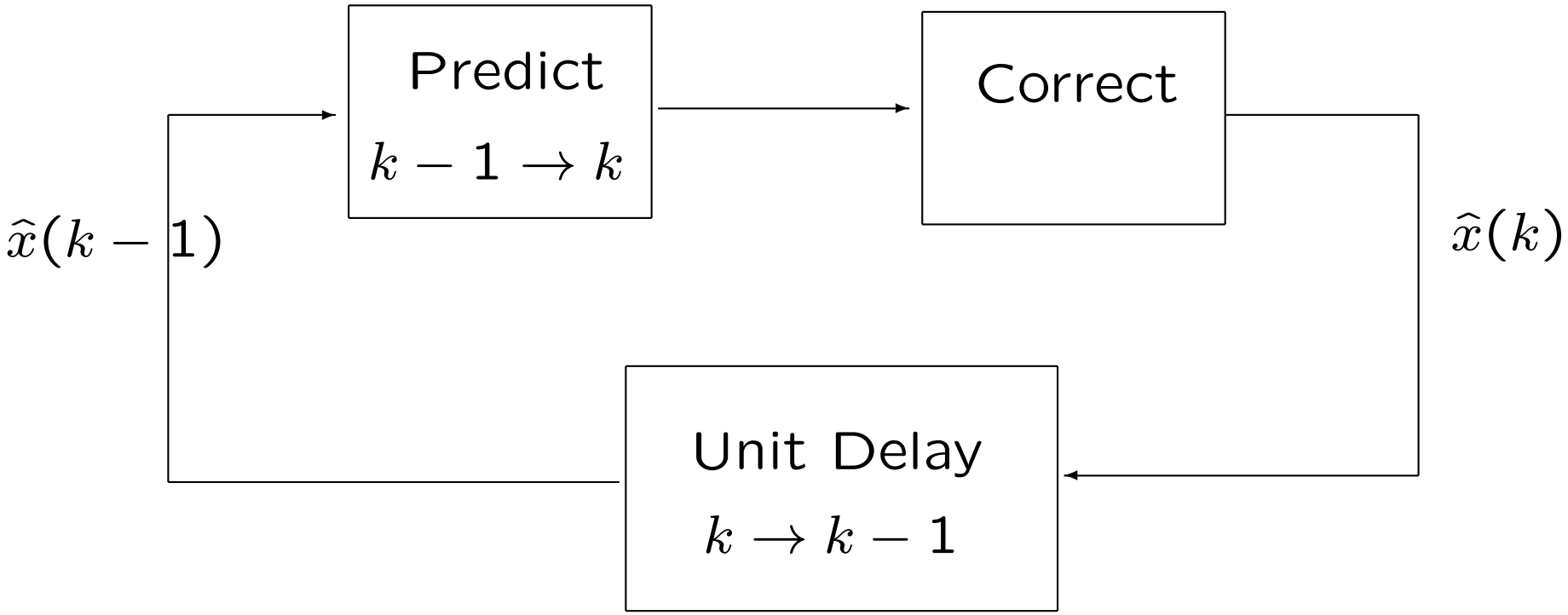
Day 1, Segment 1
Understanding the Equations: Heuristic Overview
Topics

- Recursive Predictor-Corrector Algorithms
- Running Averages
- Extrapolation (Prediction)
- Gain Computation
- Update

Recursive Predictor-Corrector Algorithm

- Prediction:
  - Kalman filter **extrapolates** the solution from the previous iteration to the present time
- Correction:
  - Kalman filter uses measurement information to **update** or **correct** the prediction
- And the cycle begins anew!

Kalman Filter Predictor-Corrector Model



Recursive Predictor-Corrector Algorithm

- Algorithm
  - Computational procedure
  - Given initial values of some quantities, new quantities are computed
- Recursive
  - The new quantities are the same as the old quantities
  - New quantities become initial values for next cycle

Simple Example: Running Averages

- Suppose we want to estimate some quantity like the resistance of a single resistor.

- Suppose we have a voltmeter with which to make measurements.

- Thus we have two distinct quantities: the "true" value x of the resistor and the measurement z given by the voltmeter.

- We are going to make an indefinite number of measurements: $z(1), z(2), \ldots$

Simple Example: Running Averages

- The **state** that we are trying to estimate is the resistance of the resistor.

- We don't know what the state is. Otherwise, we wouldn't need to take measurements.

- Our **state dynamics** or **process model** is very simple: The state is 1-dimensional and does not change with time:

$$x(k+1) = x(k)$$

Simple Example: Running Averages

- What is the best way to estimate the state of our system?
- In the absence of any other information, we should probably just average the measurements.
- We assume that the averages are unbiased, i.e., the measurement errors have zero mean.

Simple Example: Running Averages

- Simple running average is like a Kalman filter with trivial dynamics ($x(k+1) = x(k)$)

Given an infinite sequence of random measurements $z(1), z(2), \ldots$, compute at each stage the best estimate of the true value of the underlying quantity: At the $n+1^{\text{st}}$ step, your best guess would be the average of all previous values:

$$(1) \qquad \widehat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) = \frac{1}{k}[\, z(1) + z(2) + \cdots + z(k)\,]$$

Simple Example: Running Averages

- Q: Why is the average of all previous measurements the "best" estimate?

- A: The dynamics tell us that the quantity to be estimated is not changing, and the only variability stems from the measurements, i.e., the measurement noise.

- Hence, we should try to eliminate the measurement variability by averaging the measruements.

Recursive Running Average

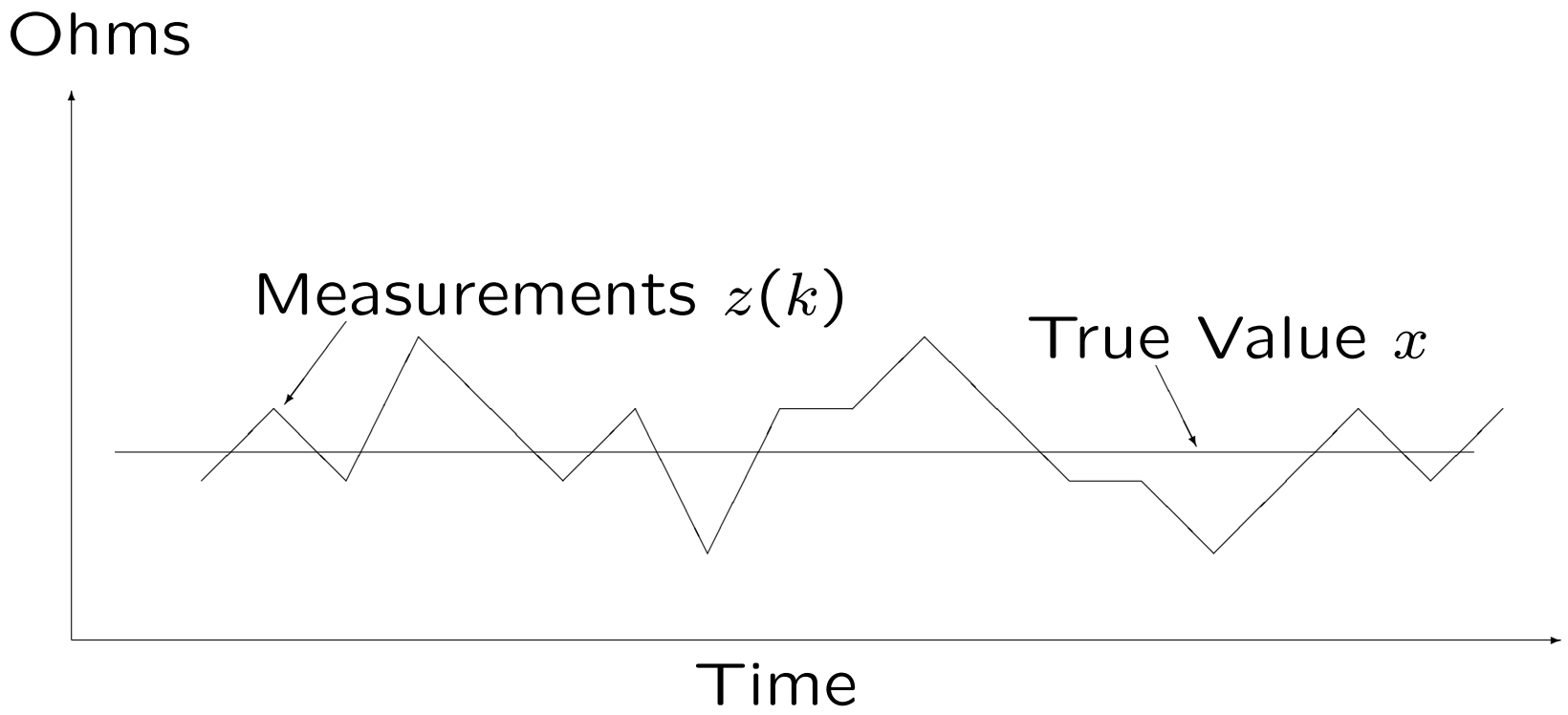
- Therefore, as much as possible, we want to rid ourselves of the noise by averaging.

- Assume that noise is zero-mean, i.e., its average tends to zero over the long run.

Recursive Running Average

- If we had a bias, we wouldn't call that noise. That would be a systematic error on the part of our voltmeter. We'd have to calibrate it out.

- Noise can't be calibrated out; we have to average it out over time.


Ohms
Measurements $z(k)$
True Value x
Time

Recursive Running Average (Eq.(1))

$$\widehat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) = \frac{1}{k}[\, z(1) + z(2) + \cdots + z(k)\,]$$

- Now suppose we are lazy and don't want to recalculate the complete sum with each new measurement.

- We just want to use the previously calculated value and add a small adjustment:

Recursive Running Average

$$\widehat{x}(k+1) = \frac{1}{k}\sum_{j=1}^{k} z(j) \qquad \text{by def. Eq. (1)}$$

$$= \frac{1}{k}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{separate last term from sum}$$

$$= \frac{1}{k}\cdot\frac{k-1}{k-1}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{multiply by 1}$$

$$= \frac{k-1}{k}\cdot\frac{1}{k-1}\sum_{j=1}^{k-1} z(j) + \frac{1}{k}z(k) \qquad \text{rearrange}$$

$$= \frac{k-1}{k}\widehat{x}(k) + \frac{1}{k}z(k) \qquad \text{by def. Eq. (1) again}$$

$$= \widehat{x}(k) - \frac{1}{k}\widehat{x}(k) + \frac{1}{k}z(k) \qquad \text{distribute and simplify}$$

$$= \widehat{x}(k) + \frac{1}{k}\left[\, z(k) - \widehat{x}(k) \,\right]. \qquad \text{rearrange and factor out } 1/n$$

Recursive Running Average

- We now have a recursive algorithm for computing a running average
  - Recipe: Take new measurement, subtract off old average and weight the resulting residual by a gain of $1/k$.
  - Add this weighted residual to the old average and, voilà, you have the new average!

Recursive Running Average

- Note that you do not have to remember all the previous values or averages, only the last average.

- Note also that as k goes to ∞, the gain $1/k$ goes to 0. (The filter is going to sleep!)

$$\widehat{x}(k+1) = \widehat{x}(k) + \frac{1}{k}\left[z(k) - \widehat{x}(k)\right] \tag{2}$$

Recursive Running Average

- The residual $z(k) - \widehat{x}(k)$ is the difference between the actual new measurement and what we expected or predicted the new measurement to be.

- Why is $\widehat{x}(k)$ our predicted measurement?

- Answer: In the absence of new information, the previous estimate is the only thing we have to go on!

Extrapolation (Prediction)

- In the running average example, the dynamics were trivial, i.e., $x(k+1) = x(k)$

- This means that the state never really changes. It's always equal to its previous value.

- We just don't know what the exact value is.

- Example: Measuring the same resistor over and over again with a voltmeter to estimate the resistor's resistance value.

Extrapolation (Prediction)

- In more general Kalman filtering, we complicate the situation by adding two more elements:
  - State dynamics with time dependence and process noise:

$$x(k+1) = g\left[k, x(k), w(k)\right]$$

  - State dynammics with additive process noise:

$$x(k+1) = f\left[k, x(k)\right] + w(k)$$

- The form of the function f can be simple or complicated. The independent variable k indicates possible explicit time dependence.

- The process noise is assumed to be Gaussian white noise (to be defined later).

Extrapolation (Prediction)

- General Kalman filtering further complicates things by allowing the state variable to be vector-valued.

- This turns the function f into a vector-valued function, often a matrix (possibly time-dependent)

- The process noise must therefore also become a vector-valued variable.

Extrapolation (Prediction)

- Note that the presence of a random forcing function like white noise turns the dynamical problem into a stochastic one.

- If the white noise were zero, then the problem would be deterministic.

- We'll see what this means when we take a look at random variables and random processes.

Extrapolation (Prediction)

- Note that we did not mention a control vector u which would also be part of the forcing function.

- We are generally going to ignore the presence of control.

- The control vector is usually considered to be deterministic although in reality you can't control things perfectly! (E.g., elevator deflection angle might be a control quantity and its precise value may not be known!)

Measurement Update (Correction)

- In the running average example, our update implicitly assumed that our measurement was corrupted by white noise.

- A model for this would be: $z(k) = x(k) + v(k)$, where $v(k)$ is a Gaussian white measurement noise sequence.
  - We'll explore what we mean by "Gaussian" later in the course.

Measurement Update (Correction)

- Because we assume that our measurement noise was white Gaussian and zero mean, our best guess was simply the mean value of the measurements.

- Remember the resistor example: our voltmeter is not perfect. Each measurement will differ slightly from the previous one, even though we are measuring the same resistor each time.

Measurement Update (Correction)

- General Kalman filtering complicates the measurement model by adding two new elements:
  - The measurement may have a more complicated relationship to the underlying system state:

$$z(k) = h\left[k, x(k)\right] + v(k),$$

    where h could be a non-linear function with possible explicit time dependence.
  - Just as the state x is allowed to be a vector variable, so the measurement z is also allowed to be a vector variable.

Kalman Gain Computation

- Even through we've discussed the measurement update or observation processing step, there is an important step just before that one.

- Before we can incorporate the information contained in both the old estimate and the new measurement, we have to first calculate the Kalman gain K_k at the kth step.

- $K_k = \frac{1}{k}$ in our running average example.

- If we have k pieces of information *of equal quality*, it seems intuitive that we weight each piece by $1/k$.

Kalman Gain Computation

- What we are weighting is the measurement residual, also called the **innovation**: $z(k) - \widehat{x}(k)$.

- The innovation contains all the new information, i.e., the new measurement, minus what we thought we already knew, i.e., the previous estimate.

- The gain computation occurs after the extrapolation but before updating our estimate with measurement information.

Day1, Segment 1 - Exercises

1. Let's re-familiarize ourselves with vectors and matrices. Addition:

$$\begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} + \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} x_1 + y_1 \\ x_2 + y_2 \\ \vdots \\ x_n + y_n \end{bmatrix}$$

For vectors in three space, this would be:

$$\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} x_1 + y_1 \\ x_2 + y_2 \\ x_3 + y_3 \end{bmatrix}$$

Compute:

$$\begin{bmatrix} -1 \\ 3 \\ 0 \end{bmatrix} + \begin{bmatrix} 7 \\ -1 \\ 5 \end{bmatrix} =$$

$$\begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix} =$$

$$\begin{bmatrix} 0.75 \\ -3 \\ 100 \end{bmatrix} + \begin{bmatrix} 0.75 \\ 0 \\ -1000 \end{bmatrix} =$$

$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} =$$

$$\begin{bmatrix} 1.1 \\ -3.2 \\ 9.7 \end{bmatrix} + \begin{bmatrix} -0.8 \\ 3.2 \\ 10 \end{bmatrix} =$$

2. Multiplying a row vector times a column vector:

$$\begin{bmatrix} a_1 & a_2 & a_3 \end{bmatrix} \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = a_1b_1 + a_2b_2 + a_3b_3$$

Now you try it:

$$\begin{bmatrix} 0 & 1 & 3 \end{bmatrix} \begin{bmatrix} 4 \\ 3 \\ 0 \end{bmatrix} =$$

$$\begin{bmatrix} -1 & 3 & -2 \end{bmatrix} \begin{bmatrix} 1 \\ -2 \\ 2 \end{bmatrix} =$$

$$\begin{bmatrix} x & y & z \end{bmatrix} \begin{bmatrix} u \\ v \\ w \end{bmatrix} =$$

$$\begin{bmatrix} 2x & .5x & -7x \end{bmatrix} \begin{bmatrix} -10 \\ 4 \\ -1 \end{bmatrix} =$$

3. Multiplying a matrix times a column vector:

$$\begin{bmatrix} 1 & 0 & 1 \\ 2 & 7 & 0 \\ 3 & 2 & 5 \end{bmatrix} \begin{bmatrix} 7 \\ -1 \\ 5 \end{bmatrix} =$$

Hint: Multiply the first row of the matrix times the column vector to get the first element of the new column vector. Then multiply by the second row of the matrix, and so on.

$$\begin{bmatrix} 2 & 1 & 0 \\ 0 & 1 & 1 \\ 2 & 2 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ -3 \\ 1 \end{bmatrix} =$$

$$\begin{bmatrix} 3 & 1 & 1 \\ -1 & 1 & -1 \\ 0 & 2 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ -3 \\ 1 \end{bmatrix} =$$

4. What happens if you multiply a column on the left by a row vector on the right?

5. What is the transpose of the following matrix?

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

6. Given the following scalar measurements of some constant scalar process, calculate the average value at each stage k by two different methods: 1) by averaging all past values and 2) by using the recursive algorithm in Eq. (2):

1. $z_1 = 0.76$

2. $z_2 = 0.67$

3. $z_3 = 0.79$

4. $z_4 = 0.81$

5. $z_5 = 0.78$

6. $z_6 = 0.79$

7. $z_7 = 0.81$

8. $z_8 = 0.82$

9. $z_9 = 0.77$

7. You are traveling in a car at 45 mph in a 35 mph speed zone. Your friendly neighborhood policeman is watching you with his radar gun. He is measuring your speed, of course, but needs to report as well your position as a function of time. Write down a dynamics and measurement model for this problem in which the state variable is your positional displacement from some arbitrary initial position. Assume that there is no process noise in your dynamics and that your velocity is not changing. Assume the policeman makes a measurement with his radar gun every 2 seconds.
Hint: You have two variables of interest! 8. How is this problem different from the resistor problem?

Day 1, Segment 2
Taking the Equations Apart
Topics

- Understanding the Notation
- Examples
- Exercises

Understanding the Notation: Kalman Filter Problem Summary

- Given two models:

  1. State Dynamics Model (sometimes called System or Process Model)

  2. Measurement Model

- Find the optimal linear gain K_k with which to weight the measurement residual or innovation in order to compute the optimal estimate of the state (we'll discuss later what we mean by "optimal")

Understanding the Notation: Kalman Filter Problem Summary (General, Non-Linear, Discrete Formulation)

- Given: State Dynamics Model: $x(k) = f[x(k-1)] + w(k)$
  - x=state,f=dynamics function,w=process noise
- Given: Measurement Model: $z(k) = h[x(k)] + v(k)$
  - z=measurement, h=measurement function, v=measurement noise
- Find optimal gain K_k that allows us to update our state estimate in the following way:

$$\widehat{x}(k) = f[\widehat{x}(k-1)] + K_k\ [\ z(k) - h(f[\widehat{x}(k-1)])\ ]$$

Understanding the Notation: Kalman Filter Problem Summary

- Note that we seek the gain in the form of a multiplier (i.e., a gain matrix)

- This is not the most general form possible. The gain could be a (possibly, non-linear) function of the innovation, perhaps something like $g_k(z(k), \widehat{x}(k))$. But we won't pursue that.

Understanding the Notation: Kalman Filter Problem Summary (Linear Discrete Formulation)

- Given: State Dynamics Model: $x(k) = \Phi_{k-1}x(k-1) + w(k)$
  - $x(k)$=state,Φ_{k-1}=state transition matrix, $w(k)$=process noise
- Given: Measurement Model: $z(k) = H_k x(k) + v(k)$
  - $z(k)$=measurement,H_k=measurement matrix, $v(k)$=measurement noise

- Find: State estimate in the form

$$\widehat{x}(k) = \Phi_{k-1}\widehat{x}(k-1) + K_k[z(k) - H_k\Phi_{k-1}\widehat{x}(k-1)]$$

Understanding the Notation: Kalman Filter Problem Summary
Slight Notational Change

- The term $\Phi_{k-1}\widehat{x}(k-1)$ is the extrapolated state estimate prior to making the measurement update or correction.

- To distinguish prior estimate from updated estimate, we use the following notation:

  $\widehat{x}^{-}(k) = \Phi_{k-1}\widehat{x}^{+}(k-1) =$ predicted estimate prior to update

  $$\widehat{x}^{+}(k) = \text{updated state estimate}$$

Understanding the Notation: Kalman Filter Problem Summary

- Thus, the solution we seek looks like this:

$$\hat{x}^{+}(k) = \hat{x}^{-}(k) + K_k[z(k) - H_k\hat{x}^{-}(k)] \tag{3}$$

Understanding the Notation: Kalman Filter Problem Summary

- We've formulated the discrete problem
- We've indicated the form of the solution: the recursive estimating algorithm with the Kalman gain
- But we haven't yet presented the general solution. Only in the case of a simple, constant, scalar system.
- When we arrive at the general solution, we'll get an added benefit: While trying to come up with $\hat{x}^+(k)$ and K_k, we'll also end up with a recursive estimate of our estimation uncertainty! This is the so-called error covariance matrix.

Understanding Kalman Filter Notation
State Dynamics

- State Dynamics Model: $x(k) = f[x(k-1)] + w(k)$

- $x(k)$ is the **state vector** with n elements

- $x(k) = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}_k = \begin{bmatrix} x_1(k) \\ x_2(k) \\ \vdots \\ x_n(k) \end{bmatrix}$

Understanding Kalman Filter Notation
State Dynamics

- Discrete State Dynamics
- General, nonlinear dynamics function f too difficult!
- Restrict to matrices: $x(k) = \Phi_k x(k-1) + w(k)$
- Φ_{k-1} is the state transition matrix. It is $n \times n$ in size.

Understanding Kalman Filter Notation
State Dynamics

- State Transition Matrix: $\Phi_{k-1} = \Phi(k, k-1)$
- Extrapolates state vector from time step $k-1$ to time step k (pulling from past to present)
- Tells you how the state would evolve in the absence of forcing functions
- Remember: this is still just a model. We don't really know what the state is, because the random forcing function messes things up!

Understanding Kalman Filter Notation
State Dynamics

- We are deliberately trying to keep the notation as simple as possible

- We could throw in other factors and terms.
  - A matrix Γ_k to distribute the Gaussian white noise to the states
  - An additional, deterministic control variable u_k, and associated distribution matrix Λ_k.

- For simplicity, we'll generally leave the control variable out of the picture.

The Kalman Filter Solution: State Extrapolation

- Given an estimate $\hat{x}^+(k-1)$ at time $k-1$, we extrapolate it forward in time by the state transition matrix:

$$\hat{x}^-(k) = \Phi_k \hat{x}^+(k-1) \tag{4}$$

- Why is this the right extrapolation equation? Because the white noise forcing function is a zero mean process. Thus, on average, the equation is homogeneous and we should therefore use the homogeneous solution, i.e., the state transition matrix.

The Kalman Filter Solution: Covariance Extrapolation

- How certain are we that our solution is right?

- The answer is given by the **estimation error covariance matrix** or often just called the covariance matrix P_k.

- Just like the state estimate, it is propagated forward in time before being corrected:

$$P_k^- = \Phi_k P_{k-1}^+ \Phi_k^\top + \Gamma_k Q_k \Gamma_k^\top \tag{5}$$

  where Q_k is a matrix called the **process noise covariance matrix**

The Kalman Filter Solution: Covariance Extrapolation

- Generally, we'll assume that the noise distribution matrix Γ_k is the identity matrix. Thus, for covariance extrapolation we'll write instead of (5) the following:

$$P_k^- = \Phi_k P_{k-1}^+ \Phi_k^\top + Q_k \tag{6}$$

- Q_k measures the amount of dispersion in the white noise forcing function $w(k)$

- Later we'll learn how to define P_k.

The Kalman Filter Solution: Covariance Extrapolation

- What does Eq. (78) really mean?

- The term $\Phi_k P_{k-1}^{+} \Phi_k^{\top}$ represents the effect that the state dynamics has on the error covariance.

- The other term Q_k represents the increased uncertainty added into each step due to the process noise inherent in the system. This is due to the white noise in the system, possibly due to the aggregate effect of many unmodelled states.

The Kalman Filter Solution: The Kalman Gain

- The heart of the solution is the **Kalman Gain**: K_k

  (7) $$K_k = P_k^- H_k^\top \left[H_k P_k^- H_k^\top + R_k \right]^{-1}$$

  where R_k is the measurement noise covariance matrix (to be defined later) governing the white measurement noise v_k in the measurement model:

$$z(k) = H_k x(k) + v(k)$$

The Kalman Filter Solution: State Update Equation

- Using the gain, the state update equation (which we've already seen) is

$$\widehat{x}^{+}(k) = \widehat{x}^{-}(k) + K_k \left[z(k) - H_k \widehat{x}^{-}(k) \right] \tag{8}$$

where z_k is the measurement and H_k is the measurment matrix at time k.

The Kalman Filter Solution: The Covariance Update Equation

- Likewise, the error covariance P_k is corrected using the gain:

$$P_k^+ = [I - K_k H_k] \, P_k^- \tag{9}$$

The Five Kalman Equations

(10) State Extrapolation: $\widehat{x}^{-}(k) = \Phi_k \widehat{x}^{+}(k-1)$

(11) Covariance Extrapolation: $P_k^{-} = \Phi_k P_{k-1}^{+} \Phi_k^{\top} + Q_k$

(12) Kalman Gain: $K_k = P_k^{-} H_k^{\top} \left[H_k P_k^{-} H_k^{\top} + R_k \right]^{-1}$

(13) State Update: $\widehat{x}^{+}(k) = \widehat{x}^{-}(k) + K_k \left[z(k) - H_k \widehat{x}^{-}(k) \right]$

(14) Covariance Update: $P_k^{+} = \left[I - K_k H_k \right] P_k^{-}$

Examples

- Scalar Example: Resistor Revisited
- Damped Harmonic Oscillator

Scalar Example: Resistor Revisited

- Number of states: 1
- System Dynamics: $x_k = x_{k-1}$
- Measurement Model: $z(k) = x(k) + v(k)$, where $v(k)$ is zero mean, Gaussian white noise with variance σ^2
- Thus, $\Phi_k = 1$, $H_k = 1$, $Q_k = 0$, $R_k = \sigma^2$

Scalar Example: Resistor Revisited

- State extrapolation and covariance extrapolation are simple:

$$\widehat{x}^{-}(k) = \widehat{x}^{+}(k-1)$$

$$P_k^{-} = P_{k-1}^{+}$$

  where P is a 1×1 matrix or scalar.

- The Kalman gain equation $K_k = P_k^{-} H_k^{\top} \left[H_k P_k^{-} H_k^{\top} + R_k \right]^{-1}$ becomes:

$$K_k = P_k^{-} \left(P_k^{-} + \sigma^2 \right)^{-1}$$

Scalar Example: Resistor Revisited

- State update equation $\widehat{x}^+(k) = \widehat{x}^-(k) + K_k \left[z(k) - H_k \widehat{x}^-(k) \right]$ becomes

$$\widehat{x}^+(k) = \widehat{x}^-(k) + \frac{P_k^-}{P_k^- + \sigma^2} \left(z(k) - \widehat{x}^-(k) \right)$$

- Covariance update equation $P_k^+ = [I - K_k H_k] P_k^-$ becomes

$$P_k^+ = \left(1 - \frac{P_k^-}{P_k^- + \sigma^2} \right) P_k^-$$

Damped Harmonic Oscillator

- Number of states = 2. System model:

$$\begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} = \exp\left(\begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix} \Delta t \right) \begin{bmatrix} x_1(k-1) \\ x_2(k-1) \end{bmatrix} + \begin{bmatrix} w_1(k) \\ w_2(k) \end{bmatrix}$$

- Thus, $\Phi_k = \exp\left(\begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix} \Delta t \right)$

- Here $\Delta t = t_k - t_{k-1}$

- Note that exp is a matrix exponential.

Damped Harmonic Oscillator

- Assume that the process noise w has the following statistics:

$$Q_k = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

- This means that there is unity variance in position uncertainty and unity variance in velocity uncertainty.

- It also means that the uncertainties in position and velocity are uncorrelated (i.e., no non-zero off-diagonal elements in Q_k.

Dampled Harmonic Oscillator

- Measurement model:

$$z(k) = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} + v(k)$$

  where the measurement noise $v(k)$ has variance σ^2

- Thus $H_k = \begin{bmatrix} 1 & 0 \end{bmatrix}$

- Note that we only have a position measurement.

Damped Harmonic Oscillator

- Φ_k can be approximated through series expansion of

$$\exp \begin{bmatrix} 0 & \Delta t \\ -k/m\Delta t & -c/m\Delta t \end{bmatrix}$$

- Or we can use Matlab's expm function for specific values of k,c,m, and Δt.

Damped Harmonic Oscillator

- Because Φ and Q are matrices, the state and covariance extrapolation equations do not simplify from their general matrix form.

- However, the gain equation $K_k = P_k^- H_k^\top \left[H_k P_k^- H_k^\top + R_k \right]^{-1}$ looks like this:

$$K_k = \begin{bmatrix} p_{11}^-(k) \\ p_{21}^-(k) \end{bmatrix} \left(\begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} p_{11}^-(k) \\ p_{21}^-(k) \end{bmatrix} + \sigma^2 \right)^{-1} = \begin{bmatrix} \dfrac{p_{11}^-(k)}{p_{11}^-(k) + \sigma^2} \\ \dfrac{p_{21}-(k)}{p_{11}^-(k) + \sigma^2} \end{bmatrix}$$

  where $p_{11}^-(k)$ and $p_{21}^-(k)$ are the (1,1) and (2,1) elements of P_k^-.

Damped Harmonic Oscillator

- State and covariance update equations left as an exercise!

Exercises 1. Crank through the resistor example using $\widehat{x}_0 = 100\Omega$, $\sigma = 0.1$, and $P_0 = 0$. Start the recursive loop with the gain equation first. Run through the loop three times.

2. Draw a rough plot of the error covariance, plotting both the extrapolated and updated covariance for any give k at the same time. (This will yield the classic Kalman sawtooth pattern.)

3. Similarly, draw a rough plot of the state estimates.

4. Write out the state update equation for the damped harmonic oscillator, using the Kalman gain derived in the example.

5. Write out the covariance update equation for the damped harmonic oscillator, using the Kalman gain dervied in the example.

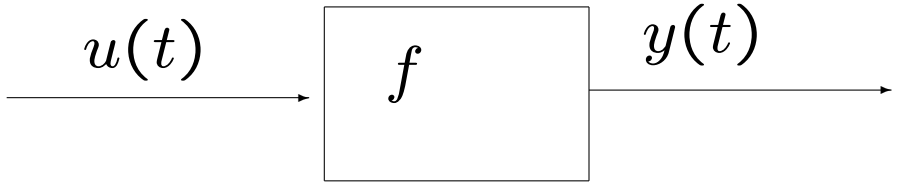
Segment 3
Differential Equations and Difference Equations
Topics

- Input-Output Representation

- Differential Equations and Difference Equations

Input-Output Representation

- Black box approach: $y(t) = f(u(t), t)$



- Output represents the resultant behavior of the dynamic system as a function of the input (and possibly time).

- Output is often one primary scalar variable of interest like position.

Input-Output Representation

- Note that output is not necessarily the same thing as the observation or measurement.

- Observation or measurement is often related directly to the output through a matrix coupling.

- In state space representation, the output is often eliminated in favor of the state vector, and the measurement is given as a function of the state vector directly.

Input-Output Representation

- The derivatives of position and how they relate to the forcing function yields a single differential equation.

- We'll see later how an nth-order, scalar differential equation is related to a vector state space model.

Differential Equations and Difference Equations

- Ordinary Differential Operators
- Ordinary Differential Equations
- Difference Equations

Ordinary Differential Operators: Vectors and Functions

- Vectors are functions and functions are vectors
- A vector $\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$ in 3-space is really a function $x : \{1, 2, 3\} \to \mathbb{R}$
- We just write them differently than functions, i.e., instead of $x(i)$ we write x_i.

Ordinary Differential Operators: Vectors and Functions

- And since the domain $\{1, 2, 3\}$ of the function x is so small, we can exhibit all values of x explicitly in one fell swoop, i.e., as $\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$

- But, by the same token, ordinary real functions $f : \mathbb{R} \to \mathbb{R}$ are vectors

- They just have so many components that we can't exhibit them all at once

Ordinary Differential Operators: Vectors and Functions

- We can add vectors by adding their components: $z = x + y$ if and only if $z_i = x_i + y_i$ for each index i

- By the same token, we can add functions componentwise: $h = f + g$ if and only if $h(t) = f(t) + g(t)$ for all t in the domain

- Componentwise multiplication of functions can also be defined (although this does not come up as often for ordinary vectors): $h = f \cdot g$ if and only if $h(t) = f(t) \cdot g(t)$ for all t in the domain

Ordinary Differential Operators: Vectors and Functions

- Finally, we can multiply functions by scalars, just like we do for vectors: $g = a \cdot f$ if and only if $g(t) = a \cdot f(t)$ for all $t \in \mathbb{R}$.

- The upshot of all this is that the set of all real functions is a vector space!

- But we are really more interested in a subspace of this vector space, i.e., we really want to restrict our attention to the set of all **differentiable** functions on some closed interval.

Ordinary Differential Operators: Differentiable Functions

- Let $I = [a, b]$ be any closed interval of the real line
- Consider the set $\mathbb{R}^I = \{f : I \to \mathbb{R}\}$ of all functions mapping $I = [a, b]$ into the real numbers
- Then, as we have just pointed out, $\mathbb{R}^I$ is a vector space over the reals

Ordinary Differential Operators

- We define the **derivative** of a function f at the point $t \in [a, b]$ as the limit as $s \to t$ of the difference quotient,

$$\frac{f(s) - f(t)}{s - t},$$

  assuming this limit exists.

- Note that not all functions have derivatives!

- Those that do have a derivative defined at every point of I we call **differentiable** on I.

Ordinary Differential Operators

- This operation turns out to be a linear operator on the vector space of functions that are differentiable on $I = [a, b]$.

- In fact, we want to consider those functions that can be differentiated as many times as we like

- Thus, we consider the space of infinitely differentiable, real-valued functions defined on the closed real interval $[a, b]$: $C^\infty[a, b]$

Ordinary Differential Operators

- The operation of taking the derivative will enable us to develop deterministic differential equations.

- Later we will define derivatives of stochastic processes in order to introduce and solve stochastic differential equations. You cannot always take the derivative (or even integrate) a random process – and sometimes not even the sample functions of a random process – in the ordinary sense.

Ordinary Differential Operators

- Since the derivatives of all orders are assumed to exist for all functions $f \in C^\infty[a,b]$, the limit of the difference quotient always exists for such functions.

- Thus, we assign to each f a new function Df given by

$$[Df](t) = \lim_{s \to t} \frac{f(s) - f(t)}{s - t} \tag{15}$$

- This assignment defines a transformation

$$D : C^\infty[a,b] \to C^\infty[a,b]$$

that is a linear operator.

Ordinary Differential Operators

- D is linear: $D[af + bg] = a[Df] + b[Dg]$

- Sometimes we get lax and simply write $Df(t)$ instead of $[Df](t)$. However, keep in mind that $Df(t)$ says "Differentiate f and then evaluate the resulting function at the point t."

- D obeys all the usual rules of differential calculus such as the product rule and the chain rule:

$$D(fg) = [Df]g + f[Dg]$$

$$D(f \circ g) = [Df] \circ g \cdot Dg$$

Ordinary Differential Operators: Examples

- If id is the identity function, i.e., $\mathrm{id}(t) = t$, then $D\mathrm{id} = 1$. (You probably know this as $\frac{dt}{dt} = 1$; Note that 1 is the function whose value at every t is 1.)

- Using the product rule, we have, for example,

$$\begin{aligned} D\mathrm{id}^2 &= D[\mathrm{id} \cdot \mathrm{id}] \\ &= D\mathrm{id} \cdot \mathrm{id} + \mathrm{id} \cdot D\mathrm{id} \\ &= 1 \cdot \mathrm{id} + \mathrm{id} \cdot 1 \\ &= \mathrm{id} + \mathrm{id} \\ &= 2\mathrm{id} \end{aligned}$$

  (In more familiar notation this is just $\frac{dt^2}{dt} = 2t$.)

Ordinary Differential Operators

- Note that we sometimes write d/dt or $\frac{d}{dt}$ for the operator D when we want to emphasize the independent variable t. Or, instead of $\frac{dy}{dt}$ we write simply $\dot{y}$.

- However, that is really unnecessary, adds no new information, and is just extra baggage.

- Nevertheless, we shall do it sometimes because it may be more familiar.

Ordinary Differential Operators

- Since there is really only one generating operator D that we are interested in here, we are less interested in multiplication than we are in function composition or iteration of this operator. For instance, $D^2 f = [D \circ D](f) = D(Df)$.

- By induction we have $D^n = \underbrace{D \circ \cdots \circ D}_{n}$.

- Just as real-valued functions inherit algebraic operations from those of their range spaces, so too do differential operators:

$$[aD^m + bD^n](f) = a(D^m f) + b(D^n f)$$

Ordinary Differential Operators

- We can therefore generate polynomials in D:

$$p(D) = a_n D^n + \cdots + a_1 D + a_o I,$$

  where I is the identity operator on $C^\infty[a, b]$. We usually just drop the identity operator, since it amounts to multiplying by 1.

- Such polynomials in D are still linear operators.

- Thus, we have rediscovered the group ring $\mathbb{R}$(Diff), where Diff in this case is the infinite cyclic group generated by the differential operator D. We'll call this group ring the **polynomial ring of differential operators**.

Ordinary Differential Equations

- With polynomials of differential operators we can now define ordinary differential equations.

- An equation of the form

$$p(D)f = q(D)g$$

where $f, g \in C^\infty[a, b]$ and $p(D), q(D) \in \mathbb{R}(\text{Diff})$, is called an ordinary differential equation with constant coefficients.

Ordinary Differential Equations

- It is probably more familiar if we use y instead of f and u instead of g, and if we show the independent variable, say t, explicitly:

$$a_n \frac{d^n y}{dt^n} + \cdots + a_1 \frac{dy}{dt} + a_0 y = b_m \frac{d^m u}{dt^m} + \cdots + b_1 \frac{du}{dt} + b_0 u$$

or

$$\sum_{i=0}^{n} a_i \frac{d^i y}{dt^i} = \sum_{i=0}^{m} b_i \frac{d^i u}{dt^i} \tag{16}$$

- For our purposes, the function $u = u(t)$ is called the **input**, while the function $y = y(t)$ is called the **output**. The independent variable t is thought of as time.

Ordinary Differential Equations

- In (16), u is ordinarily assumed to be known or given, while y is unknown and sought.

- Any function y satisfying (16) is called a solution of the differential equation.

- A differential equation such as (16) is called **homogeneous** when the input function u is identically zero for all $t \in [a, b]$.

Ordinary Differential Equations

- Thus far we have only dealt with linear differential equations with constant coefficients.

- We could allow the coefficients to be time-dependent functions. However, for most of our applications, this will not be necessary.

Ordinary Differential Equations: Examples

- From physics we have Hooke's law: the restoring force of a spring is proportional to the distance stretched: $-ky = m\frac{d^2y}{dt^2}$.

- This is just an application of Newton's $F = ma$, where a is the acceleration: $a = d^2y/dt^2$.

- Rewriting this in the form of (16) we have

$$m\frac{d^2y}{dt^2} + ky = 0$$

- Therefore, this is a homogeneous equation of order 2.

- We have used y rather than x as in most physics books, because we want to be consistent with (16).

- It is common to rewrite such equations so that the leading coefficient is 1. Thus we have:

$$\frac{d^2y}{dt^2} + \frac{k}{m}y = 0.$$

- It is easily verified that $\sin \omega_0 t$, where $\omega_0 = \sqrt{\frac{k}{m}}$, satisfies this differential equation.

- Equation (16)

$$\sum_{i=0}^{n} a_i \frac{d^i y}{dt^i} = \sum_{i=0}^{m} b_i \frac{d^i u}{dt^i}$$

  is called **time-invariant**, because the coefficients are constant and also because the independent variable t does not appear explicitly in the equation.

- Note that (16) is linear in terms of both the input and the output, because the polynomial differential operator on each side of the equation is a linear operator.

- This double linearity means, on the one hand, that the output y due to several inputs acting at the same time is equal to the sum of the outputs due to each input acting alone.

- On the other hand, it also means that if y_1 and y_2 are both solutions to the homogeneous version of (16), then so is $c_1 y_1 + c_2 y_2$ where c_1 and c_2 are any real constants.

- A maximal set of n such solutions to the homogeneous equation that are also linearly independent (as vectors) is called a **fundamental set**.

- You can find a fundamental set by solving the **characteristic equation** associated with the differential equation. This equation is nothing other than the original polynomial equation in D, but with D replaced by an unknown such as z.

- Thus, the characteristic equation for (16) is

$$a_n z^n + \cdots + a_1 z + a_0 = 0$$

- We use z because the roots of this equation are in general complex (thanks, once again, to Dr. Gauss).

- In solving the characteristic equation, two cases emerge:

  1. The roots of the characteristic equation are all distinct.

  2. Some roots are repeated, i.e., each root r_i has multiplicity n_i.

- In case 1, a fundamental set is easily constructed:

$$y_1(t) = \exp(r_1 t), \ldots, y_n(t) = \exp(r_n t)$$

where the r_i are the distinct roots.

- In case 2, each root r_i of multiplicity n_i contributes n_i functions to the fundamental set:

$$y_{i1}(t) = \exp(r_i t), y_{i2}(t) = t\exp(r_i t), \ldots, y_{in_i}(t) = t^{n_i - 1}\exp(r_i t)$$

Ordinary Differential Equations

- To fully solve the differential equation, the initial conditions need to be taken into account.

- These are the values of the function and its first $n-1$ derivatives at the start of the time interval.

- We will specialize our interval $[a, b]$ to the case in which $a = 0$ and $b = T$, some unspecified time in the future.

- The **free response** is the solution to the homogeneous equation in which the initial conditions are specified but the input is zero, i.e., there is no forcing function.:

(17) $$y(0), Dy(0), \ldots, D^{n-1}y(0)$$

- The **forced response** reverses this situation; it allows the input function to be non-zero, but it requires all the initial conditions in (17) to be zero.

- The **total response** is the sum of the free and forced responses.

Ordinary Differential Equations: Example of Free, Forced and Total Responses

- Consider $D^2y + 3Dy - 1y = u$ with initial conditions: $y(0) = 1, Dy(0) = -1$.

- The roots (found using Matlab's roots function) are $r_1 = -3.3028$ and $r_2 = 0.3028$, and they are distinct.

- Therefore, a fundamental set is $\{\exp(-3.3028t), \exp(0.3028t)\}$.

Ordinary Differential Equations: Example of Free, Forced and Total Responses

- With the fundamental set and the initial conditions we can form a set of n linear equations in n unknowns (here, $n = 2$):

$$c_1 y_1(0) + c_2 y_2(0) = y(0)$$
$$c_1 Dy_1(0) + c_2 Dy_2(0) = Dy(0)$$

or

$$1 \cdot c_1 + 1 \cdot c_2 = 0$$
$$-3.3028 \cdot c_1 + 0.3028 \cdot c_2 = -1$$

- The previous linear system is of the form $Ax = b$. Using Matlab's matrix division operator \, the solution is $x = A\backslash b$ or

$$c_1 = 0.2773$$
$$c_2 = -0.2773$$

- The total response can also be decomposed into transient and steady state responses

- The **transient** response is that part of the total response that decays to 0 as $t \to \infty$.

- The **steady state response** is the difference between the total response and the transient response.

Linear Time-Invariant Differential Equations

- Linear Time-Invariant (LTI) Differential Equations
  - Linear: Polynomial differential operators are linear:

$$p(D)[af + bg] = ap(D)f + bp(D)g$$

  - Time-Invariant means the coefficients are constant.
  - Homogeneous solution of corresponding state equation is a matrix exponential. (More on that later when we discuss the state transition matrix.)

Ordinary Difference Equations

- Auto-Regressive Moving Average (ARMA)

- n^{th} derivative corresponds to n-step advance:

$$\sum_{i=0}^{n} a_{n-i} y(k+n-i) = \sum_{i=0}^{m} b_{m-i} u(k+m-i) \tag{18}$$

Ordinary Difference Equations

- The left side of Eq. (18) is the Auto-Regressive part
- The right side of Eq. (18) is the Moving Average part
- Hence, ARMA

Ordinary Difference Equations: Example

- Let $n = 3$, $m = 2$ and consider the difference equation:

$$\begin{aligned} -3y(k+3) + 1.5y(k+2) & \\ +7y(k+1) - 3.2y(k) & \\ = 23u(k+2) - 5u(k+1) + 0.5u(k) & \end{aligned}$$

Segment 4
State Space Dynamics
Topics

- Differential Equations and Dynamic Systems
- Block Diagrams and State Space
- State Transition Matrix

Dynamic Systems

- The time-dependent behavior of physical systems can often be modeled by differential equations.

- Such systems are often called **dynamic systems** or **dynamical systems**.

Dynamic Systems

- Considering Newton's law $F = ma$ once again, we see that the force F is the resultant force acting on the mass m.

- However, the resultant force is simply the vector sum of all the forces, both those internal to the system as well as the input forces that are external.

- The restoring force, $-ky$ in the simple oscillator problem, is a force internal to the system. So we don't consider it an input *per se*.

Dynamic Systems

- The response or output of the system is motion, more specifically, accelerated motion given by a.

- Thus, the simple harmonic oscillator is a dynamic system, one in which there is only an internal force determining system behavior: $F = -ky$.

- An external forcing function may also be present, e.g.:

$$\frac{d^2y}{dt^2} + \frac{k}{m}y = \sin \omega t$$

- The most general second-order equation for a mechanical system of this sort according to Newton's law is:

(19) $$\frac{d^2y}{dt^2} = \frac{1}{m}F(y, \dot{y}, t)$$

- If we can separate the forcing function $F(y, \dot{y}, t)$ into a sum of two functions $F_1(y, \dot{y})$ and $F_2(t)$, then we can rearrange this equation into the input-output form of equation (16):

$$\frac{d^2y}{dt^2} - \frac{1}{m}F_1(y, \dot{y}) = \frac{1}{m}F_2(t)$$

Dynamic Systems

- In engineering applications, we are often interested not only in modeling the dynamics of the system, but also in measuring the output or behavior of the system.

- But the output is not always directly observable.

- Thus, in specifying dynamic systems, we often add a second equation, usually algebraic or functional in nature, that relates the output y (and possibly the input u) to some measured or observed quantities.

$$z = h(y) + g(u)$$

The Concept of State

- To determine the behavior of a deterministic system at any future time requires the knowledge of two things:

  1. The equations governing the dynamics of the system

  2. The initial conditions

- But the initial conditions for an nth-order continuous dynamic system are simply the values of the first $n-1$ derivatives at the initial time.

The Concept of State

- At any point in time, you can think of the instantaneous values of the first $n-1$ derivatives as the initial conditions for all future dynamic evolution.

- These values capture the "state" of the dynamic system at that time.

The Concept of State

- Another way of seeing what "state" means is by considering the Taylor series expansion of a function.

- Taylor's theorem says that the value of a function $y(t)$ (if it is sufficiently well-behaved) at any point t close to some nominal point t_0 can be determined from the value of all of its derivatives at the nominal point t_0.

$$y(t) = y(t_0) + \dot{y}(t_0)(t - t_0) + \frac{1}{2!}\ddot{y}(t_0)(t - t_0)^2 + \cdots$$

The Concept of State

- In the language dynamic systems, the quantity $t - t_0$ is simply the elapsed time since the initial time t_0.

- Thus, if you have knowledge of the state at time t_0, i.e., if you know the values of all the (infinitely many) derivatives, then you can determine the output at any later time (sufficiently close to t_0).

- This is because all of the dynamic information is contained in the derivatives.

Dynamic Systems: The Concept of State

- In keeping with the linear spirit of things, we will restrict ourselves for now to linear functions h.

- Also, we will now introduce the state space concept and define z as a function of the state.

- Intuitively, the concept of state involves in some sense the instantaneous internal arrangement of a system. In practical situations, the state variables may represent attributes of the system that we wish to know something about.

Dynamic Systems: The Concept of State

- Mechanical systems like a moving particle have three simultaneous attributes: position, velocity, and acceleration.

- Newton tells us that acceleration is often a function of position, velocity and perhaps time.

- However, as we just saw, a force that acts on a system as a function of time only should be considered an external forcing function, i.e., an input to the system.

- Thus we are led to think of position and velocity as the primary state variables of interest.

State Space Representation

- A general n-th order linear differential equation can be expressed as a system of first-order differential equations.

- Ignoring the input u for the moment, consider:

$$p(D)y = D^n y + a_{n-1}D^{n-1}y + \cdots + a_1 Dy + a_0 y,$$

where we have put the polynomial in normal form, i.e., such that its leading coefficient is 1.

State Space Representation

- If we consider y and its first $n-1$ derivatives as the states of this system, then we have n first order equations. Setting $x_1 = y$, we write

$$\begin{aligned} x_1(t) &= y(t) \\ x_2(t) &= Dy(t) \\ &\cdots \\ x_n(t) &= D^{n-1}y(t). \end{aligned}$$

- The functions $x_i(t)$ are called the **state variables**.

State Space Representation

- Taking the derivative of these n state variables and making use of the original differential equation yields:

$$\begin{aligned} Dx_1(t) &= x_2(t) \\ Dx_2(t) &= x_3(t) \\ &\cdots \\ Dx_n(t) &= -a_0 x_1(t) - a_1 x_2(t) - \cdots - a_{n-1} x_n(t). \end{aligned}$$

- These n equations can be written more compactly in vector matrix form: $Dx(t) = Ax(t)$ or $\dot{x}(t) = Ax(t)$.

State Space Representation

- In the equation $\dot{x}(t) = Ax(t)$, we have:

$$x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \\ \vdots \\ x_n(t) \end{bmatrix}$$

and

$$A = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & 0 & 0 & 0 & 1 \\ -a_0 & -a_1 & \cdots & -a_{n-2} & -a_{n-1} \end{bmatrix},$$

the **companion matrix** of the differential equation.

State Space Representation

- If there is an input function u and it is present via its derivatives as in equation (16), then we can write:

  (20) $$\dot{x} = Ax + Bu$$

  forming B in much the same way we formed A, and suppressing the time variable t.

- Equation (20) expresses the system's **state dynamics**.

- Later we will include white noise as an input. However, that will make the differential equations stochastic, and we are not yet ready for that.

- To obtain the original output y as a function of the internal state of the system, we require one additional vector-matrix equation:

  (21) $$y = Cx + Du$$

  where the matrix D represents a **direct feedthrough** of the input to the output.

- The output equation is derived directly from the b_i coefficients in the original matrix differential equation.

State Space Representation

- Kalman filter theory generally assumes $D = 0$ in the output equation and uses a measurement model instead:

  (22) $$z = Hx$$

- Note that H implicitly includes C as a factor.

- Note that we have not placed any restrictions on the size of the measurement column vector z. As long as H has n columns it can have as many rows as desired.

State Space Representation: Example

- Let's consider once again the simple harmonic oscillator:

$$m\frac{d^2y}{dt^2} + ky = 0$$

and convert this to state space form:

$$x_1 = y$$
$$x_2 = Dy$$

and $\dot{x}_2 = -\frac{k}{m}x_1 + 0 \cdot x_2$.

- Therefore, in matrix and vector form:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -k/m & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

- For a measurement model, we could consider a camera with a strobe flash that periodically records the instantaneous position of the sliding mass.

- In this case, only the position would be observable and we would write the measurement or observation model as:

$$z = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

(Remember that we are ignoring both process and measurement noise.)

State Space Representation: Matlab / Simulink Example

- Four parameters A, B, C, D from Matlab workspace
- Measurement model added external to Simulink's state space block

State Space Representation

- State space representation not derived only from single n^{th} order differential equation

- Sometimes state coupling comes more directly through internal dynamic coupling. E.g.,
  - coupled harmonic oscillators
  - complicated RLC networks

Ordinary Difference Equations

- For difference equations, a state space model can be derived using the companion matrix exactly as for differential equations

- Matlab / Simulink has a subsystem block in its Discrete block library in which the four matrices for an input-output model can be specified in the workspace.

State Space Representation: Summary

- The state variables are often provided by the 0th through $n-1^{\text{st}}$ derivatives of a single n^{th} plus internal dynamic coupling

- Equivalently, they are the outputs of the n integrators of the system.

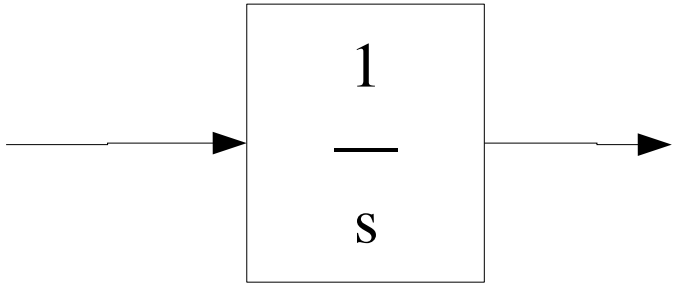
- They represent implicit or explicit internal coupling of the system.

- Similarity transformation of state vector yields equivalent representation. Changing coordinate systems.

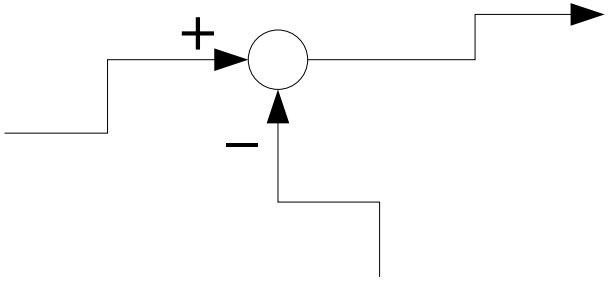
Block Diagrams
depict ...

- System signal flow
  - summation and takeoff points
- Various transformation blocks
  - integrators (continuous) and delayors (discrete)
  - differentiators
  - scalors or gain multipliers

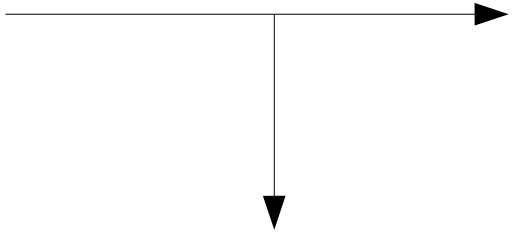
Integration Block



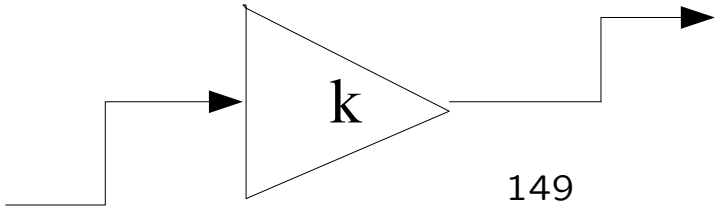
Summation Point

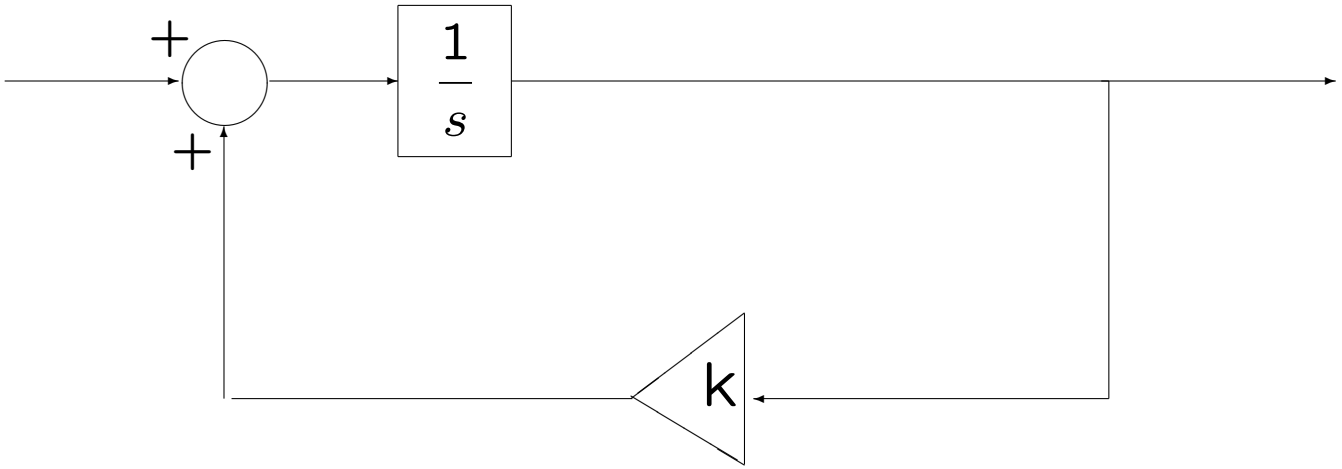


Takeoff Point



Scalor or Gain Multiplier




+
+
1
s
k

Deriving States from Diff. Eqs. and Block Diagrams

- Continuous states can be identified as outputs of integrators
- Discrete states can be identified either as sampled continous states or as outputs of delayors

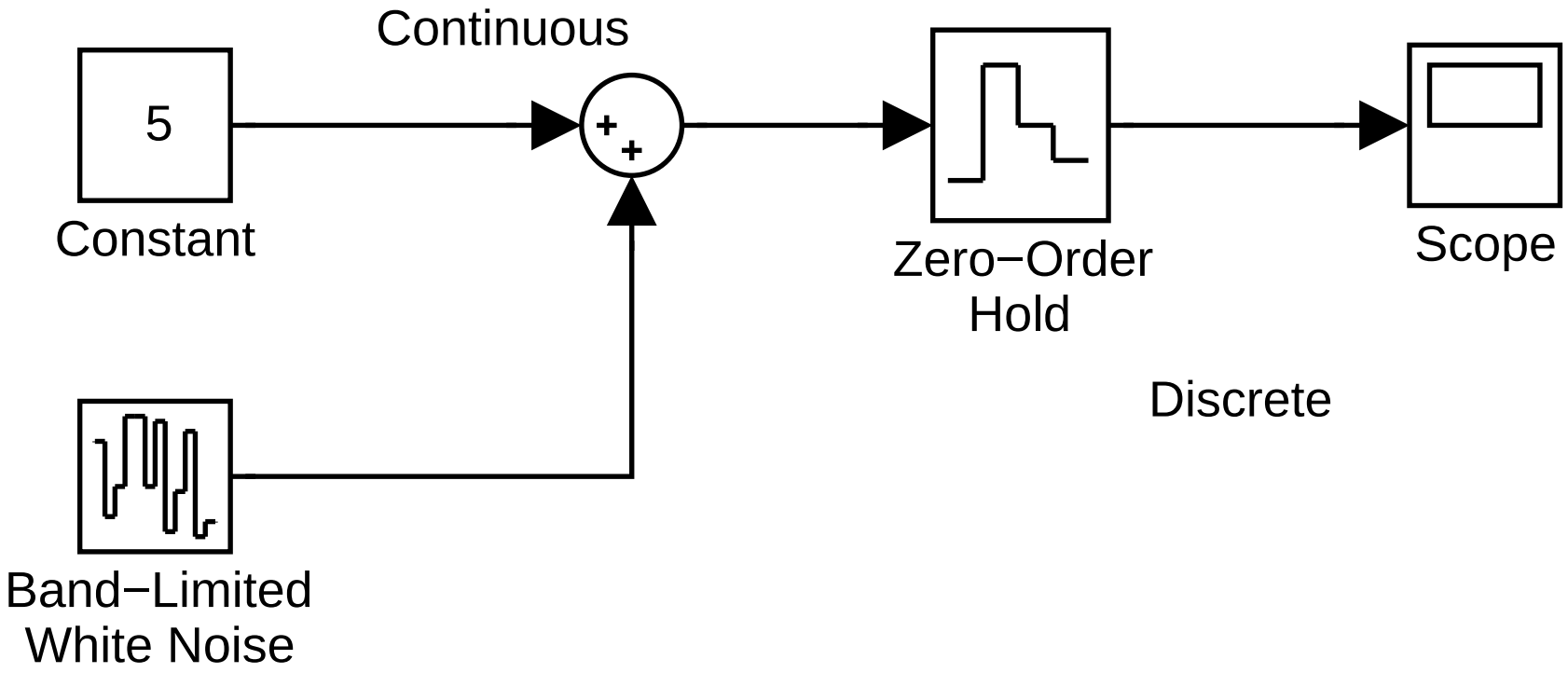
Examples

- Scalar Constant Dynamics
- Scalar Constant Dynamics with Recursive Estimator
- Scalar Ramp Dynamics with Recursive Estimator
- Damped Harmonic Oscillator

Scalar Constant Dynamics

- Matlab Model
- Very Simple
- Additive Noise
- No feedback loops

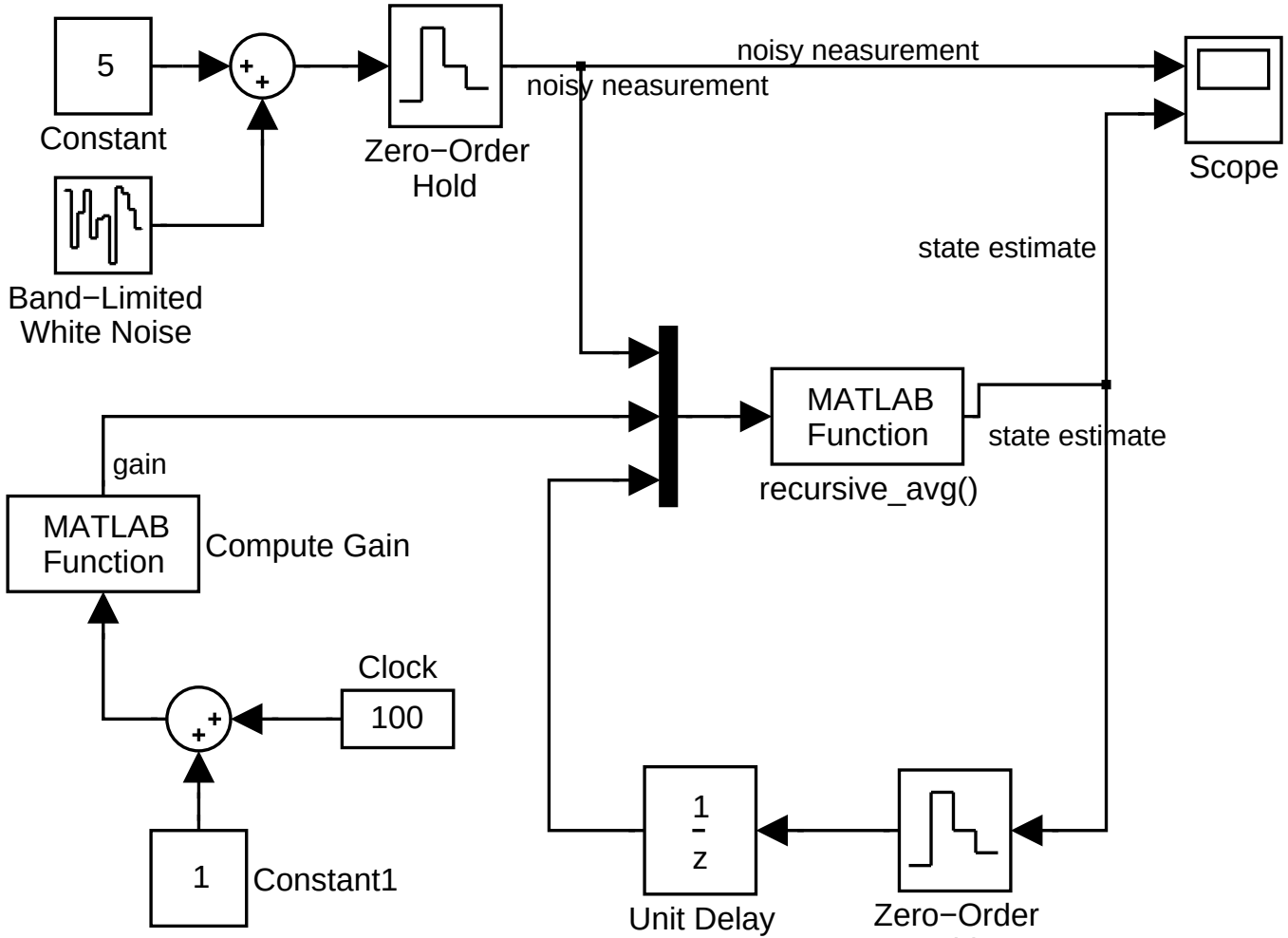
Scalar, Constant Dynamics plus White Process Noise



Scalar Constant Dynamics with Recursive Estimator

- Matlab Model
- Very Simple Dynamics
- Additive Noise
- Estimation loop implementing recursive simple averaging
- Simple open loop gain input

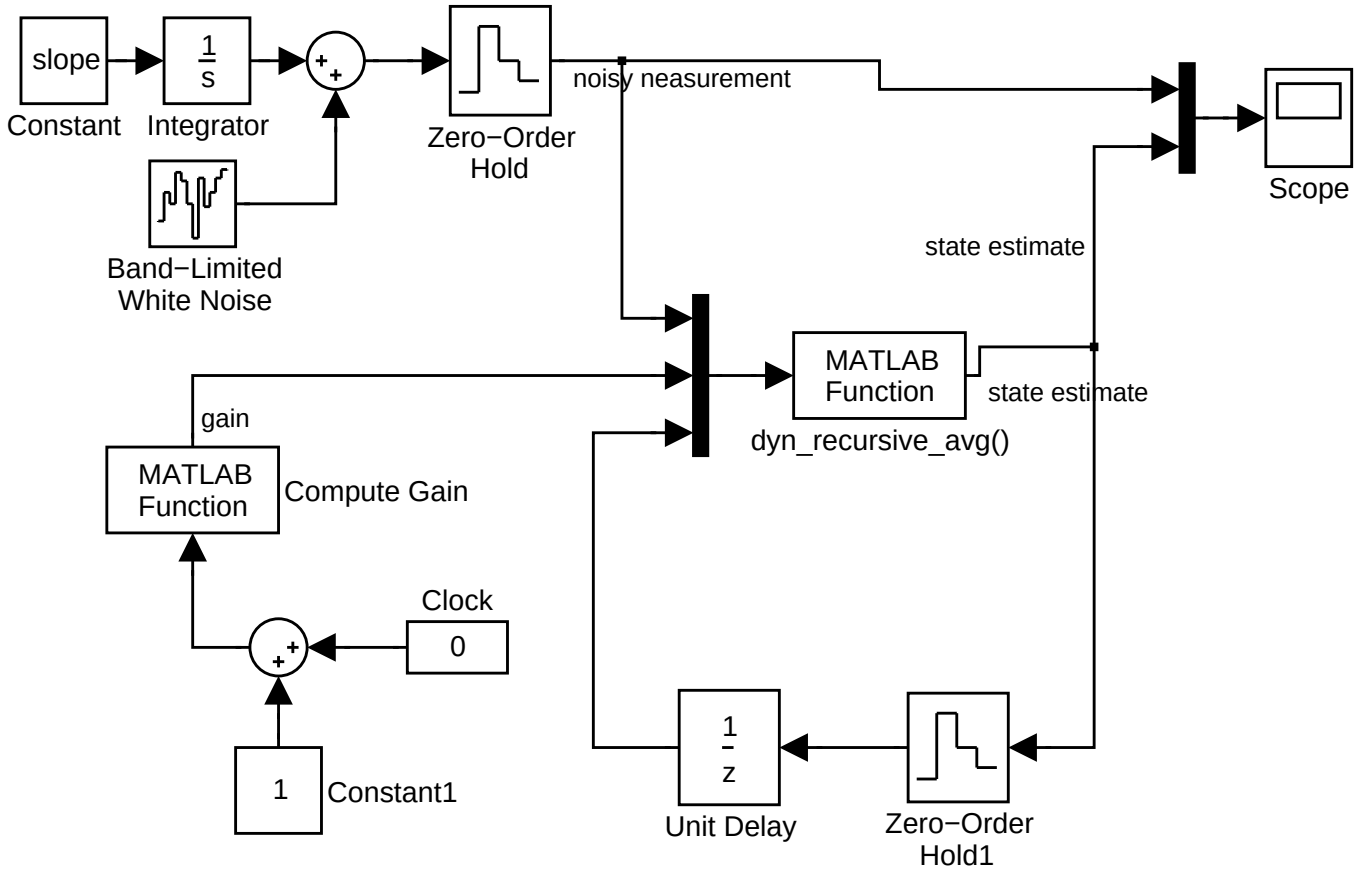
Scalar, Constant Dynamics plus White Process Noise with Estimation



Scalar Ramp Dynamics with Recursive Estimator

- Matlab Model
- Scalar Dynamics with known ramp
- Additive Noise
- Estimation loop implementing recursive simple averaging
- Simple open loop gain input

Scalar, Ramp Dynamics
plus White Process Noise
with Dynamic Average Estimation

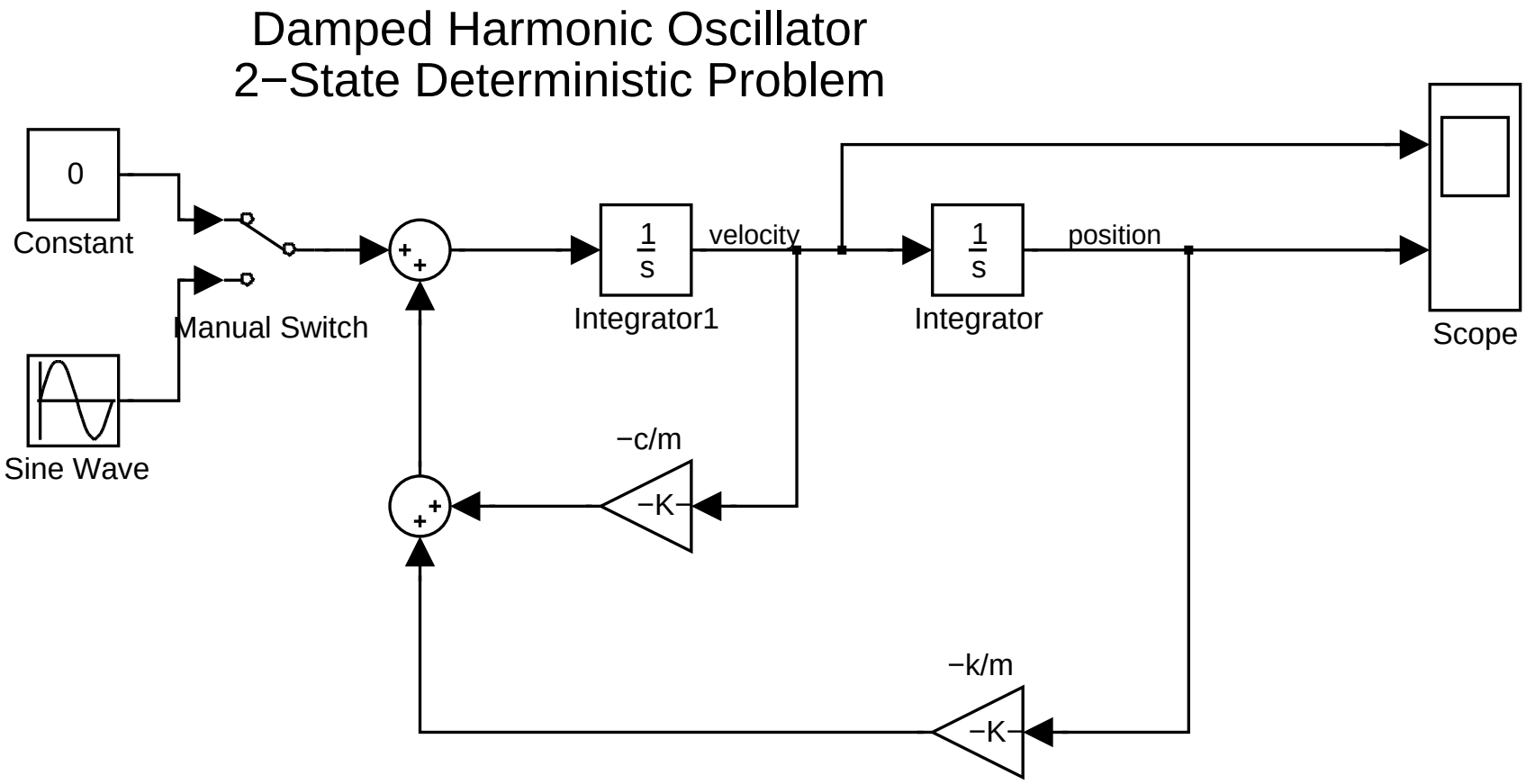


Observe how the sampling rate affects the ability of the filter to follow the input.

Vector Dynamics: Damped Harmonic Oscillator

- Matlab Model
- 2-dimensional Vector Dynamics
- Deterministic, Continuous Model

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$


Damped Harmonic Oscillator
2−State Deterministic Problem
0
Constant
Manual Switch
Sine Wave
1
s
Integrator1
velocity
1
s
Integrator
position
Scope
−c/m
−K
−k/m
−K

Vector Dynamics: Damped Harmonic Oscillator

- Discrete model would look like this

$$\begin{bmatrix} x_1(k) \\ x_2(k) \end{bmatrix} = \exp\left(\begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix} \Delta t \right) \begin{bmatrix} x_1(k-1) \\ x_2(k-1) \end{bmatrix}$$

- Here $\Delta t = t_k - t_{k-1}$

- Note that exp is a matrix exponential.

- In the continuous case, the state transition matrix is a matrix exponential whenever the system dynamics matrix is a constant (i.e., time invariant) matrix.

State Transition Matrix

- $\Phi_k = \Phi(k, k-1)$
- Solution of homogeneous equation for continuous systems
- If discrete system is derived from continuous system, then Φ_k satisfies:

$$x_k = \Phi_k x_{k-1}$$

Exercises

1. Convert the differential equation

$$5\frac{d^3y}{dt^3} - 0.3\frac{dy}{dt} + 7y = u(t)$$

   into a state space system model

2. Convert the difference equation

$$100y(k+9) = \sum_{i=0}^{2} b_i u(k+2-i)$$

3. Write the output equations for the above systems as well!

Segment 5
Probability and Statistics
Topics

- Sample Space and Probability Measures
- Probability Distributions and Densities
- Random Variables and Expectation
- Moments: Means and Variances

Probability: Sample Spaces

- Experiments

- Sample Space: Ω

  – consists of all possible outcomes ω.

- Events: Subsets of Ω forming a σ-algebra. Not necessarily *all* subsets.

Examples: Experiment and Sample Space

- 1. Experiment: Single coin toss
  - Sample Space: $\Omega = \{H, T\}$
- 2. Experiment: Toss coin twice
  - Sample Space: $\Omega = \{HH, HT, TH, TT\}$

Probability: σ-Algebras

- In probability, we are often less concerned with individual outcomes and more with sets of outcomes.

- Sets of outcomes, i.e., subsets of the sample space Ω, are called "events".

- Not every subset of the sample space Ω qualifies as an event.

- Qualified events form an "event algebra", known in mathematics as a σ-**algebra**

Probability: σ-Algebras

- A σ-algebra $\mathcal{M}$ is a collection of subsets of Ω that is closed under arbitrary unions and intersections, set complementation, and which also contains the whole space Ω as well.

- A σ-algebra can be expressed in structural notation as $< \mathcal{M}, \cup, \cap, ', \emptyset, \Omega >$

- Ω is the certain event.

Examples: σ-algebras

- Double coin toss:
  - Sample Space: $\Omega = \{HH, HT, TH, TT\}$
  - σ-algebra: $\mathcal{M} = \{\emptyset, \{HH\}, \{HT, TH, TT\}, \Omega\}$

Mutually Exclusive Events

- Events $A, B \in \mathcal{M}$ are called **mutually exclusive**, if they are disjoint as sets, i.e., if $A \cap B = \emptyset$.

- In last example, the events $\{HH\}$ and $\{HT, TH, TT\}$ are mutually exclusive.

Set Functions and Measures

- A measure is a way of assigning a number to a set.
- Intuitively, it measures the size of the set.
- In probability, a measure is a way of assigning a probability to an event.

Set Functions and Measures

- A set function μ on $\mathcal{M}$ assigns a real number $\mu(A)$ to each set $A \in \mathcal{M}$.

- A set function μ on $\mathcal{M}$ is said to be **additive**, if $\mu(A \cup B) = \mu(A) + \mu(B)$ whenever A and B are disjoint.

- μ is called **countably additive** if $\mu(\bigcup_{i \in I} A_i) = \sum_{i \in I} \mu(A_i)$, if the family of sets $\{A_i : i \in I\}$ is disjoint.

Set Functions and Measures

- A measure μ on a σ-algebra $\mathcal{M}$ is a real-valued, non-negative, countably additive set function: $\mu : \mathcal{M} \to \mathbb{R}$

- Measures are used in mathematics to define abstract integration.

- Example: Counting Measure. Ω is finite and the measure of each subset is merely the number of elements in the set.

- Example: Dirac Measure. Let $\mathcal{M}$ be any σ-algebra and let $\omega \in \Omega$ be an arbitrary but fixed element of Ω. For each $A \in \mathcal{M}$ define

$$\mu_\omega(A) = \begin{cases} 1 & \text{if } \omega \in A, \\ 0 & \text{otherwise.} \end{cases}$$

Probability Measures

- A measure μ on $\mathcal{M}$ with the property that $\mu(\Omega) = 1$ is called a **probability measure** or **probability function**.

- Instead of μ we will use P, which is more customary.

Probability: Specialized Terminology

- Null Event: $P(A) = 0$. It is important to realize this does not imply that $A = \emptyset$. In measure theory this would be called a set of measure zero.

- Independent Events: $P(A \cap B) = P(A)P(B)$

- Mutually Exclusive Events:
  $A \cap B = \emptyset \Rightarrow P(A \cup B) = P(A) + P(B)$

Examples: Probability Measure

- Double coin toss:
  - Sample Space: $\Omega = \{HH, HT, TH, TT\}$
  - σ-algebra: $\mathcal{M} = \{\emptyset, \{HH\}, \{HT, TH, TT\}, \Omega\}$
  - Probability Measure: $P(A)$ =number of elements in A divided by total number of elements in Ω
  - E.g., $P(\{HH\}) = 1/4$

Examples: Probability Measure

- Let Ω be any finite set of cardinality n. Let the σ-algebra be the set of all subsets of Ω and let $|\cdot|$ denote the counting measure, i.e. for $A \subseteq \Omega$, $|A|$ =number of elements in A. Define a probability measure by $P = \frac{1}{n}|\cdot|$.

- To illustrate the last example a little more, let $\Omega = \{\omega_1, \ldots, \omega_6\}$, and let $A = \{\omega_1, \omega_2\}$. Then $P(A) = \frac{1}{6} \cdot |A| = \frac{1}{6} \cdot 2 = \frac{1}{3}$

Examples: Probability Measure

- Let $\Omega = [0, 1]$ and let $\mathcal{B}$ be the Borel σ-algebra on the interval $[0, 1]$. Define $P = \lambda$, the Lebesgue measure on $\mathcal{B}$. Note that for any open subinterval (a, b) of $[a, b]$, we have $P(a, b) = b - a$.

Conditional Probability

- Suppose you have two events A and B and you know that the event B has in fact occured.

- How does that knowledge affect the probability that both A and B have occured?

- Intuitively, we would tend to think that the likelihood of $A \cap B$ increases with the knowledge that B has occured.

Conditional Probability

- The increased likelihood of A given that B has occured is called the **conditional probability** of A given B.

- If $P(B) \neq 0$, then the conditional probability of A, given that B has occurred, is: $P(A|B) := P(A \cap B)/P(B)$

- Sometimes we say the probability of A conditioned on B.

- Note that since $P(B) \leq 1$, $P(A \cap B)/P(B) \geq P(A \cap B)$

Conditional Probability: Bayes' Theorem

- Since $P(A|B) = P(A \cap B)/P(B)$ and $P(B|A) := P(B \cap A)/P(A)$, we can eliminate $P(A \cap B)$ and derive

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)}$$

- Using this we obtain Bayes' Theorem (simple form):

$$P(A|B) = \frac{P(A)P(B|A)}{P(A)P(B|A) + P(A')P(B|A')}. \tag{23}$$

Conditional Probability

- Bayes' Theorem (complex form): Let $A_1, \ldots, A_n$ be a partition of Ω. Then,

$$P(A_i|B) = \frac{P(A_i)P(B|A_i)}{\sum_{j=1}^{n} P(A_j)P(B|A_j)}. \tag{24}$$

- Bayes' Theorem expresses posterior probability in terms of prior probabilities and conditional probabilities.

Conditional Probability

- In one derivation of the Kalman filter, the use of Bayes' Theorem is a crucial step. We'll see this later.

- Bayes' Theorem is used to derive the Kalman gain: to determine the best estimate of state **after** measurement has occurred.

Conditional Probability

- Bayes' Theorem allows us to reverse the conditional probability.

- We can derive information about the state conditioned on the measurement from information about the measurement conditioned on the state.

Random Variables

- In measure-theoretic terms, a ra**ndom variable** is simply a real-valued measurable function on the sample space Ω.

- A **measurable function** is one such that the inverse image of any open set in $\mathbb{R}$ is measurable (i.e. is a member of the σ-algebra).

- Some elementary probability texts are not that strict and instead consider any real-valued function of the sample space to be a random variable.

Random Variables

- We use capital letters at the end of the alphabet to denote random variables: $X, Y, Z, ...$ (This helps us distinguish them from ordinary real-valued variables.)

- Later we will use lowercase letters so as not to confuse them with certain transforms.

Random Variables

- Note that random variables are not really variables in any ordinary sense – they are functions. Thus, if $\omega \in \Omega$, and X is a random variable on Ω, then $X(\omega) \in \mathbb{R}$, i.e., $X(\omega)$ is a real number.

- Also note that $X^{-1}(A) = \{\omega \in \Omega | X(\omega) \in A\}$ is always an event in the event algebra by definition.

Random Variables

- Random variables provide a way to quantify the outcomes of experiments.

- Random variables are important because we are often more interested in the consequences of an experiment than in the experiment itself.

- For example, a gambler is usually more interested in his wins and losses than in the particular game he's playing.

Random Variables

- Moreover, random variables are not really random.

- They are deterministic functions from the space of all possible outcomes to the real numbers. If you give them a particular outcome to operate on, they return the same real value every time.

- What gives them the appearance of randomness is really the fact that you don't really know what those values are. Usually, you only know (or assume you know) certain statistical properties like distributions (to be discussed soon).

Random Variables

- The "randomness" of random variables stems from the probabilistic nature of the underlying sample space. Usually, we can't say much about the individual outcomes in a sample space. We can only measure the size of events. (The connection between the "randomness" of the underlying space and the random variables will be made by means of probability distributions.)

- It is because random variables are functions that we find ourselves having to deal with infinite dimensional vector spaces, i.e., function spaces.

Equality and Equivalence of Random Variables

- Since random variables are functions, strict equality of random variables is the same as the equality of functions. Two functions are equal if and only if they have equal values when evaluated at the same point: $X = Y \Leftrightarrow X(\omega) = Y(\omega)$ for all $\omega \in \Omega$.

- However, when you take probability into account, things can get a little fuzzier. We say that two random variables X and Y are equal **almost surely** (a.s.) if $P\{\omega \in \Omega : X(\omega) \neq Y(\omega)\} = 0$.

- In other words, $X = Y$a.s., if the set of all points at which X and Y differ is insignificant, i.e., of measure 0.

Simple Random Variables

- If a random variable X takes on only finitely many values, then X is called a **simple random variable**.

- If X is a simple random variable and $\{a_1, \cdots, a_n\}$ are its values, let $A_i = \{\omega : X(\omega) = a_i\}$ for each $i = 1, \ldots, n$. Then X can bewritten as a sum:

(25) $$X = \sum_{i=1}^{n} a_i \chi_{A_i}$$

where $\chi_{A_i}(\omega) = \begin{cases} 1 & \text{if } \omega \in A_i, \\ 0 & \text{otherwise.} \end{cases}$ is the characteristic or indicator function on A_i.

Example: Simple Random Variable

- Double coin toss:
  - Sample Space: $\Omega = \{HH, HT, TH, TT\}$
  - σ-algebra: $\mathcal{M} = \{\emptyset, \{HH\}, \{HT, TH, TT\}, \Omega\}$
  - Probability Measure: $P(A)$ =number of elements in A divided by total number of elements in Ω

Example: Simple Random Variable

- Let $A = \{HT, TH, TT\}$ and define $X = \chi_A$. Then

$$X(HH) = 0$$
$$X(HT) = 1$$
$$X(TH) = 1$$
$$X(TT) = 1$$

Example: Simple Random Variable

- Let $B = \{HH\}$ and define $Y = -3\chi_A + 0.5\chi_B$
- Y is a simple random variable.
- We compute:

$$\begin{aligned} Y(HH) &= -3\chi_A(HH) + 0.5\chi_B(HH) \\ &= -3 \cdot 0 + 0.5 \cdot 1 \\ &= 0.5 \end{aligned}$$

Random Variables form a Vector Space

- Let $RV(\Omega)$ denote the set of all random variables on Ω. If $X, Y \in RV(\Omega)$ and $a, b \in \mathbb{R}$, then $aX + bY \in RV(\Omega)$.

- Recall that $(aX + bY)(\omega) = a \cdot X(\omega) + b \cdot Y(\omega)$, where $\cdot$ denotes ordinary real multiplication.

- For brevity's sake, we will drop the $\cdot$ and merely juxtapose. Likewise, we will drop Ω from $RV(\Omega)$ and merely write RV, assuming implicitly that the underlying sample space is Ω.

Probability Distributions and Densities

- Probability Distributions
- Probability Density Functions

Probability Distributions

- Every random variable X has an associated probability **distribution** function defined by

$$F_X(x) = P\{X^{-1}[-\infty, x)\} = P\{\omega \in \Omega : X(\omega) < x\}$$

- The value of the distribution function at a real number x is the probability that the value of X does not exceed x.

- We can simplify the notation somewhat by writing $\{X < x\}$ instead of $\{\omega \in \Omega : X(\omega) < x\}$.

- Thus, $F_X(x) = P\{X < x\}$.

Probability Distributions

- Distributions are nondecreasing: $x \leq y \Rightarrow F(x) \leq F(y)$.

- Distributions are always continuous from the left: $F(x) \to F(a)$ as $x \uparrow a$.

- The value of a distribution is always between 0 and 1 inclusively, and $\lim_{x \downarrow -\infty} F(x) = 0$ and $\lim_{x \uparrow \infty} F(x) = 1$

Probability Distributions
Three Flavors

- Continuous (no jumps)

- Piecewise continuous (some jumps)

- Discrete (only jumps and flat). X is discrete if and only if X takes on only finitely many, or at most countably infinite, distinct values. Note that a simple measurable function is a discrete random variable.

Some Discrete Distributions

- Discrete Uniform Distribution: Let $A = \{a_1, \ldots, a_n\}$ be a set of n distinct real numbers and assume that $a_1 < a_2 < \ldots < a_n$. Then a random variable X is said to have a discrete uniform distribution if
  $F(x) = \frac{1}{n} \cdot |A \cap [-\infty, x)|$.

- Let Ω be any sample space and $A \subseteq \Omega$. Let $X(\omega) = \chi_A(\omega)$.
  Recall that $\chi_A(\omega) = \begin{cases} 1 & \text{if } \omega \in A, \\ 0 & \text{otherwise.} \end{cases}$
  Then $F(x) = \begin{cases} 0 & \text{if } x < 0, \\ 1 - P(A) & \text{if } 0 \leq x < 1, \\ 1 & \text{if } x \geq 1. \end{cases}$

Continuous Distributions: Probability Density Functions

- X is said to have a continuous distribution if there exists a function $f : \mathbb{R} \to \mathbb{R}$ such that for each $x \in \mathbb{R}$

$$F(x) = \int_{-\infty}^{x} f(u)du. \tag{26}$$

- f is called a **probability density function**, or p.d.f, for short.

- The p.d.f of a random variable, if it exists, is not unique. Why?

Some Important Distributions

- The Gaussian or normal distribution with p.d.f:

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left[-\frac{(x-\mu)^2}{2\sigma^2}\right] \tag{27}$$

- The uniform distribution on $[a, b]$ with distribution function:

$$F(x) = \begin{cases} 0 & \text{if } x < a, \\ \frac{x-a}{b-a} & \text{if } a \leq x < b, \\ 1 & \text{otherwise.} \end{cases} \tag{28}$$

Moments: Means and Variances of Random Variables

- 1st Order Moment: Expectation or Mean
- 2nd Order Moment: Variance
- Higher Order Moments

1st Order Moment: Expectation of a Random Variable

- Let X be a simple random variable and let $a_1, \ldots, a_n$ be the distinct values taken on by X. Then, the expectation operator $\mathsf{E} : SRV \to \mathbb{R}$ is defined as

$$\mathsf{E}[X] = \sum_{i=1}^{n} a_i P(A_i) \tag{29}$$

  where $A_i = X^{-1}\{a_i\} = \{\omega \in \Omega : X(\omega) = a_i\}$.

- Thus, the expectation of X is the weighted sum of the values of X, the weights being the "size" or likelihood of the events.

- It is clear that $X = Y$ a.s. $\Rightarrow \mathsf{E}[X] = \mathsf{E}[Y]$.

Expectation

- Alternative Integral notation: $\mathsf{E}[X] = \int_\Omega X dP$, because the weighted sum given in 29 is just the definition of the integral of X with respect to the (probability) measure P.

- It can be shown that every random variable X can be written as the difference of two nonnegative random variables.

- It can also be shown that every nonnegative random variable is the limit of a monotone sequence of simple random variables. Therefore, integration can be extended from $SRV(\Omega)$ to $RV(\Omega)$

Expectation

- Note that we did not need the use of a probability distribution nor density in defining the expectation this way.

- We only needed the probability function.

- Note also that the integration we defined above is abstract in the sense that it works just as well for general measures, not just probability measures.

Expectation

- Expectation is a linear functional on the space RV of random variables over Ω:

$$\mathsf{E}[aX + bY] = a\mathsf{E}[X] + b\mathsf{E}[Y]$$

- Expectation is monotonic:

$$X \leq Y a.s. \Rightarrow \mathsf{E}[X] \leq \mathsf{E}[Y]$$

- We can compute $\mathsf{E}[X]$ by means of an ordinary Riemann-Stieltjes integral:

(30)
$$\mathsf{E}[X] = \int_{-\infty}^{\infty} x dF(x)$$

Expectation

- If X also has a p.d.f, then equation (30) becomes:

$$\text{(31)} \qquad \mathsf{E}[X] = \int_{-\infty}^{\infty} x dF(x) = \int_{-\infty}^{\infty} x F'(x) dx = \int_{-\infty}^{\infty} x f(x) dx$$

- The expectation of $\exp(juX)$ for each $u \in \mathbb{R}$ is called the **characteristic function** of X:

$$\text{(32)} \qquad \phi_X(u) = \mathsf{E}[\exp(juX)] = \int_{-\infty}^{\infty} \exp(jux) dF(x)$$

where $j = \sqrt{-1}$.

2nd Order Moment: Variance

- The variance of a random variable is defined as the mean square deviation from the mean:

$$\sigma^2 = \mathsf{E}\left[(X-\mu)^2\right]$$

Higher Order Moments

- The value of the integral

$$m_i = \int_{-\infty}^{\infty} x^n f(x) dx$$

  is called the nth **moment** of the distribution. If instead we subtract out the mean $\mu = \mathrm{E}[X]$ prior to integrating, then we obtain the nth **central moment**:

$$\mu_i = \int_{-\infty}^{\infty} (x - \mu)^n f(x) dx$$

- We note that $\mu = \mu_1$, the first moment.

Higher Order Moments

- ϕ_X is similar to the **moment generating function** defined as follows:

$$M_X(t) = \mathsf{E}\left[\exp(tX)\right]$$

- If X has a density function f, then

$$M_X(t) = \int_{-\infty}^{\infty} \exp(tx) f(x) dx$$

Higher Order Moments

- Note that

$$\mu = \mathsf{E}[X] = \frac{d}{dx} M_X(0)$$

and, in general,

$$\mathsf{E}[X^n] = \frac{d^n}{dx^n} M_X(0).$$

Higher Order Moments

- The Gaussian distribution has only two fundamental moments $\mu = \mathsf{E}[X]$ and $\sigma^2 = \mathsf{E}\left[(X-\mu)^2\right]$. All higher-order moments can be expressed in terms of these two.

- Thus, the Gaussian distribution is said to be a second-order distribution.

Individual Exercises

- Show that $A \subseteq B \Rightarrow P(A) \leq P(B)$.
  (Hint: Use $B = A \cup (B \setminus A)$ and the fact that measures are additive.)

- Show that $P(A \cup B) = P(A) + P(B) - P(A \cap B)$

- Let F be the distribution of a random variable X and suppose that $a \leq b$ are real numbers. Show that $P(x \in [a, b)) = F(b) - F(a)$.

Workgroup Exercises

- Prove the following: If two events A and B are independent, then the events A and B' are also independent.

- Show that for any distribution function F of a random variable X, $P(X > a) = \lim_{x \downarrow a} F(x)$.

Workgroup Exercises

- Let Ω be any sample space and $A \subseteq \Omega$. Let $X(\omega) = \chi_A(\omega)$.

  Recall that $\chi_A(\omega) = \begin{cases} 1 & \text{if } \omega \in A, \\ 0 & \text{otherwise.} \end{cases}$

  Prove that $F(x) = \begin{cases} 0 & \text{if } x < 0, \\ 1 - P(A) & \text{if } 0 \leq x < 1, \\ 1 & \text{if } x \geq 1. \end{cases}$

Workgroup Exercises

- A gyro manufacturer produces defective gyros with probability p, where $0 < p < 1$. After a lot of n independent gyros have been produced, they are examined. Let X denote the number of defective gyros in the lot. What is the probability mass function of this discrete distribution?

Segment 6
Random Processes
Topics

- Two Dimensions: Ensemble vs. Time
- Stationary and Ergodic
- Correlation and Covariance
- Power Spectral Density

Two Dimensions: Ensemble vs. Time

- There are two equivalent ways of defining a random process. One way is to view a random process as a time function whose value at each instant of time is a random variable.

- The other way is to define a random process as a function of two variables: the outcomes ω and time t.

- The outcomes ω belong to the sample space Ω while t is modeled as a real number, usually non-negative.

Two Dimensions: Ensemble vs. Time

- At each fixed t, the random process is simply a random variable, i.e., a function of the other variable ω alone

- For a fixed ω, we have an ordinary time function of t.

Random Processes

- A **Random Process** (also called a **stochastic process**) is a function $X : [0, T] \to RV(\Omega)$, where $[0, T]$ is a closed interval of the real line.

- We think of variables $t \in [0, T]$ as time.

- The image of $t \in [0, T]$ under X is a random variable:

$$X(t) : \Omega \to \mathbb{R}$$

.

Random Processes

- Thus a random process can be viewed as a function whose values are functions.

- We could also think of X as a two-variable function from the Cartesian product $[0, T] \times \Omega$ into the reals:

$$X(t, \omega) \in \mathbb{R}$$

- These two perspectives are interchangeable and we shall use both.

- We use all of the following notation for a random process

$$X_t$$

$$X_t(\omega)$$

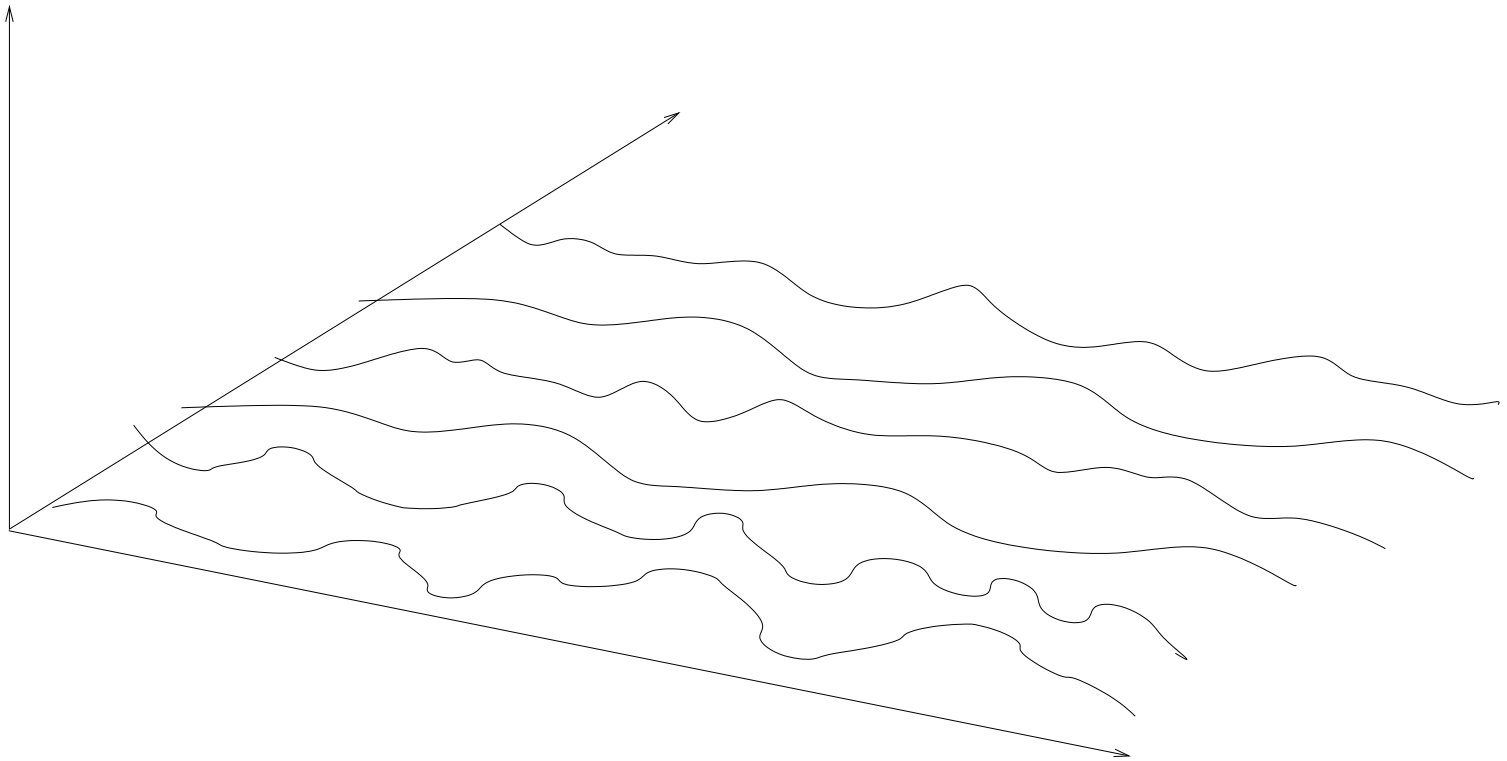
$$X(t)$$

$$X(t, \omega)$$

Random Processes

- If you fix ω and let t vary you get a **sample trajectory**.
- In this way, you can think of X as an ensemble of sample trajectories.
- Therefore, we will talk about both sample trajectory properties (continuity, differentiability, etc.) and ensemble properties (statistical).

A Random Process as an Ensemble of Sample Trajectories



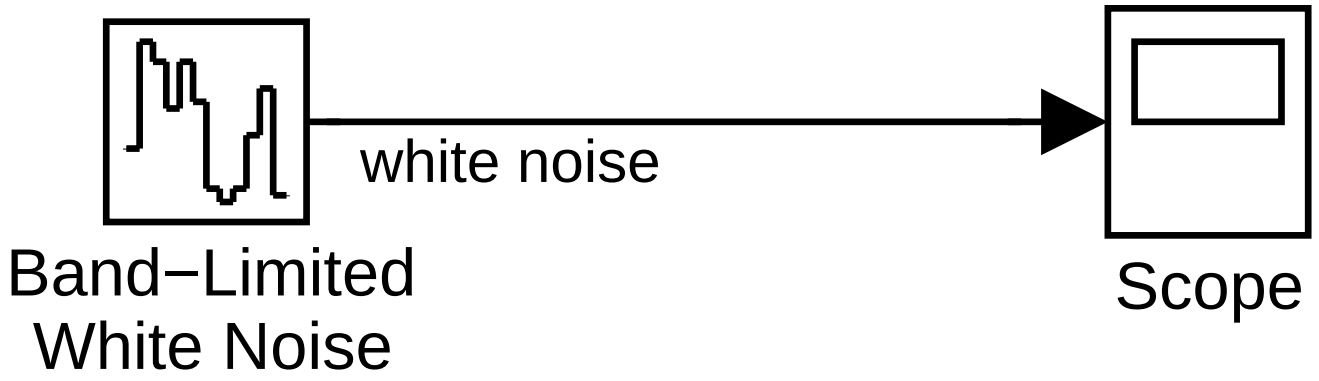
Examples of Random Processes

- A Discrete Process: Sampling with Replacement
  - A card player shuffles a deck of cards and then picks a card at random.
  - He enters the value of the card (1 for an ace through 13 for a king) in an Excel spreadsheet column.
  - He then replaces the card and repeats the process over and over again.
  - After 10 such experiments, he begins anew with a new Excel column. Each column represents a new sample trajectory.

Examples of Random Processes

- Matlab / Simulink Band-Limited White Noise
  - A single source block (band-limited white noise) and a single sink (the scope).
  - Each time the simulation is run, a new sample trajectory is generated.
  - Note that this discrete process is created by supposedly sampling a continuous process.

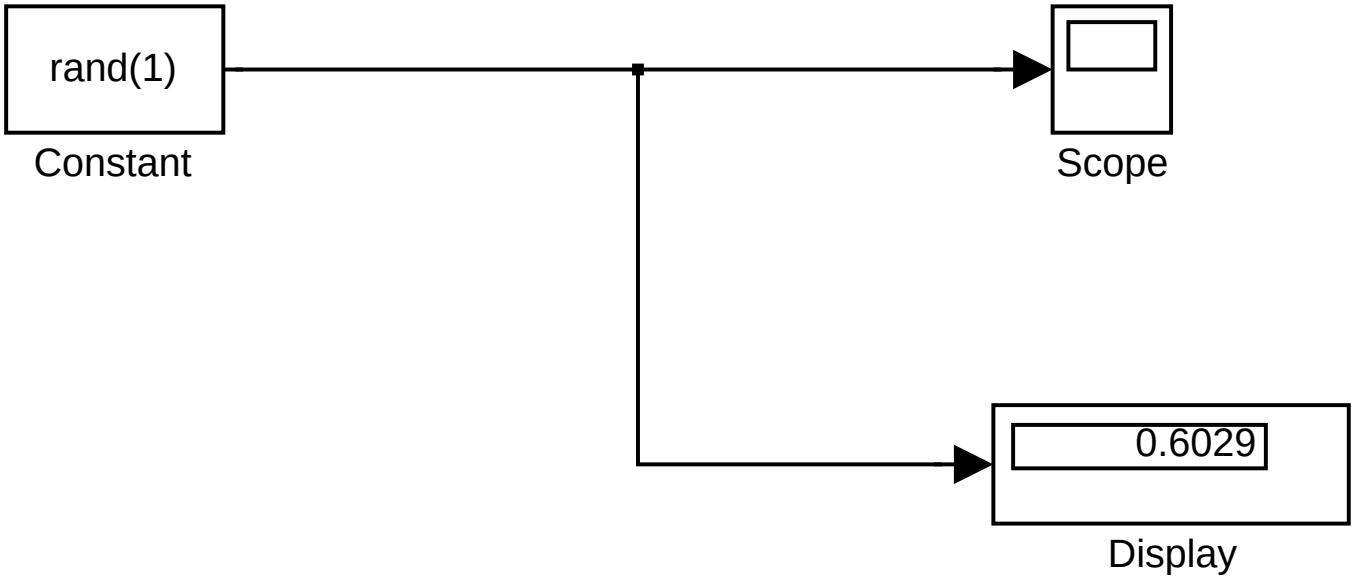
White Noise



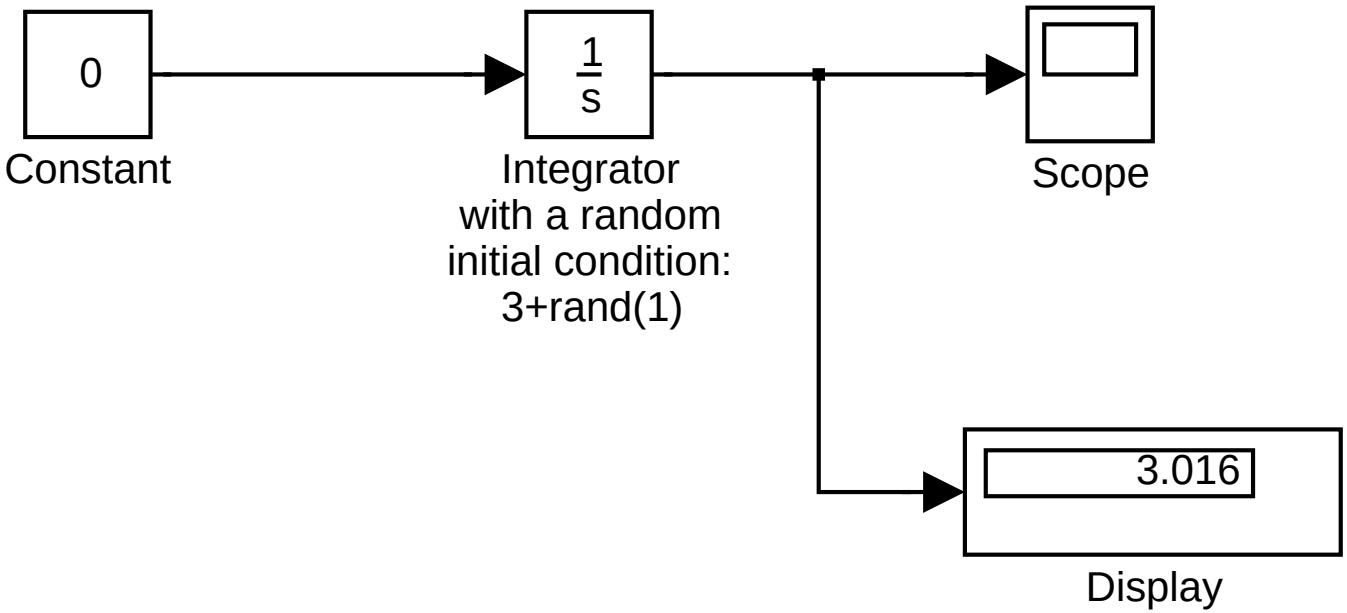
Examples of Random Processes

- Matlab / Simulink Random Constant
  - A single source block (constant block with value generated by random number generator) and a single sink (the scope).
  - Each time the simulation is run, a new sample trajectory is generated.
  - Note that this is a continuous process.

Random Constant



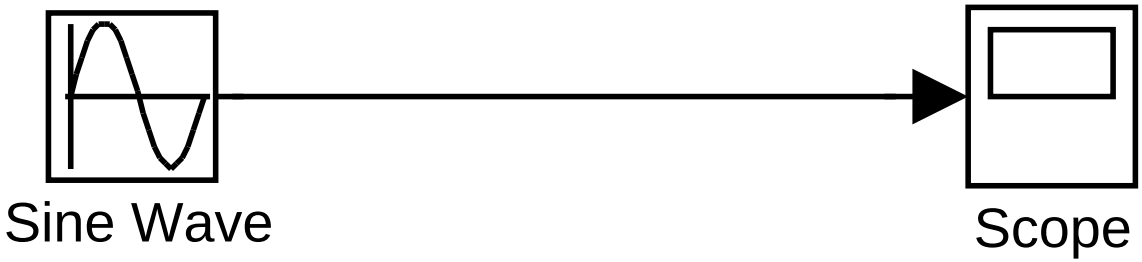
Random Constant



Examples of Random Processes

- Matlab / Simulink Sine Wave with Random Phase
  - A single source block (sine wave with random phase) and a single sink (the scope).
  - Each time the simulation is run, a new sample trajectory is generated.
  - Note that this is a continuous process.

Sine Wave with Random Phase



Stationary Random Processes

- A process X is called **strongly stationary** if for any finite set of times $\{t_1, \ldots, t_n\}$ and any $\tau > 0$ the two sets of random variables $\{X(t_1), \ldots, X(t_n)\}$ and $\{X(t_1 + \tau), \ldots, X(t_n + \tau)\}$ have identical probability distributions.

- X is called **weakly stationary** if

$$\mathsf{E}[X(s)] = \mathsf{E}[X(t)]$$

and

$$\mathsf{E}[X(s)X(t)] = \mathsf{E}[X(s+\tau)X(t+\tau)]$$

for all appropriate $\tau > 0$ and all times s, t.

Correlation and Covariance

- Since at each t $X(t)$ is a random variable in its own right, it makes sense to talk about the correlation of two such random variables $X(t)$ and $X(s)$.

- Since both random variables come from the same underlying process, we refer to this as autocorrelation.

- Similarly, autocovariance involves first centering the process about its mean process μ_t.

- The function $\mathsf{E}\left[X(s)X(t)\right]$ is called the **autocorrelation function** of X and for a weakly stationary process X is denoted by $R_{XX}(\tau)$ since it is a function of τ alone.

- The **autocovariance** function for a weakly stationary process is formed by subtracting out the means: $C_{XX}(\tau) = \mathsf{E}\left[(X(t) - \mu_t)(X(t+\tau) - \mu_{t+\tau})\right]$.

- Cross correlation and cross covariance between two different random processes X and Y are defined in the obvious way.

Examples of Autocorrelation Functions

- Consider the random process $X(t,\omega) = A(\omega)\sin(2\pi f t + \theta)$ in which $A : \Omega \to \mathbb{R}$ is a normally distributed random variable with variance σ^2.

- The autocorrelation function $R_{XX}(s,t)$ is therefore

$$\begin{aligned}
\mathrm{E}\left[X(t,\omega)X(s,\omega)\right] &= \mathrm{E}\left[A(\omega)\sin(2\pi f t + \theta)A(\omega)\sin(2\pi f s + \theta)\right] \\
&= \mathrm{E}\left[A(\omega)^2\right]\sin(2\pi f t + \theta)\sin(2\pi f s + \theta) \\
&= \sigma^2 \sin(2\pi f t + \theta)\sin(2\pi f s + \theta)
\end{aligned}$$

Examples of Autocorrelation Functions

- Consider the white noise process $W(t)$.
- It is a hallmark of white noise to be uncorrelated in time.
- Thus, $\mathsf{E}\,[W(t)W(s] = 0$ for every s and t such that $s \neq t$.

Ergodic Processes

- We shall be concerned almost exclusively with weakly stationary processes that are also **ergodic**, meaning that ensemble averaging at any one time gives the same result as time averaging one sample trajectory. Thus we may assume:

(33) $$\mu_X = \lim_{T\to\infty} \frac{1}{T} \int_0^T X(t)dt$$

(34) $$\psi_X^2 = \lim_{T\to\infty} \frac{1}{T} \int_0^T X^2(t)dt$$

(35) $$R_{XX}(\tau) = \lim_{T\to\infty} \frac{1}{T} \int_0^T X(t)X(t+\tau)dt$$

Ergodic Processes

- We haven't yet defined want we mean by such time integrals, however, so we have to hold off dealing with them.

- Once we have defined the mean square limit of Riemann-Stieltjes sums, then we'll know what these integrals really mean.

- Nevertheless, if we fix ω and consider only sample trajectories $X(t, \omega)$, then we can use ordinary integrals provided that the sample trajectories are well-enough behaved.

Ergodic Processes

- Well-behaved means that they are of bounded variation, i.e., they don't wiggle around too much.

- Unfortunately, this doesn't really work for the usual conception of white noise, because its sample trajectories are not of bounded variation.

- If we restrict our attention to band-limited white noise (like Matlab / Simulink does), then we can stick to the ordinary integrals we learned in calculus.

Power Spectral Density

- The **power spectral density** (PSD) function is the Fourier transform of the autocorrelation function:

$$P_{XX}(\omega) = \int_{-\infty}^{\infty} R_{XX}(\tau) \exp(-j\omega\tau) d\tau \tag{36}$$

  where $j = \sqrt{-1}$ and ω is angular frequency in rad/sec (not an outcome of the sample space!).

- Units: If the process X has units of volts, say, then P_{XX} will have units of V^2/Hz or V^2 sec/rad. This is due to the fact that R_{XX} is a product of samples, each of which is in volts, and the transform kernel, $\exp(-j\omega\tau)$ is the same as $\frac{1}{\exp(j\omega\tau)}$.

Power Spectral Density

- Sometimes the definition in equation (36) is referred to as the autospectral density, since the autocorrelation function is involved.

- The cross-spectral density function is defined in the same way, but with R_{XY} substituted for R_{XX}.

Power Spectral Density

- Another way to obtain spectral density functions is through Fourier transforms of the original process data.

- However, we must change our notational convention for a while: We've been using uppercase letters such as X and Y to denote random processes and we've been using ω to denote a point of the sample space Ω.

- Now we are going to use lowercase letters such as $x(t)$ and $y(t)$ for the process time data and reserve the uppercase letters for their Fourier transforms. And ω will now stand for frequency in rad/sec.

Power Spectral Density

- The PSD can be expressed as follows:

$$P_{XY}(\omega) = \lim_{T\to\infty} \frac{1}{T} \mathsf{E}\left[X^*(\omega, T)Y(\omega, T)\right] \tag{37}$$

where $X(\omega, T)$ and $Y(\omega, T)$ are the finite Fourier transforms of $x(t)$ and $y(t)$, respectively, and $X^*(\omega, t)$ is the complex conjugate of $X(\omega, t)$. ω is frequency in rad/sec.

- The finite Fourier transform of $x(t)$ is

$$X(\omega, T) = \int_0^T x(\tau)\exp(-j\omega\tau)d\tau$$

- Equation (37) Wiener-Khinchin equation.

Segment 7
Some Important Random Processes
Topics

- White Noise
- Random Walk
- Random Constant
- Random Ramp
- Markov Processes

- State Space Augmentation

White Noise

- White noise can be defined as a process whose autocorrelation function is a Dirac delta function:

(38) $$\delta(\tau) = \begin{cases} \infty & \text{if } ,\tau = 0 \\ 0 & \text{otherwise.} \end{cases}$$

- There's only one mathematical problem: this is not a real-valued function.

- There's also a physical problem: such a process would have infinite mean-squared value and hence infinite energy.

White Noise

- There is a way to properly define the Dirac delta as a linear functional, not a function, and show that all of its desirable properties are preserved.

- However, we'll proceed as if this concept of white noise were well-defined and use the delta function's integration properties to define some other processes such as random walk.

- The hallmark of a white noise process W is that it is uncorrelated in time. This is the meaning of the statement that W has the Dirac function as its autocorrelation function.

White Noise Process

- The Matlab / Simulink approach to white noise is to use bandlimited white noise, which means that the bandwidth is not infinite as it is for pure white noise.

- This is valid as long as the bandwidth of the noise is sufficiently greater than that of the system you are dealing with.

- In Matlab, you can simulate a discrete white noise sequence by drawing a random number at each time step from a normal distribution.

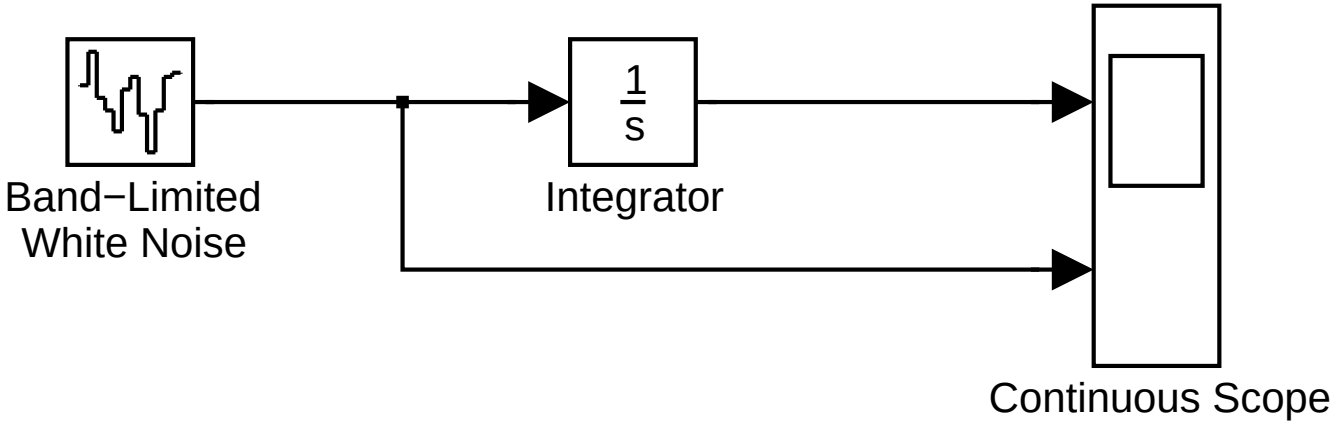
Random Walk

- In engineering applications, random walk is a discrete time process.

- Some authors (Gelb, for example) refer to random walk in the continuous case as the solution to a differential equation driven by white noise: $\dot{X} = W$. However, this would make W the derivative of the Wiener-Levy process, but that process is not differentiable.

- So we'll stick to the discrete case: $X_{k+1} = X_k + W_k$, where the W_k are normally distributed and statistically uncorrelated at distinct sample times.

Random Walk



Random Ramp and Random Constant

- The random ramp requires two state variables (to be discussed more later): a random constant (y-intercept) and the integral of a random constant (slope)

- A random constant is a process X such that the sample trajectories are constant with respect to time.

Random Ramp and Random Constant

- In principle, however, this constant value could vary with ω, i.e., across the ensemble, thus making the process non-Ergodic.

- Random constants are very useful in many applications to model turn-on to turn-on biases, for example.

Random Ramp and Random Constant

- See Matlab Simulink demos: rand_const.mdl and rand_ramp.mdl
- In modeling gyro biases, a combination of a random constant (bias stability) and random walk (bias drift) is common.
- Gyro scale factors are modeled similarly.
- Deterministic biases and scale factors are also part of the model, but these generally are calibrated out.

Second Order Processes

- We will generally restrict our attention to **second-order processes**, i.e., those such that $X(t) \in L^2(\Omega)$.

- Recall what this means:

$$\mathsf{E}\left[X^2(t)\right] < \infty$$

- This allows us to use Hilbert space techniques because, for every $s, t \in [0, T]$, $\mathsf{E}\left[X(s)Y(t)\right]$ is an inner product. Inner products, norms and metrics allow us to do calculus and geometry (orthogonal projection).

Gaussian Processes

- An important class of second-order processes are the Gaussian processes.

- Let $X : [0, T] \to L^2(\Omega)$ be a second-order process. X is said to be **Gaussian** if for every finite set of times $\{t_1, \ldots, t_n\}$ the vector of random variables $(X(t_1), \ldots, X(t_n))$ has a multivariate normal distribution.

- To fully appreciate what this means, we need to explain what a multivariate normal distribution is.

Gaussian Processes

- A function $Q_C : \mathbb{R}^n \to \mathbb{R}$ of the form

$$Q_C(x) = x^\top C x$$

where $x \in \mathbb{R}^n$ is a column vector and C is a real symmetric matrix., is called a **quadratic form** associated with C. (I.e., $C = (c_{ij})$ and $c_{ij} = c_{ji}$).

- We will further assume that C is positive definite, which means that $Q_C(x) > 0$ for every non-zero vector x.

- Then $X_1, \ldots, X_n$ are said to have a **multivariate normal distribution** if their joint p.d.f. is of the form
(39)
$$f(x_1, \ldots, x_n) = \frac{1}{\sqrt{(2\pi)^n \det(C)}} \exp\left[-\frac{1}{2}(x-\mu)^\top C^{-1}(x-\mu)\right]$$
for a real, positive definite symmetric matrix C and $\mu = (\mu_1, \ldots, \mu_n)$.

- C is called the **covariance matrix** and μ is called the **mean** vector.

Gaussian Processes

- In terms of quadratic forms, this is

$$f(x_1, \ldots, x_n) = \sqrt{\frac{\det(C^{-1})}{(2\pi)^n}} \exp\left[-\frac{1}{2} Q_{C^{-1}}(x - \mu)\right]$$

- C^{-1} is sometimes called the information matrix, because its inverse, the covariance matrix, often expresses the uncertainty (or lack of information) associated with the random variables.

Gaussian Processes

- Thus a Gaussian process is a random process for which the joint distribution of every finite collection of range random variables is a Gaussian distribution.

- The Gaussian process is important in connection with the kinds of noise that drive the dynamic systems we are interested in.

- The linear Kalman filter is optimal only under assumptions that the process noise and the measurement noise are Gaussian.

Gauss-Markov Processes

- A Gauss-Markov process is for which the autocorrelation function has an exponential form:

(40) $$R_{XX}(\tau) = \sigma^2 \exp(-\beta\,|\tau|)$$

- In dynamic terms, this would correspond to passing white noise through first-order feedback system:

$$\dot{X} = -\beta X + W$$

- See Matlab Simulink file gauss_markov.mdl

Gauss-Markov Processes

- Discrete version of a Gauss-Markov process:

$$X_{k+1} = \exp(-\beta \Delta t)\, X_k + W_k$$

where $\Delta t = t_{k+1} - t_k$ and is assumed to be the same regardless of k.

State Space Augmentation

- Kalman filter theory assumes forcing function is white noise
- If this is not satisfied, then new states need to be added
- The most frequently used models can all be understood as differential equations (or difference equations) driven by white noise
- We'll see an example of state space augmentation later when we discuss filter maintenance

Exercises

1. Perform the following random walk experiment:
a. Create a graph with two axes marked off at integral (discrete) points.
b. Plot the first point at the origin.
c. Flip a coin. If the result is heads, move one step to the right and plot a point 1 unit above the previous y value. If the result is tails, move one step to the right and plot a point 1 unit below the previous y value.
d. Repeat step c ten times.

2. What is your estimate for how the variance of this process grows with time?

3. As a class exercise, we'll compute the ensemble averages across the class results at each point. Is this pocess ergodic?

Segment 8
Kalman Statistics
Topics

- The Static Estimation Problem: Derivation of the Kalman Gain
- Covariance Matrices
- Process Noise
- Measurement Noise

- Kalman Assumptions

- Error Covariance Matrix (P)

The Static Estimation Problem: Derivation of the Kalman Gain

- Given: A known measurement vector $z = (z_1, z_2, \ldots, z_m)$ that is believed to be causally related to the state vector $x = (x_1, \ldots, x_n)$.

- Find: The "best" **estimate** of the state as a function of the measurements: $\widehat{x} = g(z) = g(z_1, \ldots, z_m)$.

The Static Estimation Problem

- Note that we are ignoring the state dynamics for now – hence the term "static."

- Since the problem for now is static, we do not have any stochastic differential equations to worry about. Hence, we don't have to use the mean square calculus.

- We are only concerned with the statistical problem of estimating a vector random variable x from a finite set of measurements.

The Static Estimation Problem: Why should we care?

- The solution to this problem will introduce concepts that are central to traditional Kalman filtering.

- The solution presented here is nearly identical to the one given by Kalman himself in his 1960 paper. Hence, the result is now classical.

- What we will be deriving is a matrix K, called the **Kalman gain**, that will be used to weight the measurement residual in order to update the prior state estimate.

The Static Estimation Problem (continued)

- What do we mean by "best" estimate?

- Need to define a measure of "goodness", i.e., a performance measure and maximize it.

- Actually, it turns out to be more convenient to define a measure of "badness" that we can minimize.

  - This is called a "loss function" or "penalty function".

The Static Estimation Problem (continued)

- Estimation Error: The difference between the truth and the estimate: $\tilde{x} = \hat{x} - x$

- Estimation error is bad.
  - Our estimators should try to minimize this error.
  - At the very least, we should try to keep track of how well we're doing (i.e., should have an error covariance matrix).

The Static Estimation Problem (continued)

- Admissible Loss Functions:
  - nonnegative, real-valued function of the estimation error;
  - non-decreasing, i.e., the loss gets bigger as the estimation error gets bigger;
  - should be zero when there is no error, i.e., when we are certain;
  - should not be dependent on the sign of the error, i.e., should be a symmetric function.

- Note: As a function of a random variable, the loss function will yield a random variable. Hence, we'll want to minimize the mean of the loss function.

The Static Estimation Problem (continued)

- Estimation problem becomes: Find estimate $\hat{x}$ of x that minimizes the mean of the loss function L:
  $E\{L(\tilde{x})\} = E\{L(\hat{x} - x)\}$

- A common loss function is a quadratic function: the square of the norm of the error vector: $L(\tilde{x}) = \tilde{x}^\top \tilde{x}$
  Often referred to as the "square error."

- Therefore, we want to minimize the mean: $E\{L(\tilde{x})\} = E\{\tilde{x}^\top \tilde{x}\}$
  i.e., we want to minimize the "mean square error."

- Note that L is really a function of two variables: $L(\tilde{x}) = L(\widehat{x}, x)$.

The Static Estimation Problem (continued)

- Estimation problem becomes: Find estimate $\widehat{x}$ of x that minimizes the integral:

$$(41)\quad E\{\tilde{x}^\top \tilde{x}\} = \underbrace{\int_{-\infty}^{+\infty} \cdots \int_{-\infty}^{+\infty}}_{n+m} \tilde{x}^\top \tilde{x} f(x,z) dx_1 \cdots dx_n dz_1 \cdots dz_m$$

  where $f(x,z)$ is the joint probability density function for x and z.

- x is assumed to be an n-vector, while z is assumed to be an m-vector.

The Static Estimation Problem (continued) But we can factor the joint p.d.f, through the conditional p.d.f:

$$f(x, z) = f(x|z)f(z)$$

where

$f(z) = \underbrace{\int_{-\infty}^{+\infty} \cdots \int_{-\infty}^{+\infty}}_{n} f(x, z)dx_1 \cdots dx_n$ is the marginal p.d.f of z.

The Static Estimation Problem (continued) Therefore, we can modify Eq. (41), our integral expression for $E\{\tilde{x}^\top \tilde{x}\}$, as follows:

$$
\begin{aligned}
E\{\tilde{x}^\top \tilde{x}\} &= \underbrace{\int_{-\infty}^{+\infty} \cdots \int_{-\infty}^{+\infty}}_{n+m} \tilde{x}^\top \tilde{x} f(x,z) dx_1 \cdots dx_n dz_1 \cdots dz_m \\
&= \underbrace{\int_{-\infty}^{+\infty} \cdots \int_{-\infty}^{+\infty}}_{n+m} \tilde{x}^\top \tilde{x} f(x|z) f(z) dx_1 \cdots dx_n dz_1 \cdots dz_m \\
&= \underbrace{\int_{-\infty}^{+\infty} \cdots \int_{-\infty}^{+\infty}}_{m} f(z) \left[\underbrace{\int_{-\infty}^{+\infty} \cdots \int_{-\infty}^{+\infty}}_{n} \tilde{x}^\top \tilde{x} f(x|z) dx_1 \cdots dx_n \right] dz_1 \cdots dz_m
\end{aligned}
$$

Note that we have pulled $f(z)$ out of the x-integral, because $f(z)$ does not depend on x.

The Static Estimation Problem (continued)

Notice that we now only need to minimize the integral within the brackets, i.e.,

$E\{\tilde{x}^\top \tilde{x}\}$ is minimized if and only if

$$\underbrace{\int_{-\infty}^{+\infty} \cdots \int_{-\infty}^{+\infty}}_{n} \tilde{x}^\top \tilde{x} f(x|z) dx_1 \cdots dx_n \text{ is minimized.}$$

Remember that we are trying to find the value of $\hat{x}$ that will cause this loss function to be minimized.

The Static Estimation Problem (continued)

Let's expand $\tilde{x}^\top \tilde{x}$:

$$\begin{aligned}\tilde{x}^\top \tilde{x} &= (\hat{x} - x)^\top (\hat{x} - x) \\ &= \hat{x}^\top \hat{x} - \hat{x}^\top x - x^\top \hat{x} + x^\top x \\ &= \hat{x}^\top \hat{x} + x^\top x - 2\hat{x}^\top x\end{aligned}$$

The Static Estimation Problem (continued)

Thus, we expand the integral to be minimized accordingly:

$$\underbrace{\int_{-\infty}^{+\infty}\cdots}\tilde{x}^{\top}\tilde{x}f(x|z)dx_1\cdots dx_n = \hat{x}^{\top}\hat{x}$$
$$+\underbrace{\int_{-\infty}^{+\infty}\cdots}x^{\top}xf(x|z)dx_1\cdots dx_n$$
$$-2\hat{x}\underbrace{\int_{-\infty}^{+\infty}\cdots}xf(x|z)dx_1\cdots dx_n$$

This last expansion is justified because $\widehat{x}^\top \widehat{x}$ is a function of the z alone: $\widehat{x} = g(z_1, \ldots, z_m)$. The equation of interest has the form:

$$(42) \quad \underbrace{\int_{-\infty}^{+\infty} \cdots}_{} \tilde{x}^\top \tilde{x} f(x|z) dx_1 \cdots dx_n = f_1(\widehat{x}) + f_2(z) - 2f_3(\widehat{x}, z)$$

where $f_1(\widehat{x}) = \widehat{x}^\top \widehat{x}$, $f_2(z) = \underbrace{\int_{-\infty}^{+\infty} \cdots}_{} x^\top x f(x|z) dx_1 \cdots dx_n$ and

$f_3(\widehat{x}, z) = \widehat{x}^\top \underbrace{\int_{-\infty}^{+\infty} \cdots}_{} f(x|z) dx_1 \cdots dx_n.$

The Static Estimation Problem (continued)

Now we need to minimize the right side of the last equation. To do so, we must take the gradient with respect to $\widehat{x}$, set it equal to zero, and solve for $\widehat{x}$ – a typical min/max problem. Recall that the gradient of the dot product is: $[\partial/\partial x]\,(y^\top x) = y$ and also $[\partial/\partial x]\,(x^\top x) = 2x$.

The Static Estimation Problem (continued)

Applying this to our problem, we note that the middle term of Eq. (42), $f_2(z)$ is a function of z alone and thus vanishes when the gradient with respect to $\hat{x}$ is applied to it.
We are thus left with the first and third terms only:

$$[\partial/\partial\hat{x}]\,(\hat{x}^\top\hat{x}) = 2\hat{x} \quad \text{and}$$

$$[\partial/\partial\hat{x}]\,(\hat{x}^\top \underbrace{\int_{-\infty}^{+\infty}\cdots}\, x f(x|z)dx_1\cdots dx_n)$$
$$= \underbrace{\int_{-\infty}^{+\infty}\cdots}\, x f(x|z)dx_1\cdots dx_n.$$

The Static Estimation Problem (continued)

Thus, taking the gradient with respect to $\hat{x}$ of Eq. (42) we obtain

$$0 = 2\hat{x} - 2\underbrace{\int_{-\infty}^{+\infty} \cdots}\, x f(x|z) dx_1 \cdots dx_n \qquad \text{or}$$

$$\hat{x} = \underbrace{\int_{-\infty}^{+\infty} \cdots}\, x f(x|z) dx_1 \cdots dx_n \tag{43}$$

The Static Estimation Problem (continued)

This solution is just the conditional expectation for x, conditioned on the z:

$$\widehat{x} = \underbrace{\int_{-\infty}^{+\infty} \cdots}_{} x f(x|z) dx_1 \cdots dx_n = E\left[x|z\right].$$

The Static Estimation Problem (continued)

Now let's see what this looks like with Gaussian statistics for $f(x|z)$:

$$E[x|z] = m_x + P_{xz}P_{zz}^{-1}(z - m_z), \tag{44}$$

where m_x, m_z are the means of x and z, respectively, and P_{xz} is the cross-covariance matrix between x and z, while P_{zz} is the covariance matrix for z alone.

The Static Estimation Problem (continued)

In other words, we have for Eq. (44):

$$
\begin{aligned}
E\left[x|z\right] &= m_x + P_{xz}P_{zz}^{-1}(z - m_z) \\
m_x &= E[x] \\
m_z &= E[z] \\
P_{xz} &= E[(x - m_x)(x - m_z)^\top] \\
P_{zz} &= E[(z - m_z)(z - m_z)^\top]
\end{aligned}
$$

The Static Estimation Problem (continued)

Eq. (44) represents the essence of what will become the Kalman update algorithm. It will give us a new estimate of the state vector x conditioned on the actually observed measurements and all associated statistics.

Now consider again the relationship between x and z that we have previously assumed:

$$z = Hx + v$$

Remember that x and z are statistically independent of each other.

The Static Estimation Problem (continued)

Since $m_z = E[z] = E[Hx + v] = H \cdot E[x] + E[v] = Hm_x + m_v$, we have

$$\begin{aligned}
P_{xz} &= E[(x - m_x)(z - m_z)^\top] \\
&= E[(x - m_x)\{Hx + v - (Hm_x + m_v)\}^\top] \\
&= E[(x - m_x)\{(Hx - Hm_x) + (v - m_v)\}^\top] \\
&= E[(x - m_x)\{H(x - m_x) + (v - m_v)\}^\top] \\
&= E[(x - m_x)\{(x - m_x)^\top H^\top + (v - m_v)^\top\}] \\
&= E[(x - m_x)(x - m_x)^\top H^\top] + E[(x - m_x)(v - m_v)^\top] \\
&= E[(x - m_x)(x - m_x)^\top H^\top]
\end{aligned}$$

because x,z independent $\Rightarrow E[(x - m_x)(v - m_v)^\top] = 0$.

The Static Estimation Problem (continued)

But, $E[(x - m_x)(x - m_x)^\top] = P_{xx}$ by definition.

Thus

$$\begin{aligned} P_{xz} &= E[(x - m_x)(x - m_x)^\top H^\top] \\ &= P_{xx} H^\top \end{aligned}$$

A similar derivation shows that

$$P_{zz} = H P_{xx} H^\top + P_{vv}$$

Later we'll see this in the form of $H P_{xx} H^\top + R$ since $P_{vv} = R$ by definition.

The Static Estimation Problem (continued)

Inserting these last two results into Eq. (44) yields

$$\begin{aligned} E\left[x|z\right] &= m_x + P_{xz}P_{zz}^{-1}(z - m_z) \\ &= m_x + P_{xx}H^\top[HP_{xx}H^\top + P_{vv}]^{-1}(z - m_z) \\ &= m_x + P_{xx}H^\top[HP_{xx}H^\top + P_{vv}]^{-1}(z - Hm_x) \end{aligned}$$

This last step results because
$m_z = E[z] = E[Hx + v] = H \cdot E[x] + E[v] = Hm_x + 0 = Hm_x$
by definition.

The Static Estimation Problem (continued)
Kalman Gain

Eq. (44) now becomes the central result of classical Kalman Filter theory:

(45) $$\widehat{x} = E[x|z] = m_x + K(z - Hm_x)$$

where $K = P_{xx}H^\top[HPH^\top + P_{vv}]^{-1}$ is the **Kalman gain**.

Covariance Matrices

- There are several covariance matrices associated with the Kalman filter: $P_k, w(k), v(k)$

- In general, a covariance matrix is associated with a random vector X as in the following definition:

$$C_X = \mathsf{E}\left[XX^\top\right]$$

Covariance Matrices

- This is precisely how $w(k)$ and $v(k)$ are defined:

$$w(k) = \mathsf{E}\left[w_k w_k^\top\right]$$

and

$$v(k) = \mathsf{E}\left[v_k v_k^\top\right]$$

- P_k^+, on the other hand is called the error covariance, because it is associated with $\tilde{x}$

$$P_k^+ = \mathsf{E}\left[\tilde{x}^+(k)[\tilde{x}^+(k)]^\top\right]$$

where $\tilde{x} = \hat{x}^+(k) - x(k)$.

Process Noise

- The process noise w_k is assumed to be a white, zero-mean, Gaussian sequence.

- Although the continuous form of white noise does not exist, it is sufficient in the discrete case to consider a random process sequence w_k that is uncorrelated from step to step: $\mathrm{E}\left[w_j w_k^\top\right] = 0$ if $j \neq k$.

- This can be achieved if w_k results from sampling a Gauss-Markov process whose exponential autocorrelation function rapidly approaches 0 before the next sample is taken.

Process Noise

- The process noise covariance affects the Kalman gain indirectly through the covariance propagation equation:

$$P_k^- = \Phi_k P_{k-1}^+ \Phi_k^\top + Q_k$$

- This updated error covariance is then included in both the "numerator" and "denominator" of the Kalman gain:

$$K_k = P_k^- H_k^\top \left[H_k P_k^- H_k^\top + R_k \right]^{-1}$$

- If the process noise covariance is large relative to the measurement noise then the gain will be nearly unity, the extrapolated state estimate will cancel out and the updated state estimate will be nearly identical to the measurement itself.

Measurement Noise

- The measurement noise covariance matrix $v(k)$ indicates the amount of spread or dispersion and internal cross covariance within the measurement itself at any time k.

- The diagonal elements of $v(k)$ give the expected variance in each of the individual measurements.

Measurement Noise

- The off-diagonal elements indicate the cross covariance between noise random variables associated with each individual measurement.

- If the noise random variables of individual measurements are uncorrelated, then R_k will be a diagonal matrix.

Comparison of Process Noise and Measurement Noise

- If the process noise covariance is large relative to the measurement noise then the gain will be nearly unity, the extrapolated state estimate will cancel out and the updated state estimate will consist essentially only of the measurement itself.

- This can be seen as follows in the special case that H_k is

square and invertible:

$$
\begin{aligned}
\widehat{x}^{+}(k) &= \widehat{x}^{-}(k) + K_k \left[z(k) - H_k \widehat{x}^{-}(k) \right] \\
&\approx \widehat{x}^{-}(k) + P_k^{-} H_k^{\top} \left[H_k P_k^{-} H_k^{\top} \right]^{-1} \left[z(k) - H_k \widehat{x}^{-}(k) \right] \\
&= \widehat{x}^{-}(k) + P_k^{-} H_k^{\top} \left[H_k^{\top} \right]^{-1} \left[P_k^{-} \right]^{-1} H_k^{-1} \left[z(k) - H_k \widehat{x}^{-}(k) \right] \\
&= \widehat{x}^{-}(k) + H_k^{-1} z_k - \widehat{x}^{-}(k) \\
&= H_k^{-1} z_k
\end{aligned}
$$

Comparison of Process Noise and Measurement Noise

- On the other hand, when R_k dominates Q_k (and thus P_k^-), the gain is nearly 0, and the new updated estimate is simply the previously extrapolated estimate:

$$\begin{aligned}\widehat{x}^+(k) &= \widehat{x}^-(k) + K_k\left[z(k) - H_k\widehat{x}^-(k)\right] \\ &\approx \widehat{x}^-(k) + 0\left[z(k) - H_k\widehat{x}^-(k)\right] \\ &= \widehat{x}^-(k)\end{aligned}$$

Error Covariance Matrix

- P is the **estimation error covariance**, although it is commonly referred to simply as THE covariance matrix.

- $P_k^+ = \mathsf{E}\left[\tilde{x}^+(k)[\tilde{x}^+(k)]^\top\right]$

- The report card for the Kalman filter

- Contains most of the information about how well, or poorly your filter is doing.

Error Covariance Matrix

- P brings Q and R together

- In continuous framework, P is solution to matrix Riccatti differential equation:

(46) $$\dot{P} = FP + PF^{\top} + Q - PH^{\top}R^{-1}HP$$

- Covariance analysis is primarily concerned with solving this equation or its discrete equivalent

Kalman Assumptions

- Your initial state estimate $\widehat{x}_0$ is not too far off
- Your initial error covariance estimate P_0 is not too far off
- Process noise is truly white, i.e., uncorrelated in time
- Measurement noise is truly white, ie., uncorrelated in time
- Measurement noise and process noise are uncorrelated with each other

Kalman Assumptions: Measurement Noise and Process Noise Are Uncorrelated

- The Kalman assumptions require that the process noise and measurement noise be uncorrelated

$$\mathsf{E}\left[w_j v_k^\top\right] = 0$$

  for all j and k.

- If this turns out not to be the case, then you have to re-model your filter, perhaps using state space augmentation. Otherwise, your filter could diverge.

Kalman Assumptions: Measurement Noise and Process Noise Are Uncorrelated: An Exception

- It is possible to allow for the case:

$$\mathsf{E}\left[w_j v_k^\top\right] = C_j$$

  when $j = k$ (and 0 otherwise).

- This leads to modification of the Kalman equations which incorporate the C_j cross covariance matrix.

Kalman Assumptions: Linearity

- System dynamics are linear
  - This means that to use Kalman filtering in non-linear situations, some sort of linearization has to introduced

- Relationship of measurement to state is linear
  - Again, if this is not the case, then some kind of linearization has to take place

Exercises

- Show that any covariance matrix C symmetric, i.e., that $C^\top = C$.

- Suppose that λ is an eigenvalue of P corresponding to the eigenvector v, i.e., $Pv = \lambda v$. Prove that $\lambda > 0$. Hint examine the quadratic form induced by P: $v^\top P v$.

Day 3: Strapdown Navigation and Inertial Error Dynamics Segments

- Some Practical Considerations (Numerical)
- The Navigation Problem and Reference Frames
- Inertial Error Dynamics
- The Basic 9-State Kalman Filter
- Challenges of INS Filters

Segment 9
Some Practical Considerations (Numerical)
Topics

- Kalman Filter Theory vs. Reality
- Numerical Stability
- Round-Off Errors
- Matrix Inversion
- UD Factorization

Kalman Filter Theory vs. Reality

- Kalman theory based on multiple assumptions
  - All states are modeled correctly
  - Noise parameters are accurately known
  - Initial values are appropriate
  - Process and measurement noise are both Gaussian
  - Process noise and measurement noise are uncorrelated
  - Measurements are taken at regularly scheduled intervals

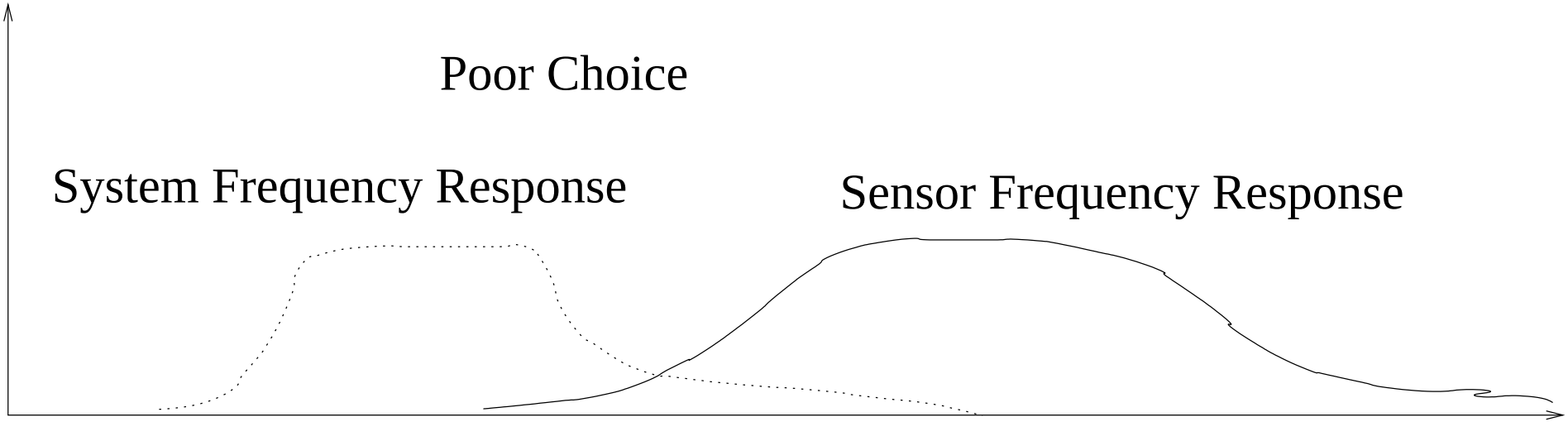
Kalman Filter Theory vs. Reality

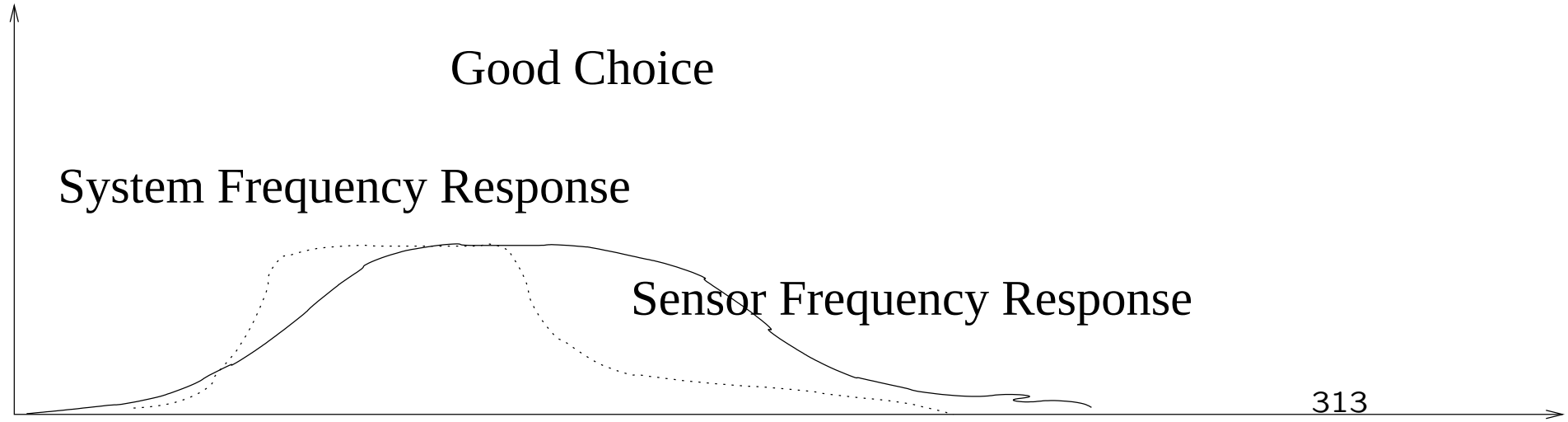
- How do you select appropriate sensors for the problem at hand?

- Some obvious general rules:
  - the lower the sensor noise characteristics, the more accurate the estimates (narrow sensor noise variance)
  - the more the merrier, i.e., the more sensors the better
  - maximize the number of directly observable states

Kalman Filter Theory vs. Reality

- Some less obvious general rules:
  - short-term / long-term complementarity (e.g, GPS vs. INS)
  - frequency response of sensor should match that of system
  - noise spectrum of sensor should be maximally disjoint from that of system


Poor Choice
System Frequency Response
Sensor Frequency Response


Good Choice
System Frequency Response
Sensor Frequency Response

Kalman Filter Theory vs. Reality

- Choose sensors to match states.
  E.g., consider the state model

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

  with the measurement model

$$z = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + v$$

  Here x_1 is observable but x_2 is not.

Kalman Filter Theory vs. Reality: Sensor Selection

- Examine the H matrix that you would use with this sensor!

- If it renders some states unobservable, then consider adding additional sensors for those states, or replacing this sensor with one that is more favorable.

Filter Divergence

- Kinds of Divergence
- Causes and Remedies

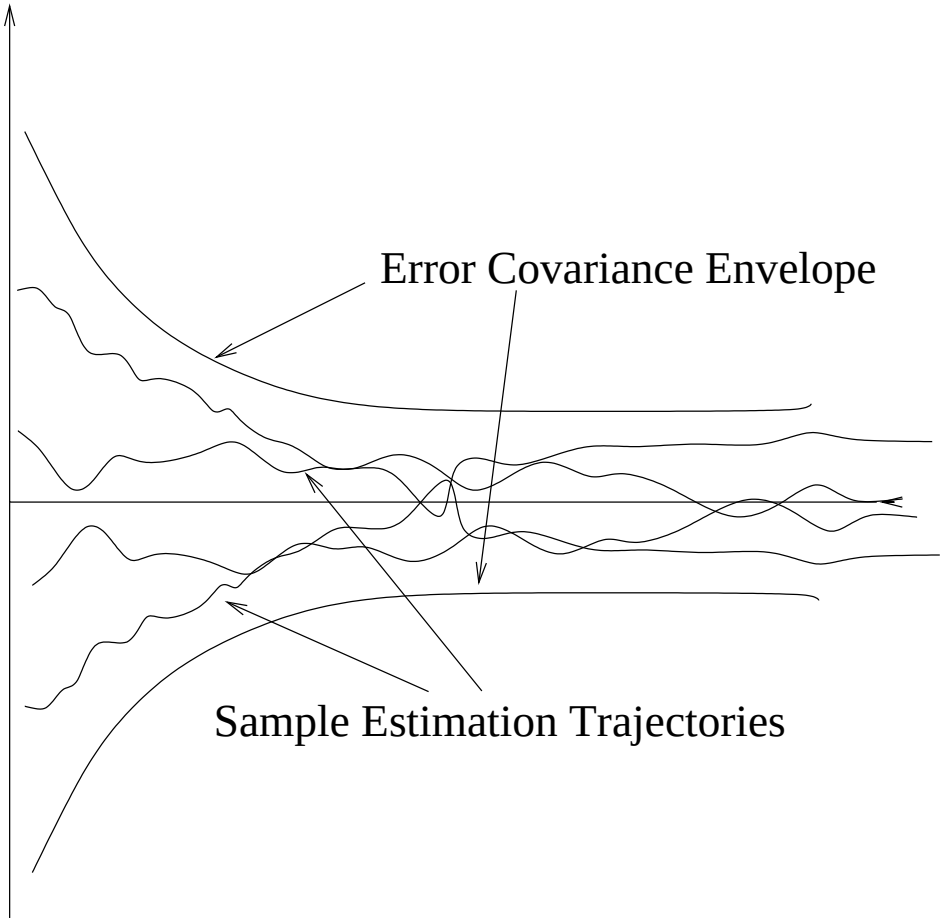
Kinds of Divergence

- Mean estimation error is supposed to converge to 0
  - $\tilde{x} = \hat{x}^{+}(k) - x(k)$
  - $\mathsf{E}\left[\tilde{x}\right] \to 0$
  - Estimate is unbiased: $\mathsf{E}\left[\hat{x}_k\right] = x_k$
- Covariance matrix for estimation error is same as state estimation error covariance matrix: $P_k^{+} = \mathsf{E}\left[\tilde{x}^{+}(k)[\tilde{x}^{+}(k)]^{\top}\right]$

Kinds of Divergence

- For optimal filter, P will indicate convergence or divergence.
  - One common form of divergence due to round-off will send P to zero

- For suboptimal filters, P will not necessarily indicate divergence
  - P may seem to indicate nominal steady-state values and yet the state estimate may go diverge from the true value


Error Covariance Envelope
Sample Estimation Trajectories

Kinds of Divergence

- Going to sleep: error covariance becomes so small that measurements are no longer believed by the filter
- Loss of symmetry in P
- Loss of positive definiteness: negative numbers on the diagonal!

Numerical Stability

- System stability: Bounded input results in bounded output
- Numerical stability: small errors at each computation step do not grow without bound
- Matrix Conditioning

Matrix Conditioning

- Ill-Conditioned vs. Well-Conditioned

- Condition number of matrix:

$$\text{cond}(M) = \frac{\lambda_{\text{max}}}{\lambda_{\text{min}}}$$

  where λ_{max} is maximum eigenvalue of M, while λ_{max} is the minimum eigenvalue of M.

Matrix Conditioning

- If cond(M) ≈ 1, then M is well-conditioned for matrix inversion

- If cond(M) $>> 1$, then M is ill-conditioned for inversion

Round-Off Errors

- Computer round-off in floating point arithmetic
- Unit round-off value is largest number ε such that $1 + \varepsilon = 1$
- In Matlab, the global symbol ”eps” stores the unit round-off value

Round-Off Errors

- Round-off errors generally plague the classical covariance update algorithm

$$P_k^+ = [I - K_k H_k] \, P_k^-$$

- Alternative, called the Joseph form, is more robust:

$$P_k^+ = [I - K_k H_k] \, P_k^- \, [I - K_k H_k]^\top + K_k R_k K_k^\top$$

This form in turn leads to the De Vries mechanization of the Kalman gain equation. To see this, first expand the Joseph form (omitting subscripts for clarity):

$$P(+) = P(-) - KHP(-) - P(-)H^\top K^\top + KHP(-)H^\top K^\top + KRK^\top.$$

Define

$$F = PH^\top \tag{47}$$

and

$$D = HF + R. \tag{48}$$

We note that D is symmetric, because

$$\begin{aligned}
D^\top &= [HF + R]^\top \\
&= [HPH^\top + R]^\top \\
&= [HPH^\top]^\top + R^\top \\
&= [H^\top]^\top P^\top H^\top + R^\top \\
&= HPH^\top + R \qquad \text{(because } P \text{ and } R \text{ are symmetric)} \\
&= HF + R \\
&= D.
\end{aligned}$$

The gain in standard Kalman gain equation

$$K_k = P_k^- H_k^\top \left[H_k P_k^- H_k^\top + R_k \right]^{-1}$$

can be written as

$$K = FD^{-1}.$$

Using Eqs. (47) and (48), we compute

$$\begin{aligned} P(+) &= P(-) - KF^\top - FK^\top + KDK^\top \\ &= P(-) - K\left[F^\top - \frac{DK^\top}{2}\right] - \left[F - \frac{KD}{2}\right] K^\top. \end{aligned}$$

Letting $G = F - \dfrac{KD}{2}$, we obtain

$$P(+) = P(-) - GK^\top - \left[GK^\top\right]^\top. \tag{49}$$

The De Vries mechanization implements Eq. (49) by subtracting the ij^{th} element of $GK^\top$ from both the ij^{th} and ji^{th} elements of $P(-)$. The De Vries form of the covariance update equation is valid for any arbitrary gain matrix K, not just the optimal gains computed from Eqs. (47) and (48).

Matrix Inversion

- Matrix inversion is always risky due to potential ill-conditioning
- One way to avoid this is to process measurements sequentially as scalars
- Or at least as low-dimensional matrices

UD Factorization

- Based on Cholesky Decomposition

- Unit Triangular Matrices: U has 1's on the diagonal and all 0's below the diagonal. E.g.,

$$U = \begin{bmatrix} 1 & 5 & 7 \\ 0 & 1 & 3 \\ 0 & 0 & 1 \end{bmatrix}$$

UD Factorization

- Factor $HPH^\top + R$ as $UDU^\top$, where U is unit upper triangular and D is diagonal.

- This then allows us to "invert" $HPH^\top + R$ by solving the equation unit upper triangular equation $UDU^\top X = H$ for the matrix X, which will yield $X = \left[HPH^\top + R\right]^{-1} H$

- This last expression is used on the covariance update equation in KH

Exercises

1. Let $v = [1 - 2]^\top$. Compute the positive definite, symmetrix matrix $M = vv^\top$

2. Use the following algorithm (Grewal & Andrews) to factor M into $UDU^\top$ form:

```
for j=m:-1:1
    for i=j:-1:1
        covar=P(i,j);
        for k=j+1:m
            covar=covar-U(i,k)*D(k,k)*U(j,k);
        end
        if i==j
            D(j,j)=covar;
            U(j,j)=1;
        else
            U(i,j)=covar/D(j,j);
        end
    end
end
```

Segment 10
The Navigation Problem and Reference Frames
Topics

- Basic Navigation Problem
- Earth Referenced Navigation
- Coordinate Frame Conversion
- Basic Strapdown Navigation Equations

Basic Navigation Problem

- Where am I?
- Where am I headed?
- How fast am I going?

Basic Navigation Problem

- What's my position?
- What's my heading and attitude?
- What's my velocity?

Solution in Inertial Space

- In inertial space, it's no problem!

- Just integrate the acceleration once to get velocity, twice to get position.

$$v(T) = v(0) + \int_0^T \frac{d^2 r}{dt^2} dt$$

$$r(T) = r(0) + \int_0^T v(T) dt$$

- And integrate the angular rate once to get attitude.

Solution in Inertial Space

- Solution easy in inertial space because ...
  - inertial instruments sense acceleration and rotational rate with respect to inertial coordinate frame
  - inertial frame attached to sensor platform
  - Note: You still need to rotate accelerations from body to inertial coordinates prior to integration (strapdown)
- But we don't necessarily want the answer to our questions expressed in terms of inertial coordinates!

Earth Referenced Navigation

- Terrestial navigation seeks to answer nav questions with respect to earth
  - Global earth reference
  - Local or relative navigation
- Complications:
  - Earth surface is curved (ellipsoid)
  - Earth is turning (can usually ignore motion about the Sun for terrestrial nav)

Coordinate Frames

- Inertial
- Earth Fixed
- Local Level
- Body

Coordinate Frames

- Technically speaking, all frames are earth-centered
- Practically speaking, we think of the Body frame as centered in the body of the vehicle (usually at center of gravity).
- Local Level is thought to be located at surface of earth directly beneath the vehicle.

Inertial Reference Frames

- Non-Rotating
- Non-Accelerating
- Fixed or moving uniformly with respect to another inertial frame (fixed stars)

Inertial Reference Frames

- Nav problem "easy" in inertial frames
- Integrate gyro rates once to get attitude (rotational motion is absolute)
- Rotate accelerometer outputs to inertial frame using attitude matrix, then integrate to get inertial velocity; integrate velocity to get position.
- Would still have to contend with instrument imperfections.

Inertial Reference Frame Integrations

(50) $$\ddot{r}(t) = a_{\mathsf{sf}} + \text{other external forces}$$

(51)
$$v(T) = \dot{r}(T) = v(0) + \int_0^T a_{\mathsf{sf}} dt + \int_0^T (\text{other external forces}) dt$$

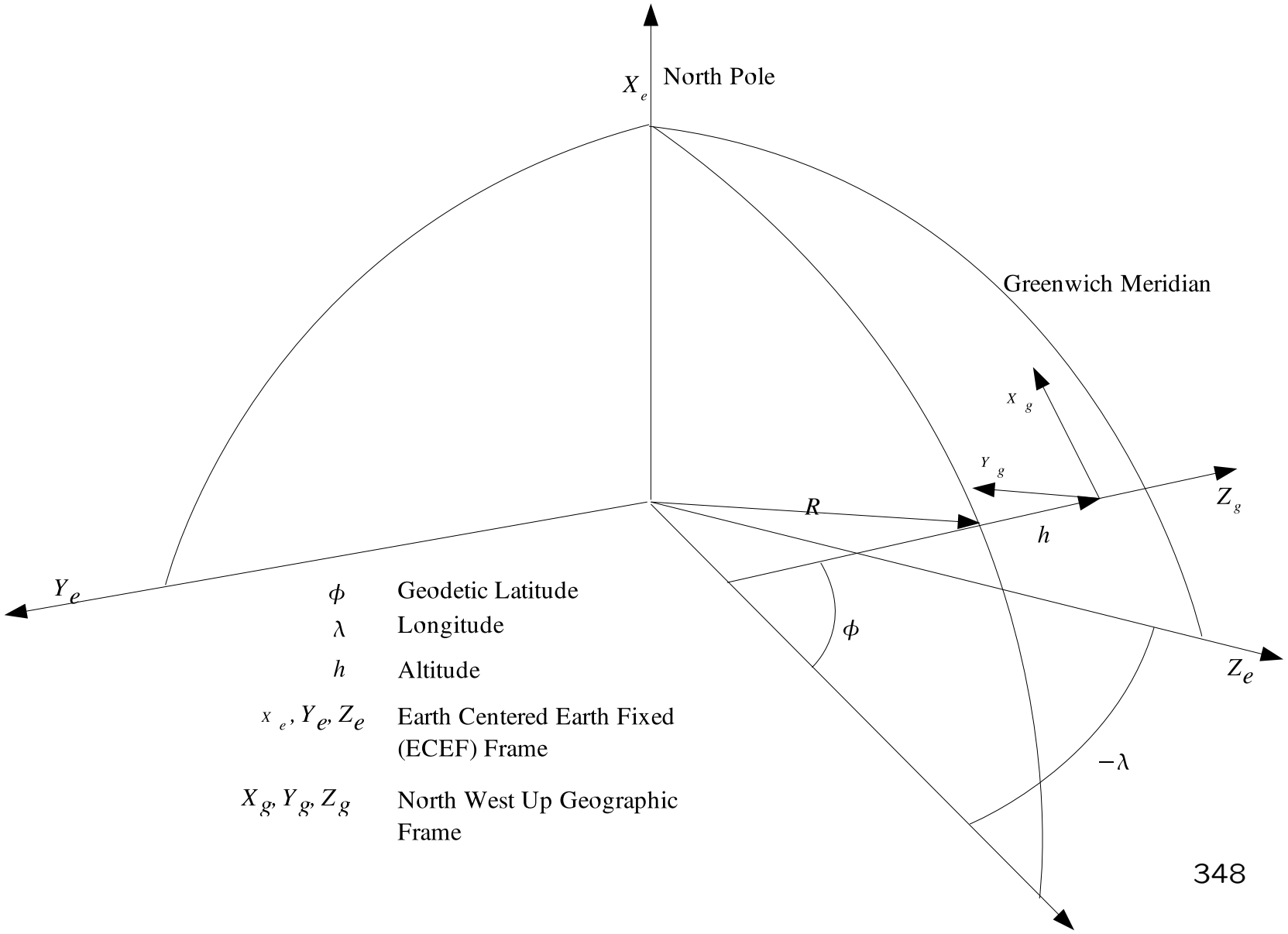
(52) $$r(T) = r(0) + \int_0^T v dt$$

Earth Fixed = ECEF = Earth Centered Earth Fixed

- Rotating frame, i.e., non-inertial
- Cartesian (X, Y, Z in meters)
- Spherical (latitude ϕ, longitude λ, altitude h)
  - geodetic

Local Level

- Tangent to earth's surface but translated to center of earth.
- Vertical Z-axis is normal to earth's surface
- X, Y-axes are level
- Several kinds: Y north-slaved, wander (0 transport rate) or free azimuth
- Y makes wander angle α with respect to true north (ENU=East, North, Up).



ϕ	Geodetic Latitude
λ	Longitude
h	Altitude
X_e, Y_e, Z_e	Earth Centered Earth Fixed (ECEF) Frame
X_g, Y_g, Z_g	North West Up Geographic Frame

Body Frame

- Attached to vehicle

- For aircraft:
  - X out the nose
  - Y out the right wing
  - Z down

Coordinate Frame Conversion

- All frames are earth-centered, hence differ only by rotation
- Rotation represented by direction cosine matrices (DCM)
  - Represents the angles made by each of the basis vectors of one frame with those of the other
- C_A^B = DCM transformation from frame A to frame B

Coordinate Frame Conversion

- "Direction Cosine" comes from dot product:

$$\langle x, y \rangle = x \cdot y = \|x\| \, \|y\| \cos \theta_{xy}$$

  where θ_{xy} is the angle between the two vectors x and y.

- If x and y are unit vectors, then $\|x\| = \|y\| = 1$ and we have:

$$\langle x, y \rangle = x \cdot y = \cos \theta_{xy}$$

Coordinate Frame Conversion

- If $U = \{u_1, u_2, u_3\}$ and $V = \{v_1, v_2, v_3\}$ are two sets of orthonormal basis vectors, then the DCM relating one frame to the other is given by:

$$C_U^V = \begin{bmatrix} \langle u_1, v_1 \rangle & \langle u_1, v_2 \rangle & \langle u_1, v_3 \rangle \\ \langle u_2, v_1 \rangle & \langle u_2, v_1 \rangle & \langle u_2, v_3 \rangle \\ \langle u_3, v_1 \rangle & \langle u_3, v_2 \rangle & \langle u_3, v_3 \rangle \end{bmatrix}$$
$$= \begin{bmatrix} \cos\theta_{11} & \cos\theta_{12} & \cos\theta_{13} \\ \cos\theta_{21} & \cos\theta_{22} & \cos\theta_{23} \\ \cos\theta_{31} & \cos\theta_{32} & \cos\theta_{33} \end{bmatrix}$$

Coordinate Frame Conversion

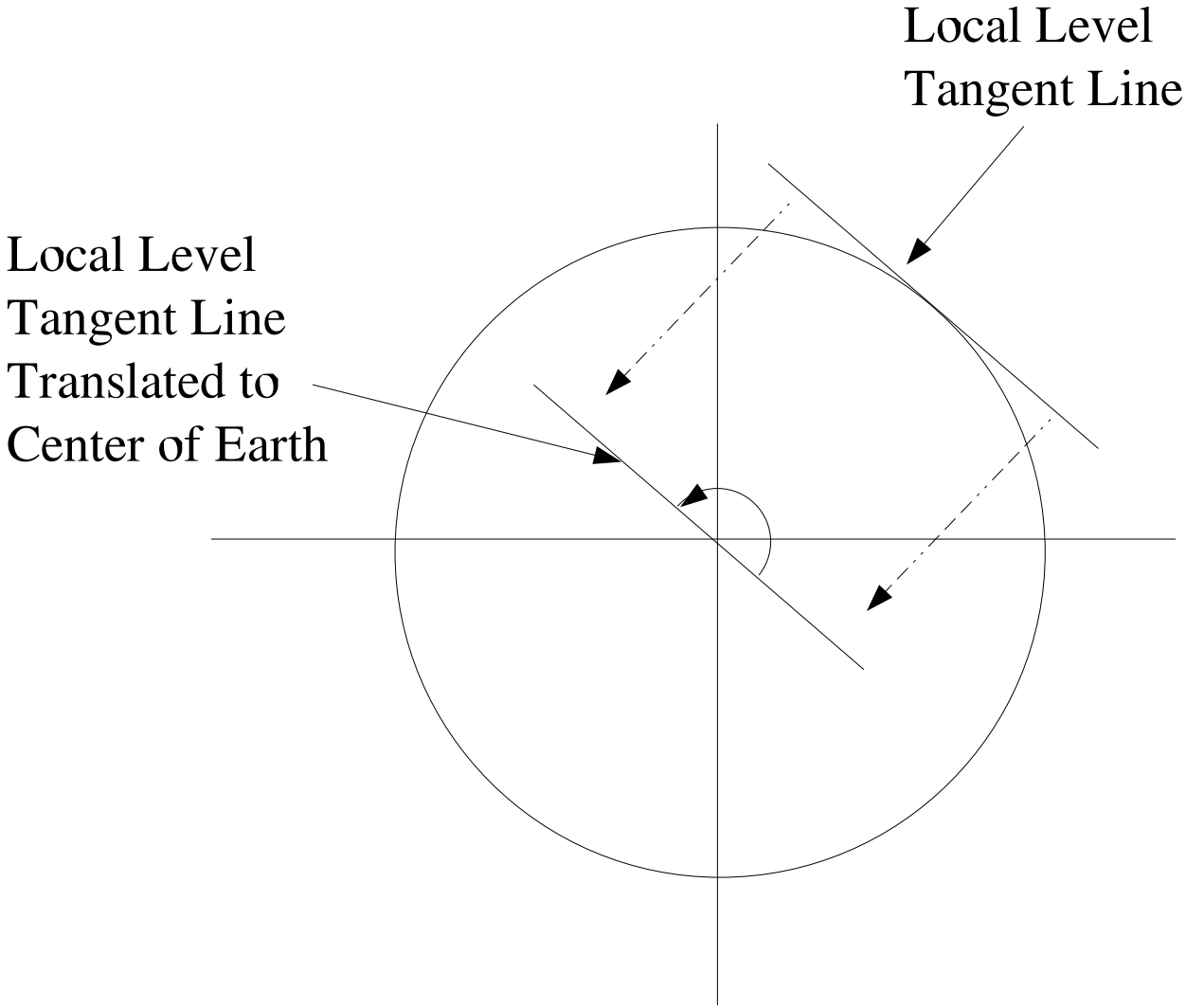
- DCMs are orthogonal matrices $\Rightarrow \left[C_A^B\right]^{-1} = \left[C_A^B\right]^\top$
- Third row (or column) is vector cross product of other two $\Rightarrow$ only six independent parameters
- DCMs satisfy: $C_B^A C_C^B = C_C^A$

Coordinate Frame Conversion: Most important DCMs

- Body to Local Level: C_B^L
  - sometimes called Body to Nav and notated C_B^N
  - yields attitude info (Euler angles for roll, pitch, yaw)
- Local Level to Earth: C_L^E
  - yields position info and also used to transform earth rate to local level

Coordinate Frame Conversion: Most important DCMs

- Visualize C_L^E's determination of position as follows:
  - As vehicle moves, local level frame changes its orienation with respect to earth's center in order to maintain local level with respect to earth's surface
  - If you think of local level frame as attached to earth's surface, you get distracted by the displacement of the frame
  - If you move local level frame to center of earth, the displacement disappears and all that remains are the rotations


Local Level
Tangent Line
Local Level
Tangent Line
Translated to
Center of Earth

Strapdown Navigation: Strapdown vs. Gimbaled

- Gimbaled INS:
  - Gyro stabilized platform
  - Gimbal assembly and torque motors
  - Resolvers, slip rings, bearings, etc. to control platform
  - Goal: Maintain stable, locally level or inertial reference frame

Strapdown Navigation: Strapdown vs. Gimbaled

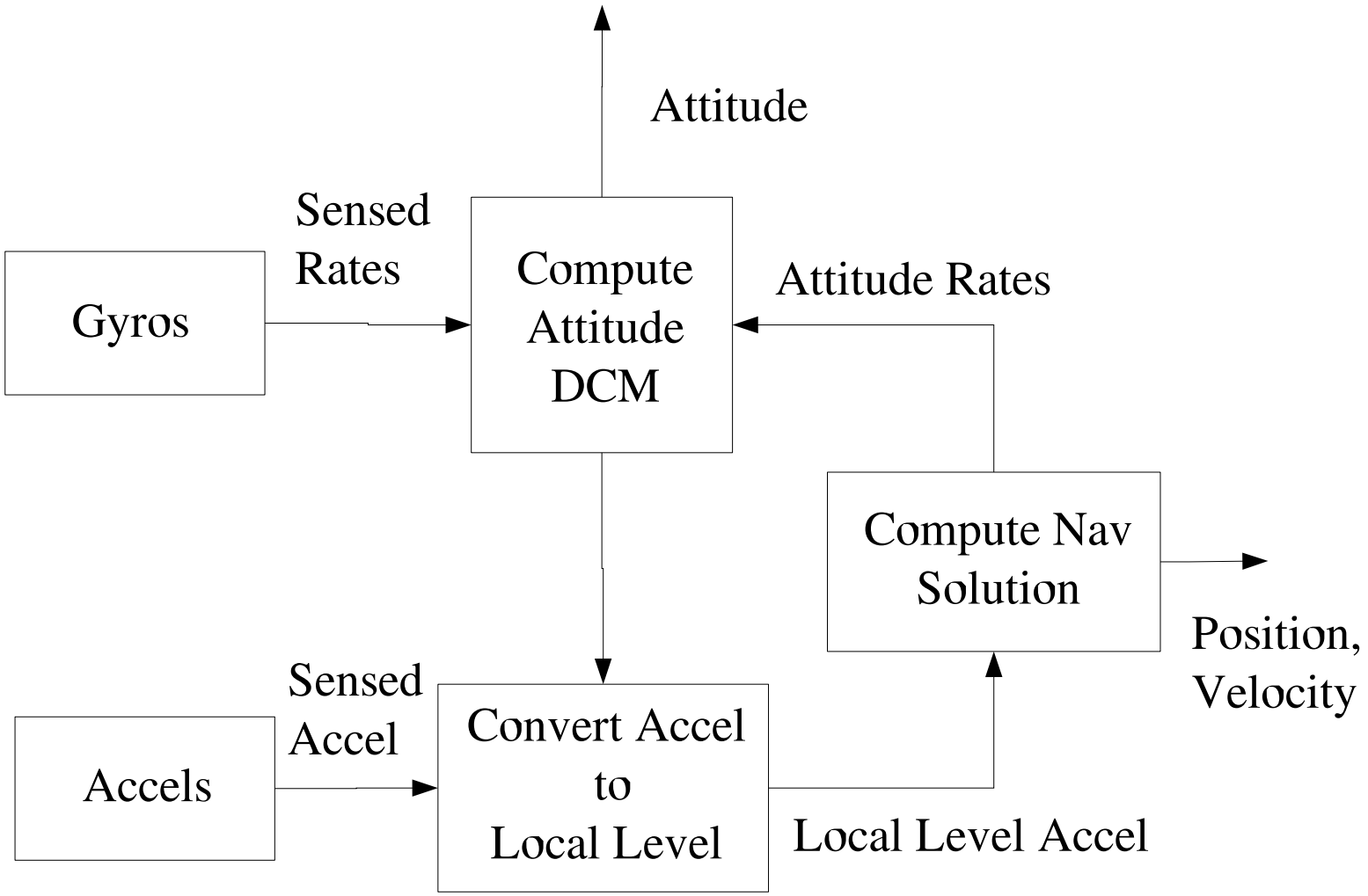
- Gimbaled INS Issues
  - Reliability and Cost: Lots of moving parts!
  - Body rate and acceleration information not directly available
  - Rate signals are generally noisy and have low bandwidth

Strapdown Navigation: Strapdown vs. Gimbaled

- Strapdown INS:
  - Inertial instruments strapped to body frame
  - Requires local level accelerations to be computed using direction cosine matrix C_B^L
  - C_B^L derived from gyro-sensed body rates
  - Strapdown systems can be implemented with either rate gyros or attitude gyros

Strapdown Navigation: Strapdown vs. Gimbaled

- Strapdown INS Advantages
  - Immediate detection of body rates and accelerations
  - Fewer moving parts
  - Potential redundancy advantages
- Gimbaled systems are nearly obsolete



Strapdown INS Block Diagram

Basic Strapdown Navigation Equations

- Need to compute attitude, velocity, and position
- I.e., need to compute C_B^L and C_L^E
- These DCMs are dynamic quantities
- C_B^L determines attitude and can be rapidly varying
- C_L^E determines position and is slowly varying (relative to attitude dynamics)

Rotating Coordinate Frames

- We first need to understand how an arbitrary vector and its time derivatives in one coordinate frame are expressed in another frame that is rotating with respect to the first.

- Let A and B denote two coordinate frames and assume that B is rotating with respect to A.

- We use the notation ω_{AB}^{A} to express the direction and magnitude of the rotation vector as seen in the A frame.

- Every rotation vector can also be expressed as a skew symmetric matrix.

- If ω is a rotation vector (without mentioning the frames involved), then

$$\Omega = \begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix}.$$

Rotating Coordinate Frames

- Since multiplication of a vector by the skew symmetric matrix equivalent is the same thing as left multiplying the cross product of the original vector, we can write:

$$\Omega v = \omega \times v$$

- Therefore the notation $[\,\omega\times\,] = \Omega$ is common*

*One must be careful in making this identification since left matrix multiplication is associative while left cross product multiplication is not.

Rotating Coordinate Frames

- Let v be an arbitrary vector in 3-space.
- Let $\left\{e^1_A, e^2_A, e^3_A\right\}$ be orthonormal basis vectors in the A frame
- Let $\left\{e^1_B, e^2_B, e^3_B\right\}$ be orthonormal basis vectors in the B frame*.

*We reverse the roles of superscripts and subscripts for basis vectors so that we may employ the Einstein summation convention on occasion.

Rotating Coordinate Frames

- Thus, we can write v as $\sum_{i=1}^{3} v_i^A e_A^i$ and also $\sum_{i=1}^{3} v_i^B e_B^i$.

- This is one and the same vector expressed in two different coordinate frames*.

*A vector is a linear combination of basis vectors and not necessarily an ordered triple of real numbers.

Rotating Coordinate Frames

- Note that, with respect to an arbitrary inertial reference frame, we have for the time derivative:

$$
\begin{aligned}
v &= \sum_{i=1}^{3} v_i^A e_A^i = \sum_{i=1}^{3} v_i^B e_B^i \\
\Rightarrow \dot{v} &= \frac{d}{dt}\left[\sum_{i=1}^{3} v_i^A e_A^i\right] = \frac{d}{dt}\left[\sum_{i=1}^{3} v_i^B e_B^i\right] \\
\Rightarrow & \sum_{i=1}^{3}\left(\dot{v}_i^A e_A^i + v_i^A \dot{e}_A^i\right) = \sum_{i=1}^{3}\left(\dot{v}_i^B e_B^i + v_i^B \dot{e}_B^i\right)
\end{aligned}
$$

Rotating Coordinate Frames

- In the special case that their respective coordinate frames are not rotating relative to one another, each $\dot{e}^i_A = 0 = \dot{e}^i_B$.

- Nevertheless, regardless of rotation, we must always have:

(53) $$\sum_{i=1}^{3} \dot{v}^A_i e^i_A = \sum_{i=1}^{3} \dot{v}^B_i e^i_B.$$

(Note that, in general, $\dot{v}^A_i \neq \dot{v}^B_i$.)

Rotating Coordinate Frames

- Let frame B rotate with respect to frame A at a constant rotational rate ω_{AB}^{A} as expressed in A frame coordinates.

- Since B is not rotating with respect to itself, let us agree to label the quantity in Eq. (53) as $\dot{v}^{B}$.

- We'll assume that A is an inertial frame and thus $\dot{v} = \dot{v}^{A}$.

- When viewed from the A frame, however, it is not true that each of the $\dot{e_i}^B$ is zero, since these unit vectors are slaved to the B frame and B is rotating with respect to A at rate ω_{AB}^A.

- Therefore, we have:

$$\begin{aligned} \dot{v}^A &= \sum_{i=1}^{3} \dot{v}_i^A e_A^i + \sum_{i=1}^{3} v_i^B \dot{e}_B^i \\ &= \sum_{i=1}^{3} \dot{v}_i^B e_B^i + \sum_{i=1}^{3} v_i^B \dot{e}_B^i \quad \text{(by Eq. (53))} \\ &= \dot{v}^B + \sum_{i=1}^{3} v_i^B \dot{e}_B^i. \end{aligned} \tag{54}$$

- The last sum on the right side of Eq. (54) can be rewritten as the cross product $\omega_{AB}^{A} \times v$, because $\dot{e}_B^i = \omega_{AB}^{A} \times e_B^i$, and

$$\begin{aligned}\sum_{i=1}^{3} v_i^B \dot{e}_B^i &= \sum_{i=1}^{3} v_i^B \omega_{AB}^{A} \times e_B^i \\ &= \omega_{AB}^{A} \times \sum_{i=1}^{3} v_i^B e_B^i \\ &= \omega_{AB}^{A} \times v.\end{aligned}$$

- We thus arrive at the all-important Coriolis Equation which is used often in navigation analysis:

(55) $$\dot{v}^A = \dot{v}^B + \omega^A_{AB} \times v^B.$$

Alternatively, we can express Eq. (55) in skew-symmetric matrix form using $\Omega_{AB} = [\,\omega_{AB}\times\,]$:

$$\dot{v}^A = \dot{v}^B + \Omega^A_{AB} v^B. \tag{56}$$

Note that we must use superscripts on both Ω_{AB} and v as soon as we make a commitment to a particular coordinate frame representation.*.

*Since matrices are defined in terms of their action on basis vectors, it is appropriate to specify the coordinate system. On the other hand, if Ω_{AB} is taken to be a linear transformation, then superscripts are not necessary.

Strapdown Navigation Equations

- Three differential equations
  - Equation for body to local level transformation C_B^L
  - Equation for velocity (uses C_B^L)
  - Equation for local level to earth for position DCM C_L^E

Strapdown Navigation Equations

- Two Algebraic Equations
  - Equation for converting velocity to transport rate (to be explained)
  - Equation for converting earth rate to local level coordinates

Strapdown Navigation Equations
The Three Differential Equations

(57) $$\dot{C}_B^L = C_B^L \Omega_{IB}^B - \left(\Omega_{IE}^L + \Omega_{EL}^L\right) C_B^L$$

(58) $$\dot{v}^L = C_B^L a_{\mathsf{sf}}^B + g' - \left(2\Omega_{IE}^L + \Omega_{EL}^L\right) v^L$$

(59) $$\dot{C}_L^E = C_L^E \Omega_{EL}^L$$

Strapdown Navigation Equations
The Two Algebraic Equations

(60) $$\omega_{EL}^{L} = \frac{1}{r} u_r^L \times v^L \text{ (wander azimuth } \rho_z = 0)$$

(61) $$\omega_{IE}^{L} = \left[C_L^E\right]^\top \omega_{IE}^{E}$$

Strapdown Navigation Equations

The C_B^L Equation (57)

$$\dot{C}_B^L = C_B^L \Omega_{IB}^B - \left(\Omega_{IE}^L + \Omega_{EL}^L\right) C_B^L$$

- All the Ω matrices represent rotation vector cross products in skew-symmetric matrix format
  - Ω_{IB}^B is rotational rate of aircraft in body coordinates
  - Ω_{IE}^L is earth rate in local level coordinates
  - Ω_{EL}^L is transport rate in local level coordinates

Strapdown Navigation Equations: Detour into Cross Product Matrices

- The cross product of $\omega = \begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \end{bmatrix}$ with any other vector v is the same as multiplying v on the left by $\Omega = \begin{bmatrix} 0 & -\omega_3 & \omega_2 \\ \omega_3 & 0 & -\omega_1 \\ -\omega_2 & \omega_1 & 0 \end{bmatrix}$

- $\omega \times v = \Omega v$

Strapdown Navigation Equations

The C_B^L Equation (57)

$$\dot{C}_B^L = C_B^L \Omega_{IB}^B - \left(\Omega_{IE}^L + \Omega_{EL}^L\right) C_B^L$$

- Transport rate is the rotational rate of the aircraft due to motion over the surface of the earth. Is a function of velocity.

- Ω_{IB}^B is total inertial angular rate sensed by gyros and includes effects of transport and earth rates.

- Hence transport and earth rate effects have to be subtracted off.

Strapdown Navigation Equations

The Strapdown Velocity Equation (58)

$$\dot{v}^L = C_B^L a_{\text{sf}}^B + g' - \left(2\Omega_{IE}^L + \Omega_{EL}^L\right) v^L$$

- Sometimes referred to as the velocity rate equation

- a_{sf}^B is specific force sensed by accelerometers in body coordinates (because the accels are strapped to the body – hence the term strapdown)

Strapdown Navigation Equations
The Transport Rate Equation (59)

$$\dot{C}_L^E = C_L^E \Omega_{EL}^L$$

- Connected to velocity rate equation by the first algebraic equation (60):

$$\omega_{EL}^L = \frac{1}{r} u_r^L \times v^L$$

- Eq. (59) Sometimes referred to as position rate equation, because C_L^E directly implies the vehicle position

Strapdown Navigation Equations
Position Rate / Transport Rate Equations (Geographic)

- North transport rate:

(62) $$\rho_N = \frac{v_E}{R_e}\left[1 - \frac{h}{R_e} - e\sin^2\phi\right]$$

- East transport rate:

(63) $$\rho_E = -\frac{v_N}{R_e}\left[1 - \frac{h}{R_e} - e\left(1 - 3\cos^2\phi\right)\right]$$

where e is ellipticity (approx. $\frac{1}{298}$) and R_e is mean equatorial earth radius

Strapdown Navigation Equations
Position Rate / Transport Rate Equations (Geographic)

- Latitude Rate:

(64) $$\dot{\phi} = -\rho_E$$

- Longitude Rate:

(65) $$\dot{\lambda} = \rho_N \sec \phi$$

Strapdown Navigation Equations

- Geographic (north-slaved) mechanization has singularities at poles

- Two other approaches to local level coordinates:
  - Free azimuth: vertical component of transport rate ρ_z= additive inverse of vertical component of earth rate
  - Wander Azimuth: vertical component of transport rate ρ_z = 0

Strapdown Navigation Equations
Position Rate / Transport Rate Equations (Wander and Free Azimuth)

- X Transport Rate:

  (66)
  $$\rho_x = -\frac{v_y}{R_e}\left[1 - \frac{h}{R_e} - e\left(1 - 3d_{22}^2 - d_{21}^2\right)\right] - \frac{v_x}{R_e}\left(2d_{21}d_{22}\right)e$$

- Y Transport Rate:

  (67) $$\rho_y = \frac{v_x}{R_e}\left[1 - \frac{h}{R_e} - e\left(1 - 3d_{21}^2 - d_{22}^2\right)\right] + \frac{v_y}{R_e}\left(2d_{21}d_{22}\right)e$$

Strapdown Navigation Equations
Position Rate / Transport Rate Equations (Wander and Free Azimuth)

- Azimuth Rate:

$$\rho_z = \begin{cases} -\Omega_e d_{23}, & \text{if free azimuth;} \\ 0, & \text{if wander azimuth} \end{cases}$$

- In equations above, the matrix $\left[d_{ij}\right]$ is the direction cosine matrix C_L^E from local level (L) frame to earth centered earth fixed (E) frame.

Exercises

1. Compute direction cosine matrix C_U^V, where

$$U = \{(1,0,0),(0,1,0),(0,0,1)\}$$

and

$$V = \{(-1,0,0),(0,1,0),(0,0,-1)\}$$

2. Suppose you are starting at the equator and travelling due north, and your velocity is 5 m/sec. Compute the transport rates ρ_n and ρ_e associated with this velocity and heading. You may use the following constants: $\Omega_e = 7.2921151467 \times 10^{-5}$ rad/sec and $R_e = 6,378,137$ meters

Day 3, Segment 2 (Continued)
Inertial Error Dynamics
Topics

- Imperfect Inertial Instruments
- From Strapdown Equations to Error Equations
- Initialization: Alignment

Imperfect Inertial Instruments

- Accelerometers and Gyros have inherent imperfections
- Bias, scale factor, g^2 terms, noise, random walks, etc.
- Inertial sensor clusters also have errors: misalignments
- Many error sources are calibrated out using rate tables during ATP
- Residual effects will influence strapdown nav equations

Imperfect Inertial Instruments

- Strapdown nav equations very sensitive to gyro errors
- Gyro biases yield erroneous C_B^L matrix which in turn causes erroneous accelerations to be computed in local level frame
- Thus, gyro errors affect not only attitude, but also position and velocity

Simple Inertial Error Models

- Gyros

$$\epsilon\omega_x = \Delta B_{gx} + \Delta K_{gx}\omega_x + \theta_{gxy}\omega_y + \theta_{gxz}\omega_z$$

$$\epsilon\omega_y = \Delta B_{gy} + \Delta K_{gy}\omega_y + \theta_{gyz}\omega_z + \theta_{gyx}\omega_x$$

$$\epsilon\omega_z = \Delta B_{gz} + \Delta K_{gz}\omega_z + \theta_{gzx}\omega_x + \theta_{gzy}\omega_y$$

where ΔB_{gi} is the ith gyro bias, ΔK_{gi} the ith gyro scale factor, and θ_{gij} is the misalignment of the ith gyro axis toward the jth gyro axis.

Simple Inertial Error Models

- Accelerometers

$$\epsilon a_x = \Delta B_{ax} + \Delta K_{ax} a_x + \theta_{axy} a_y + \theta_{axz} a_z$$

$$\epsilon a_y = \Delta B_{ay} + \Delta K_{ay} a_y + \theta_{ayz} a_z + \theta_{ayx} a_x$$

$$\epsilon a_z = \Delta B_{az} + \Delta K_{az} a_z + \theta_{azx} a_x + \theta_{azy} a_y$$

where ΔB_{ai} is the ith accelerometer bias, ΔK_{ai} the ith accelerometer scale factor, and θ_{aij} is the misalignment of the ith accelerometer axis toward the jth accelerometer axis.

Bias Models

- Bias in both types of instruments can be further broken down into several components:
  - fixed bias – constant from turn-on to turn-on – should be calibrated out
  - bias stability – varies from turn-on to turn-on – can't be calibrated out, but can be characterized in terms of variance – can model as random constant
  - bias drift – usually modeled as random walk

Scale Factor Models

- Scale factor can also be modeled as a sum of three components
- Often in nav filters, scale factor residual is modeled as a random constant
- Important to keep in mind that the contribution of scale factor error is a function of input (rate or acceleration)

Higher Order Models

- In expanding the output error as a Taylor series function of input, bias, scale factor, and misalignment show up as zero and first order terms.

- Higher order terms are also present, especially second order terms.

- In some calibration and alignment filters, such higher order effects are modelled

- SNU-84-1 includes error budgets associated with all the combined, higher order effects.

Taylor Series of Two Variables:

$$\begin{aligned} f(x,y) = f(x_0,y_0) &+ f_x(x_0,y_0)(x-x_0) + f_y(x_0,y_0)(y-y_0) \\ &+ \frac{1}{2!}(f_{xx}(x_0,y_0)(x-x_0)^2 \\ &+ f_{xy}(x_0,y_0)(x-x_0)(y-y_0) \\ &+ f_{yy}(x_0,y_0)(y-y_0)^2) + \cdots \end{aligned}$$

Taylor Series provides basis for all error analysis.

From Strapdown Equations to Error Equations

- Nav equations require perfect inertial instruments
- Instruments are not perfect
- Inertial instrument error sources (bias, scale factor errors, etc.) generate erroneous rate and acceleration inputs to the nav equations
- Need to figure out the effects on the outputs of these equations

Strapdown Error Equations

- Nature of Errors
- Error Operator
- Error Equations

Strapdown Error Equations: Nature of Errors

- Errors propagate according to derived equations

- Errors are generally small, so approximating assumptions help simplify the math

  - linearize otherwise non-linear dynamics

  - small angle assumptions: $\sin\theta \approx \theta$

Strapdown Error Equations: Error Operator

If A is any quantity, we define error operator as follows:

$$\delta A = \widehat{A} - A, \tag{68}$$

where A is the true quantity of interest and $\widehat{A}$ is an apparent quantity. The latter could be a measured value, and estimated value, or some computed value. This means that

$$\widehat{A} = A + \delta A$$

i.e., the apparent value is the true value plus some error.

Strapdown Error Equations: Error Operator

Error operator satisfies:

(69) $$\delta(AB) = \delta A \cdot B + A \cdot \delta B$$

Proof:

$$\begin{aligned}
\delta(AB) &= \widehat{AB} - AB \\
&= \hat{A}\hat{B} - AB \\
&= (A + \delta A)(B + \delta B) - AB \\
&= AB + A \cdot \delta B + \delta A \cdot B + \delta A \cdot \delta B - AB \\
&= \delta A \cdot B + A \cdot \delta B
\end{aligned}$$

since we may assume that $\delta A \cdot \delta B \approx 0$.

Strapdown Error Equations

Applying these to the three differential equations, yields three error equations:

(70)

$$\delta\dot{C}_B^L = \delta C_B^L \Omega_{IB}^B + C_B^L \delta\Omega_{IB}^B - \left(\delta\Omega_{IE}^L + \delta\Omega_{EL}^L\right) C_B^L - \left(\Omega_{IE}^L + \Omega_{EL}^L\right) \delta C_B^L$$

(71)

$$\delta\dot{v}^L = \delta C_B^L a_{\mathsf{sf}}^B + C_B^L \delta a_{\mathsf{sf}}^B + \delta g' - \left(2\delta\Omega_{IE}^L + \delta\Omega_{EL}^L\right) v^L - \left(2\Omega_{IE}^L + \Omega_{EL}^L\right) \delta v^L$$

(72)

$$\delta\dot{C}_L^E = \delta C_L^E \Omega_{EL}^L + C_L^E \delta\Omega_{EL}^L$$

Introducing Small Angular Rotations for Tilt and Position

- $\widehat{C}_B^L$, the computed (i.e., apparent) version of C_B^L is related additively to the true version by the error δC_B^L

$$\widehat{C}_B^L = C_B^L + \delta C_B^L$$

- It is also related to C_B^L by a small direction cosine matrix relating the local level frame to the computer frame $\widehat{L}$:

$$\widehat{C}_B^L = C_{\widehat{L}}^{\widehat{L}} C_B^L$$

Introducing Small Angular Rotations for Tilt and Position

- Substituting the second equation into the first allows us to solve for δC_B^L:

$$\begin{aligned}\delta C_B^L &= \widehat{C}_B^L - C_B^L \\ &= C_L^{\widehat{L}} C_B^L - C_B^L \\ &= (C_L^{\widehat{L}} - I) C_B^L\end{aligned}$$

  Now, let's introduce $\Phi = I - C_L^{\widehat{L}}$ to obtain:

$$\delta C_B^L = -\Phi C_B^L$$

Introducing Small Angular Rotations for Tilt and Position

- Using small angle approximations, $C_L^{\widehat{L}}$ can be written in the form

$$C_L^{\widehat{L}} = \begin{bmatrix} 1 & \phi_z & -\phi_y \\ -\phi_z & 1 & \phi_x \\ \phi_y & -\phi_x & 1 \end{bmatrix}.$$

  Thus, $\Phi = I - C_L^{\widehat{L}}$ is seen to be a skew-symmetric rotation matrix:

$$\Phi = \begin{bmatrix} 0 & -\phi_z & \phi_y \\ \phi_z & 0 & -\phi_x \\ -\phi_y & \phi_x & 0 \end{bmatrix}.$$

Introducing Small Angular Rotations for Tilt and Position

- Φ is therefore equal to the cross product operator associated with a vector $\phi^L = \begin{bmatrix} \phi_x^L & \phi_y^L & \psi_z^L \end{bmatrix}^\top$. I.e.,

$$\Phi = \left[\, \phi^L \times \, \right]$$

- The three components of ϕ^L will become three states in our inertial error dynamics Kalman filter.

- ϕ_x and ϕ_y are called tilts, while ψ_z is called the azimuth error.

Introducing Small Angular Rotations for Tilt and Position

- By a similar derivation, we can introduce a small angular position angle error $e^L = \begin{bmatrix} e_x^L & e_y^L & e_z^L \end{bmatrix}^\top$

- e^L is associated with δC_E^L just as ϕ^L is associated with δC_B^L:

$$\delta C_E^L = -E^L C_E^L,$$

  where E^L is the skew-symmetric matrix form of e^L.

- The three components of e^L yield the three position error states in a typical navigation Kalman filter.

Introducing Small Angular Rotations for Tilt and Position

- We are working to simplify Eq. (70):

$$\delta\dot{C}_B^L = \delta C_B^L \Omega_{IB}^B + C_B^L \delta\Omega_{IB}^B - \left(\delta\Omega_{IE}^L + \delta\Omega_{EL}^L\right) C_B^L - \left(\Omega_{IE}^L + \Omega_{EL}^L\right) \delta C_B^L$$

- We have an expression for δC_B^L.

Introducing Small Angular Rotations for Tilt and Position

- Consider the earth rate error term $\delta\Omega^L_{IE}$. Since $\Omega^L_{IE} = C^L_E \Omega^E_{IE}$ and $\delta\Omega^E_{IE} = 0$, we derive:

$$\delta\Omega^L_{IE} = \delta C^L_E \Omega^E_{IE}.$$

- But $\delta C^L_E = -E^L C^L_E$. Therefore,

$$\begin{aligned} \delta\Omega^L_{IE} &= -E^L C^L_E \Omega^E_{IE} \\ &= -E^L \Omega^L_{IE}. \end{aligned}$$

Introducing Small Angular Rotations for Tilt and Position

- For the error in transport rate, $\delta\Omega_{EL}^{L}$, using the error operator on Eq. (60), a new error terms having to do with altitude error shows up:

$$\begin{aligned}\delta\omega_{EL}^{L} &= \delta\left[\frac{1}{R}u_r^L \times v^L\right] \\ &= \frac{1}{R}u_r^L \times \delta v^L - \frac{\delta h}{R^2}\left[u_r^L \times v^L\right]\end{aligned}$$

- Here, $R = R_e + h$ for a spherical earth of radius R_e.

Introducing Small Angular Rotations for Tilt and Position

- Eq. (70) can now be rewritten as:

  (73) $\dot{\Phi}^L = -\left(\Omega^L_{IE} + \Omega^L_{EL}\right)\Phi^L - C^L_B \delta\Omega^B_{IB} + \Omega^L_{IE} E^L + \delta\Omega^L_{EL}$

  where $\delta\Omega^L_{EL}$ is the skew-symmetric matrix form of

$$\delta\omega^L_{EL} = \frac{1}{R}\left(u^L_R \times \delta v^L\right) - \frac{\delta h}{R^2}\left(u^L_R \times v^L\right)$$

Introducing Small Angular Rotations for Tilt and Position

- Similarly, the velocity equation can be rewritten:

$$\delta\dot{v}^L = C_B^L \delta a_{\mathsf{sf}}^B + a_{\mathsf{sf}}^B \times \phi^L - \left(2\omega_{IE}^L + \omega_{EL}^L\right) \times \delta v^L + \\ - \left(\delta\omega_{EL}^L - 2e^L \times \omega_{IE}^L\right) \times v^L + \frac{2g\delta h}{R_e} u_R^L + \delta g^L$$

  while the equation for the position angle error e^L itself is

(74) $$\dot{e}^L = -\Omega_{EL}^L e^L + \delta\omega_{EL}^L$$

Further Simplifications Using Telescope Pointing Angle Error

- $\psi^L = \phi^L - e^L$

- Leads to:

(75) $$\delta\dot{R}^L = \delta v^L - \Omega^L_{EL}\delta R^L$$

and

(76) $$\delta\dot{v}^L = C^L_B\delta a^B_{\mathsf{sf}} + a^B_{\mathsf{sf}} \times \psi^L - \frac{g}{R_e}\Delta R^L_H - \left(2\omega^L_{IE} + \omega^L_{EL}\right) \times \delta v^L + \\ - \left(\delta\omega^L_{EL} - 2e^L \times \omega^L_{IE}\right) \times v^L + \frac{2g\delta h}{R_e}u^L_R + \delta g^L$$

Further Simplifications Using Telescope Pointing Angle Error

- Finally, the equations for the telescope pointing angle errors are:

(77) $$\dot{\psi}^L = -\left(\Omega^L_{IE} + \Omega^L_{EL}\right)\psi^L - C^L_B \delta\omega^B_{IB}$$

Strapdown Error Equations

- Three error equations yield three vector quantities of interest
  - position error vector (3) δR derived from δC_L^E
  - velocity error vector (3) δV^L (or δv^L)
  - attitude error vector (3) ϕ_x, ϕ_y, ψ_z from δC_B^L

Strapdown Equations

- See Matlab / Simulink Models
  - Strapdown Navigation Equations: stravnav.mdl
  - Strapdown Navigation Error Equations: straperr.mdl

Initialization: Alignment

- Coarse Leveling
  - Gimbal system limited by platform torquing rates
  - Strapdown systems have no real rate limits
  - Time average of accel outputs scaled by $1/g$
  - Yields third row of DCM to begin fine alignment
- Fine Alignment
- Alignment filter aids in further leveling

Segment 11
The Basic 9-State Kalman Filter
Topics

- Whole State Filters vs. Error State Filters
- The Canonical Nine States
- INS Error State Dynamics and Transition Matrix

Whole State vs. Error State Filters

- Whole state filters ...
  - model whole position, velocity, attitude
  - deal with large and rapidly varying quantities
  - are highly non-linear

Whole State vs. Error State Filters

- Error state filters ...
  - remove some of the linearities by differencing outputs from aiding sensors with corresponding quantities from INS
  - exhibit slowly varying dynamics: easier to linearize
  - have smaller quantities that allow us to take advantage of small angle assmptions

The Canonical Nine States

- 3 position errors: $\delta R_x, \delta R_y, \delta R_z$ in local level coordinates
- 3 velocity errors: $\delta V_x, \delta V_y, \delta V_z$ in local level coordinates
- 3 attitude errors: ϕ_x, ϕ_y, ψ_z
  - ϕ_x, ϕ_y known as tilt errors
  - ψ_z known as azimuth error

Exercises

1. Rewrite equation (70)

$$\delta\dot{C}_B^L = \delta C_B^L \Omega_{IB}^B + C_B^L \delta\Omega_{IB}^B - \left(\delta\Omega_{IE}^L + \delta\Omega_{EL}^L\right) C_B^L - \left(\Omega_{IE}^L + \Omega_{EL}^L\right) \delta C_B^L$$

to show the terms in the canonical order with position error, velocity error, and then attitude error.

2. Determine the 7th row of the system matrix F for the canonical 9-state error model. Use the cross-product operator matrix as required, e.g., $\Omega_{IE}^L = \left[\,\omega_{IE}^L\times\,\right] = \begin{bmatrix} 0 & \left[\omega_{IE}^L\right]_z & -\left[\omega_{IE}^L\right]_y \\ -\left[\omega_{IE}^L\right]_z & 0 & \left[\omega_{IE}^L\right]_x \\ \left[\omega_{IE}^L\right]_y & -\left[\omega_{IE}^L\right]_x & 0 \end{bmatrix}$.
Introduce simplifying notation! E.g., set $\alpha = \omega_{IE}^L$ Hint: Use telescope pointint angle error equation (77)

A 15-State Inertial Error Model: State Vector

$$x = \begin{bmatrix} \delta R_X \\ \delta R_Y \\ \delta V_X \\ \delta V_Y \\ \psi_X \\ \psi_Y \\ \psi_Z \\ \epsilon_x \\ \epsilon_y \\ \epsilon_z \\ \alpha_x \\ \alpha_y \\ \delta\theta_Z \\ \beta_1 \\ \beta_2 \end{bmatrix}$$

A 15-State Inertial Error Model: State Dynamics Matrix

$$
\begin{bmatrix}
1 & 0 & T & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -V_Y T & 0 & 0 \\
0 & 1 & 0 & T & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -V_X T & 0 & 0 \\
\phi_{31} & \phi_{32} & 1 & 2\omega_Z T & 0 & -a_Z T & a_Y T & 0 & 0 & 0 & c_{11} T & c_{12} T & a_Y T & 0 & 0 \\
\phi_{41} & \phi_{42} & \phi_{43} & 1 & a_Z T & 0 & -a_X T & 0 & 0 & 0 & c_{21} T & c_{22} T & -a_X T & 0 & 0 \\
0 & 0 & 0 & 0 & 1 & \omega_Z T & -\omega_Y T & c_{11} T & c_{12} T & c_{13} T & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & -\omega_Z T & 1 & -\omega_X T & c_{21} T & c_{22} T & c_{23} T & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & \omega_Y T & -\omega_X T & 1 & c_{31} T & c_{32} T & c_{33} T & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & e^{-T/\tau_g} & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & e^{-T/\tau_g} & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & e^{-T/\tau_g} & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & e^{-T/\tau_a} & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & e^{-T/\tau_a} & 0 & 0 & 0 \\
\frac{V_Y T}{R^2} & \frac{V_X T}{R^2} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & e^{-T/\tau_N} & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & e^{-T}
\end{bmatrix}
$$

where T is the delta time interval, τ_g,τ_a,τ_N,τ_E are Markov time corelation constants for gyro bias, accelerometer bias and GPS north and east biases, respectively.

Segment 12
Challenges of INS Filters
Topics

- Nine States Are Not Enough
- Not Constant Coefficient
- Non-Linearities: Extended Kalman Filter
- Observability

Nine States Are Not Enough

- The canoncial 9 states only characterize the inertial error dynamics
- They treat of the inner coupling between inertial error sources
- However, they don't help characterize dynamics of aiding sensor errors

Nine States Are Not Enough

- They don't help estimate aiding states such as Doppler bias and boresight errors
- Or GPS receiver clock errors
- They also don't help us to estimate inertial instrument drifts such as bias drift or scale factor errors

Not Constant Coefficient

- Notice that the F matrix that has emerged (implicitly) from the 9-state inertial error dynamics is not time invariant
- It is highly dependent on trajectory data such as acceleration and angular rate profiles
- These are functions of time
- Position dependent items such as latitude are also involved

Non-Linearities: Extended Kalman Filter

- When non-linearities are present, the F matrix emerges as Jacobian of non-linear system function f: $F(t) = \dfrac{\partial f}{\partial x}$

- If Jacobian is evaluated at a nominal value of the trajectory, we call this a linearized filter

- If Jacobian is evaluated at the previous optimal estimate, we call it an Extended Kalman filter

Observability

- Structural observability is concerned with how H maps measurements into states

- Will making measurements as governed by H actually enable us to estimate all of the states in the state vector?

Observability

- Observability means there is a linear path from the measurements to states

- Structural observability determined by Φ and H:

If H is $1 \times n$, where n is the size of the state vector, then the state x_0 is said to be **observable** from n scalar measurements $\{z_0, \ldots, z_{n-1}\}$ if the block matrix

$$\begin{bmatrix} H^\top & \Phi^\top H^\top & \cdots & (\Phi^\top)^{n-1} H^\top \end{bmatrix}$$

is of rank n

Observability

- Observability challenge for INS filters is usually a matter of choosing right navaids
- And also choosing the right measurement scheduling
- Also, certain kinds of divergence can be the result of poor observability (more on this later)

Exercises

- Use Eq. (75)

$$\delta\dot{R}^L = \delta v^L - \Omega^L_{EL}\delta R^L,$$

write the first three rows of the system dynamics matrix F in the continuous Kalman formulation:

$$\dot{x} = Fx + w$$

Day 4: General Code Implementation Issues
Topics

- Simulation vs. Real-Time Implementation
- Survey of Navaids
- Filter Maintenance
- Filter Redundancy and Fault Tolerance

Day 4, Segment 1
Simulation vs. Real-Time Implementation
Topics

- Simulation

- Real-Time Implementation

Simulation

- Introduction to Matlab / Simulink
- Covariance Analysis
- Trajectory Simulation

Introduction to Simulink

- Matlab
- Simulink

Matlab

- Matrix-oriented scientfic computing tool from Mathworks
  - Everything is a matrix: scalars are 1-by-1 matrices
- Lots of built-in numerical tools for matrix manipulation
- Industry standard

Matlab

```
%   Row vector:
v = [1 2 3]; % Or separate columns with comma: v = [1, 2, 3];
%   Column vector:
u = [1;2;3]; % Note: Rows separated with semicolon
% Natural multiplication:
x=v*u;
```

Matlab

```
%Displaying values:
v
u
x
```

Matlab

```
% Matrices: Combinations of rows and columns:
A = [1 2 3; 4 5 6; 7 8 9];
w = A*u;
% Matrix operators like inverse and transpose:
B = inv(A);
C = A';
D = u*u'; % Symmetric matrix
```

Matlab

- Full-fledged programming language with control constructs and looping
- Data structures
- Many c-like functions
- Extensive plotting

Matlab

- Expensive: >$2K / license
- Toolboxes and add-ons are extra
- Freeware alternative: Octave (go to www.gnu.org)

Simulink

- Simulation of Continuous and Discrete Systems
- Linear and Non-Linear
- Easy to Use, Drag and Drop, Block Diagram Interface
  - Large collection of block libraries
- Easy integration with external routines, Matlab functions / scripts, S-functions

Simulink

- Hierarchical Nesting of Subsystems
- Can build hybrid discrete / continuous systems
- Integrate inputs and outputs with Matlab Workspace
- Models can be executed in batch (from script or command line) or interactively in GUI

Simulink

- See demo of Model Construction

Covariance Analysis

- Overview
- Various Approaches to Covariance Simulation
- SIMLTIC: **SIM**ulation of **L**inear **T**ime-**I**nvariant **C**ovariance

Covariance Analysis: Overview

- Covariance propagation and covariance update can be be accomplished without actually dealing with the state vector at all.

(78) $$P_k^- = \Phi_k P_{k-1}^+ \Phi_k^\top + Q_k$$

(79) $$P_k^+ = [I - K_k H_k] P_k^-$$

- Of course, the Kalman gain still has to be computed for use in Eq. (79) above, the update equation. The gain is:

(80) $$K_k = P_k^- H_k^\top \left[H_k P_k^- H_k^\top + R_k \right]^{-1}$$

Covariance Analysis: Overview

- As in real-time implementation, the Joseph form of the gain equation is generally preferred, because of its superior numerical properties:

(81) $$P_k^+ = [I - K_k H_k]\, P_k^-\, [I - K_k H_k]^\top + K_k R_k K_k^\top$$

Covariance Analysis: Overview

- The only items driving the covariance equations are the state dynamics F, the process noise Q, and the measurement noise R

- Thus, covariance analysis can provide good insight into how well balanced the Q and R matrices are

- More importantly, covariance analysis is crucial in exploring what-if scenarios with new measurement sources

Covariance Analysis: Overview

- As an example, suppose you want to evaluate several different GPS receivers as navaids

- You can use a manufacturer's published 1σ accuracy specifications in your measurement noise matrix R and run several covariance simulations to see what the expected performance is

Covariance Analysis

- Of course, if GPS is your only navaid, this is a no brainer: the receiver with the least measurement noise performs best

- However, if this sensor is just one of several, you might find that some less expensive receiver when thrown into the mix with other navaids is good enough

Covariance Analysis: Overview

- Other uses: Debugging the filter

- If you save the actually used process noise Q and measurement noise R from a real-time mission (using a maintenance logging facility), then thse matrices can be fed into the covariance simulation to see what effect these have on the gains.

- Or to see how the offline gains compare with the real-time gains.

Covariance Analysis: Overview

- Covariance analysis is quite often used to determine proper error budgets

- Error budget: How error sources are allocated to various states and sensors

- E.g., in a nav application, to meet overall system position accuracy requirements, certain combinations of misalignement, instrument bias, scale factor, and inertial noise error, etc., will work when coupled with sensor errors

Covariance Analysis: Overview

- Optimal vs. Suboptimal
- Evaluating finite wordlength effects
- Bottom Line: Covariance analysis essential to filter maintenance
  - If you are planning to modify a Kalman filter at all, you need to run covariance analysis simulations
  - It is very risky not to do so!

Various Approaches to Covariance Simulation

- In the continuous formulation, covariance extrapolation can be accomplished by solving the linear variance equation

(82) $$\dot{P} = FP + PF^{\top} + Q$$

- This can be useful in exploring how a navigation system "coasts" in the absence of measurements
  - It can give you an idea of how fast the solution degrades in free-inertial

Various Approaches to Covariance Simulation

- In the continuous case, we can also eliminate the gain because

$$\lim_{\Delta t \to 0} \frac{K(t)}{\Delta t} = PH^\top R^{-1}$$

- This leads to a single equation, the Matrix Riccati equation, for covariance propadation and update:

(83) $$\dot{P} = FP + PF^\top + Q - PH^\top R^{-1} HP$$

Various Approaches to Covariance Simulation

- Matrix Riccati Equation reveals essential characteristics of filter

$$\dot{P} = FP + PF^{\top} + Q - PH^{\top}R^{-1}HP$$

- Note that in this equation we increase estimation uncertainty by adding in process noise

- And we decrease estimation uncertainty by the amount of information (R^{-1}) inherent in the measurement

Various Approaches to Covariance Simulation

- See Matlab / Simulink Model

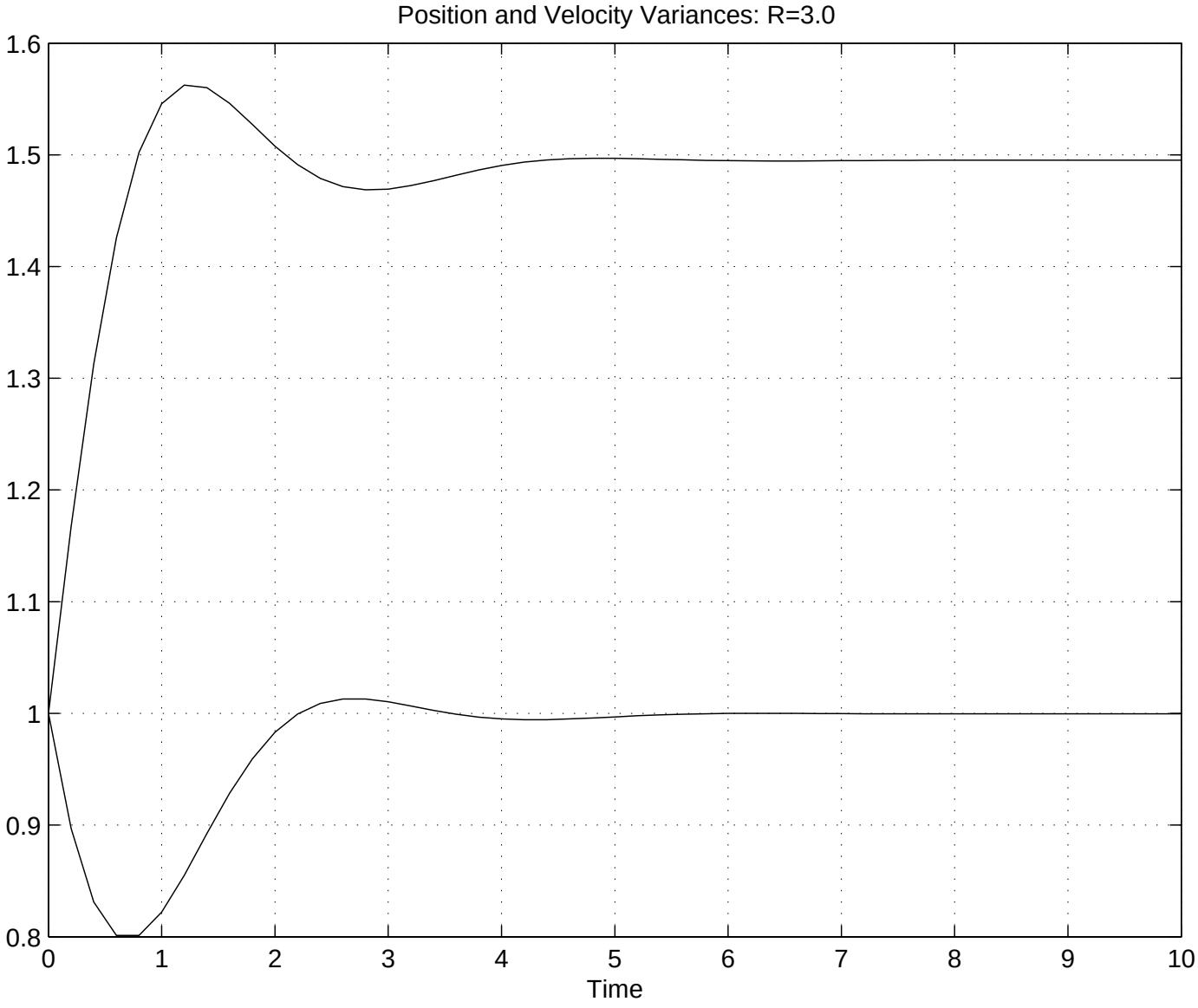
  `matrix_riccati.mdl`

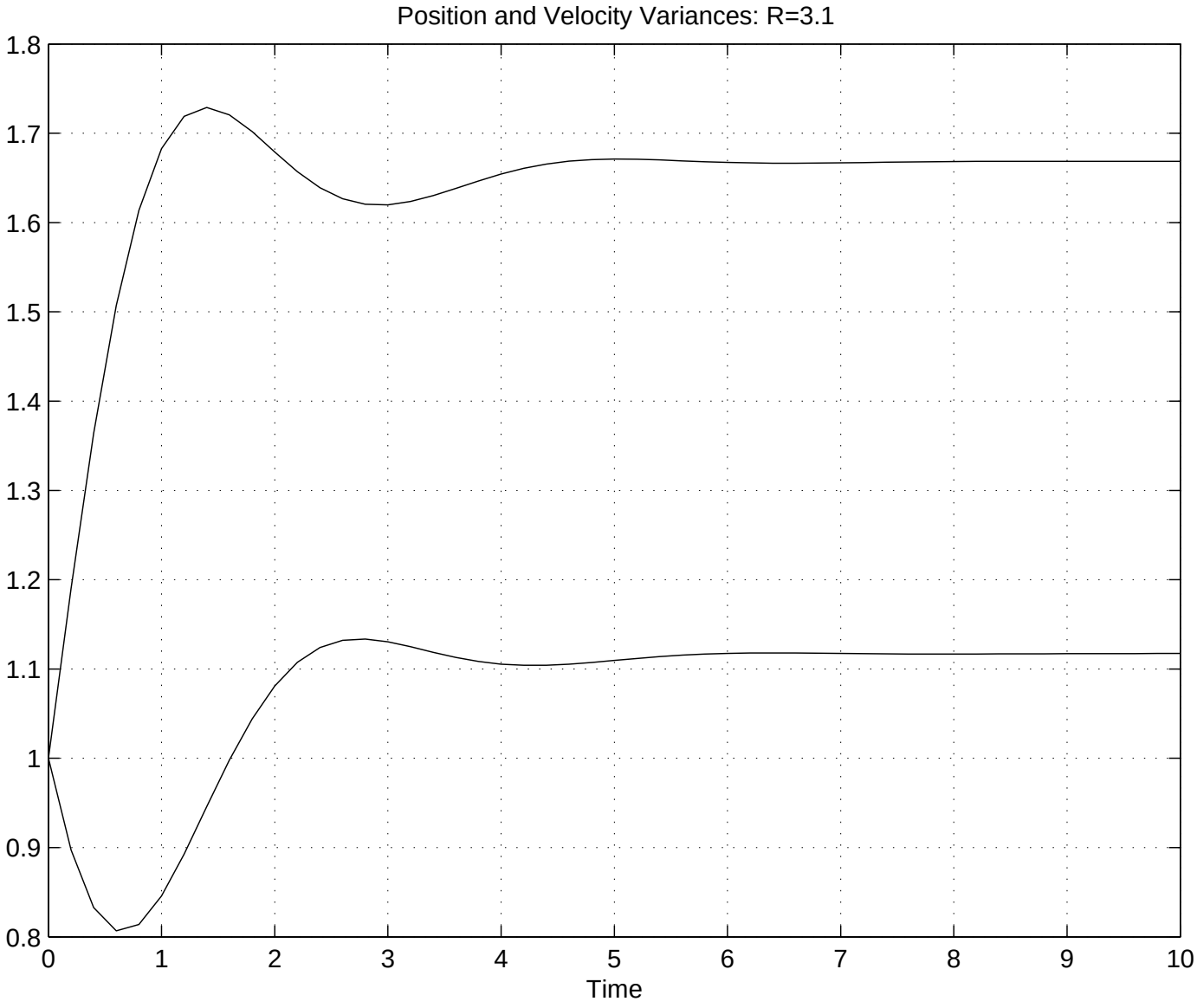
- Dampled harmonic oscillator; parameter file

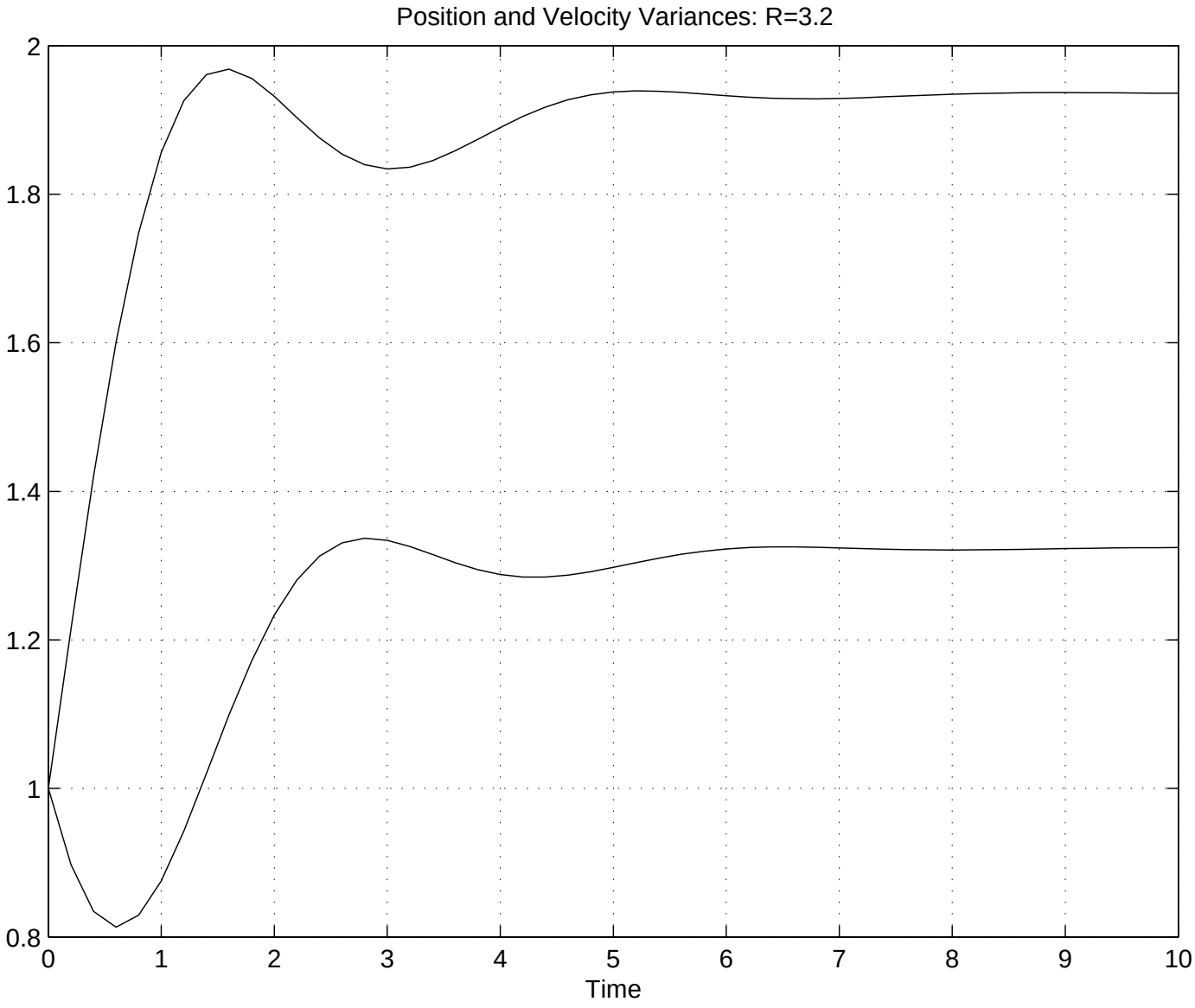
  `harmosc_riccati.mat`

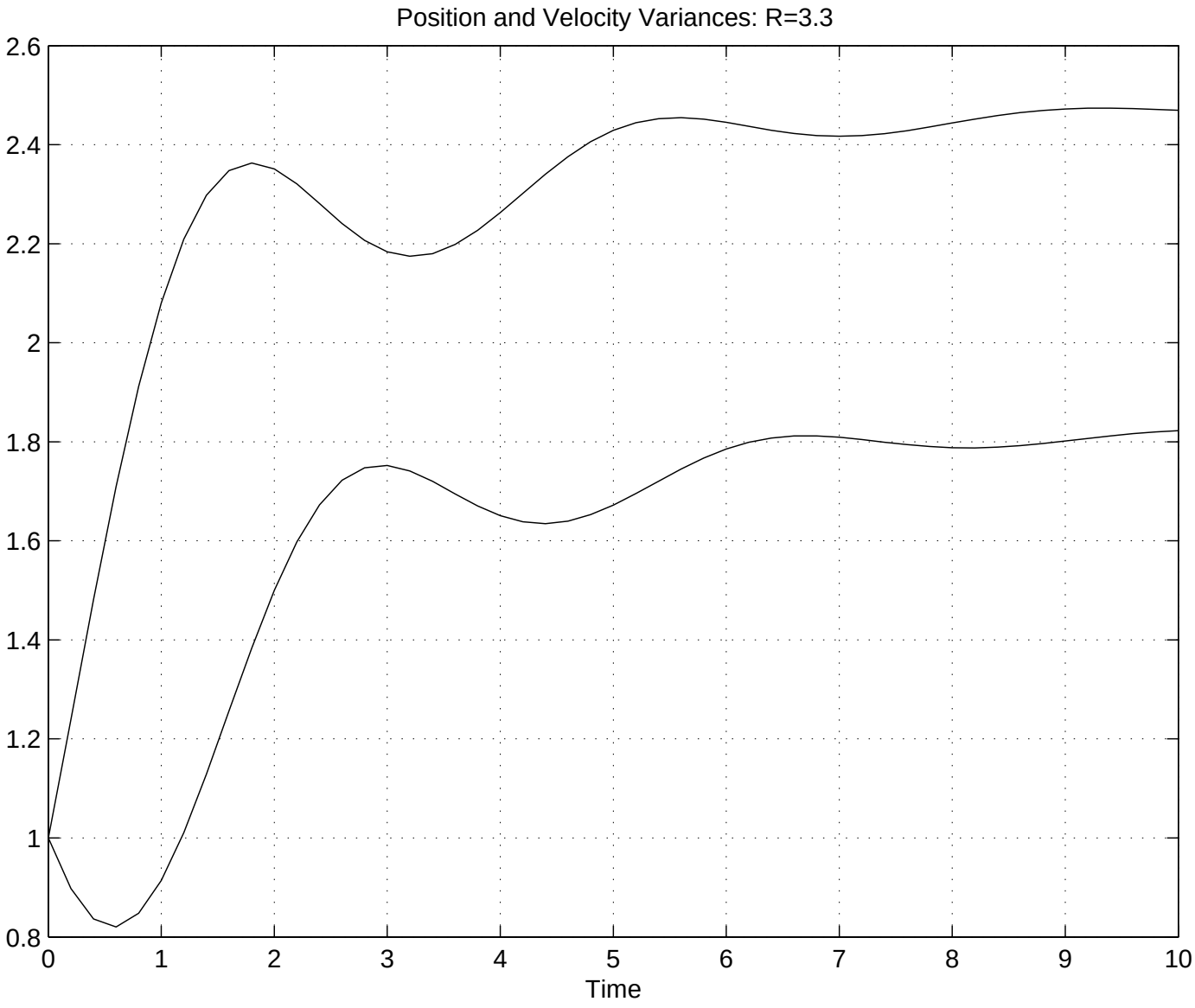
  – 2d state vector: position and velocity

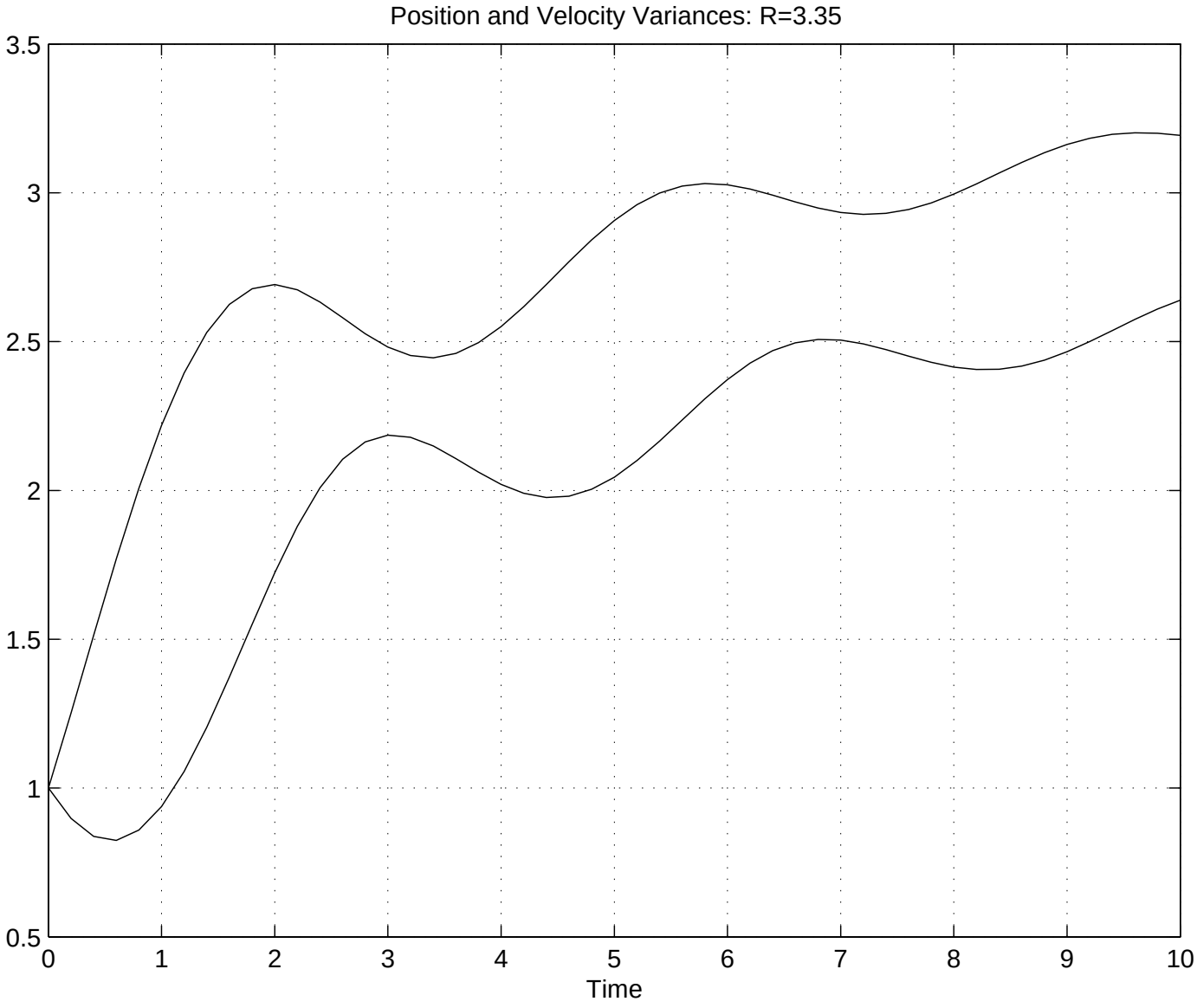
  – 1 scalar measurement and scalar measurement noise R

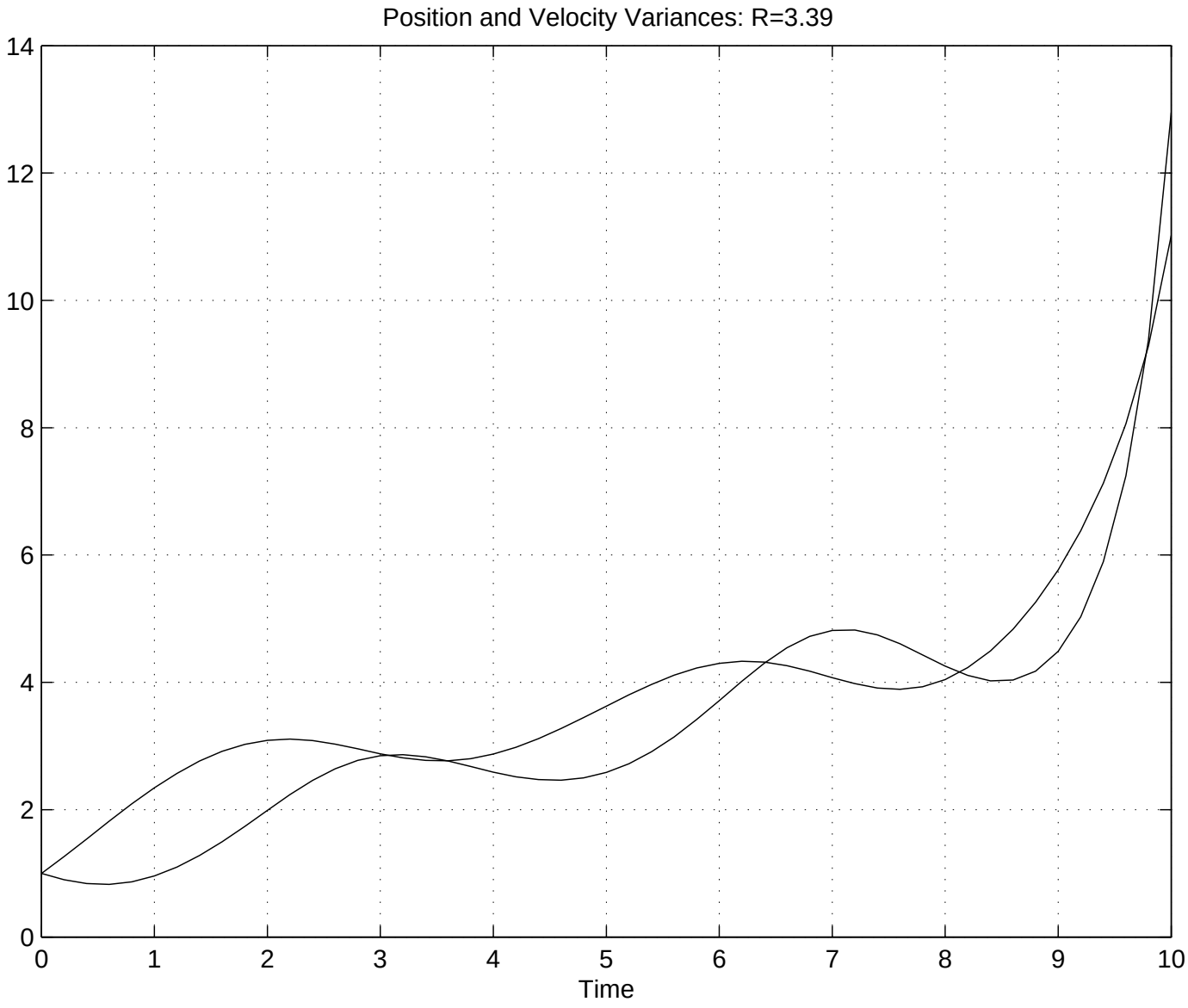

Position and Velocity Variances: R=3.0
1.6
1.5
1.4
1.3
1.2
1.1
1
0.9
0.8
0
1
2
3
4
5
6
7
8
9
10
Time


Position and Velocity Variances: R=3.1
1.8
1.7
1.6
1.5
1.4
1.3
1.2
1.1
1
0.9
0.8
0
1
2
3
4
5
6
7
8
9
10
Time


Position and Velocity Variances: R=3.2
2
1.8
1.6
1.4
1.2
1
0.8
0
1
2
3
4
5
6
7
8
9
10
Time


Position and Velocity Variances: R=3.3
2.6
2.4
2.2
2
1.8
1.6
1.4
1.2
1
0.8
0
1
2
3
4
5
6
7
8
9
10
Time


Position and Velocity Variances: R=3.35
3.5
3
2.5
2
1.5
1
0.5
0
1
2
3
4
5
6
7
8
9
10
Time


Position and Velocity Variances: R=3.39
14
12
10
8
6
4
2
0
0
1
2
3
4
5
6
7
8
9
10
Time

Damped Harmonic Oscillator Covariance Analysis

- Matrix Riccati Differential Equation with constant F, H, Q, R
- Varying R demonstrates extreme sensivitiy of P solution to changes in R
- $R = 3.3$ is stable; but $R = 3.4$ is unstable

Damped Harmonic Oscillator Covariance Analysis

- Example illustrates how damping coefficient (reflected in system dynamics matrix F) and process noise uncertainty Q interact with measurement noise uncertainty R

- Highly coupled, non-linear equation

$$\dot{P} = FP + PF^{\top} + Q - PH^{\top}R^{-1}HP$$

Alternative Forms of Matrix Riccati Equation

- Can be factored into two first-order, linear matrix

- May be better behaved numerically
  - with Matlab / Simulink it is very easy to implment even non-linear matrix differential equations
  - Simulink's ODE solvers may not always be up to the task, however

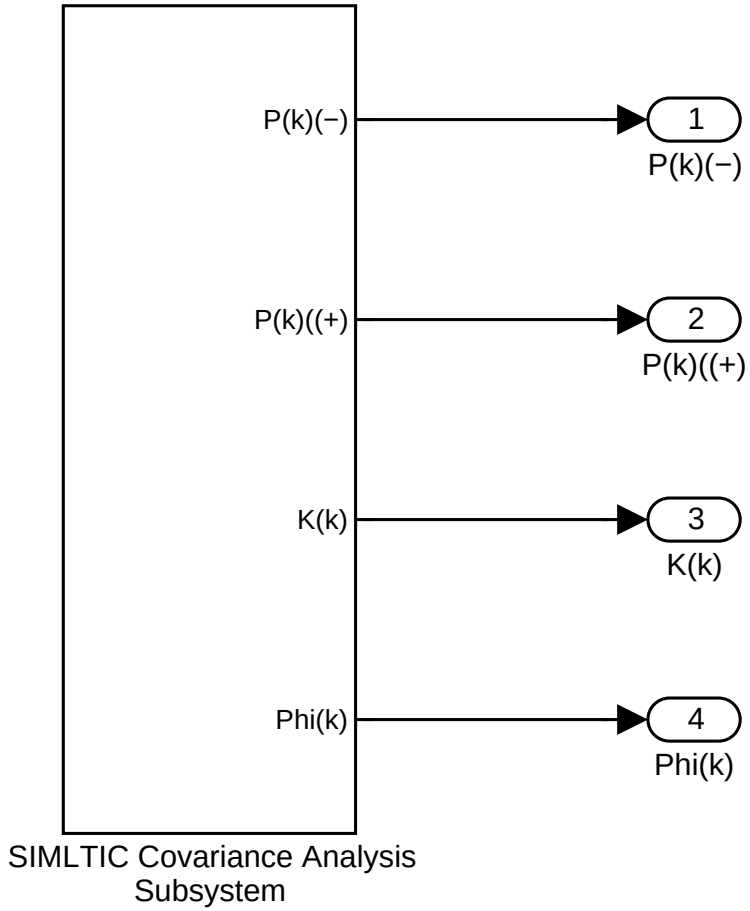
Various Approachs to Covariance Simulation

- Discrete models:
  - Implement regular filter processing less the state propagation and update

SIMLTIC: **SIM**ulation of **L**inear **T**ime-**I**nvariant **C**ovariance

- Discrete, Generic Covariance Simulation
- LTI = Constant Coefficients in System Dynamics
- Implemented as Simulink Subsystem Block
  - Can be dropped into higher level models as required
  - Output can be integrated with Matlab Workspace

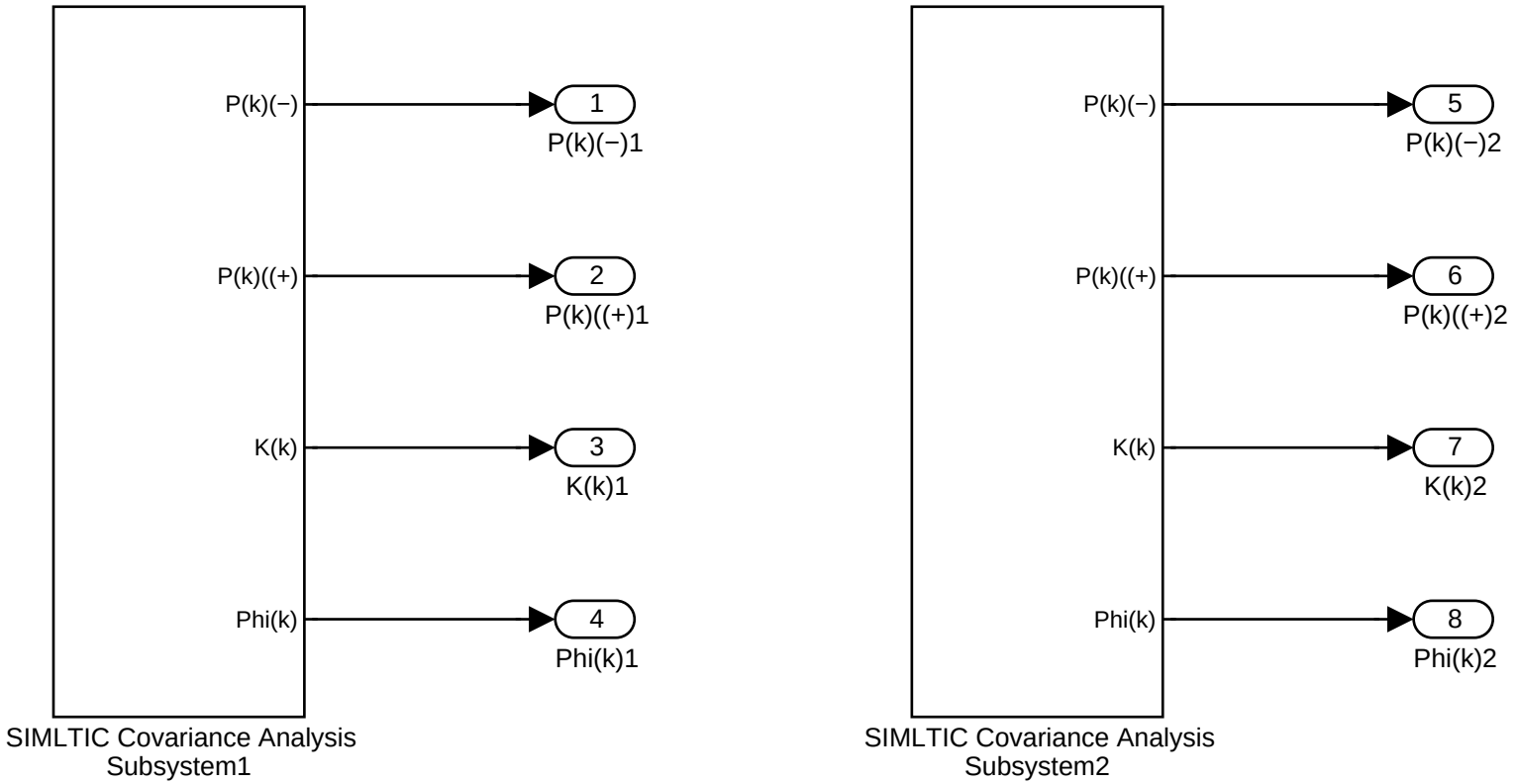
SIMLTIC
SIMulation of Linear Time Invariant Covariance



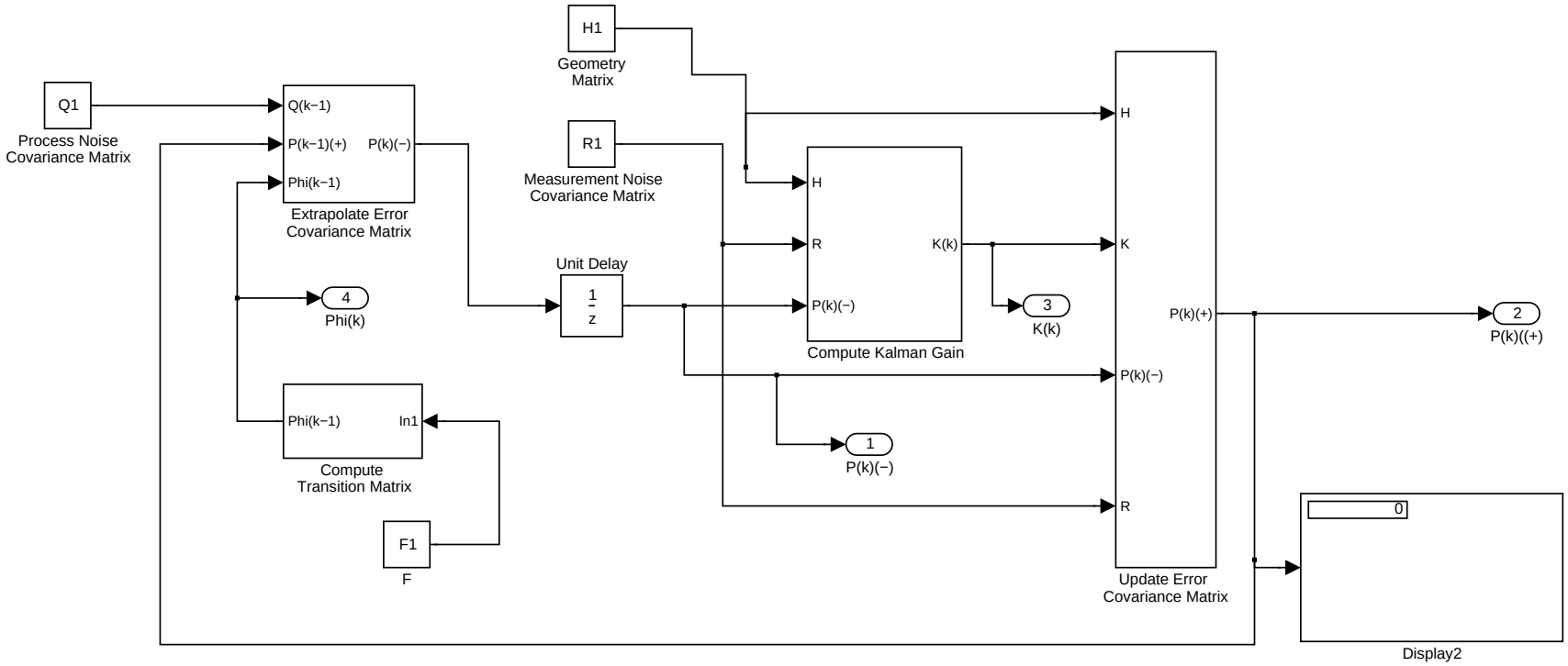
SIMLTIC: **SIM**ulation of **L**inear **T**ime-**I**nvariant **C**ovariance

- Example: Covariance Comparison of Two Harmonic Oscillator Models

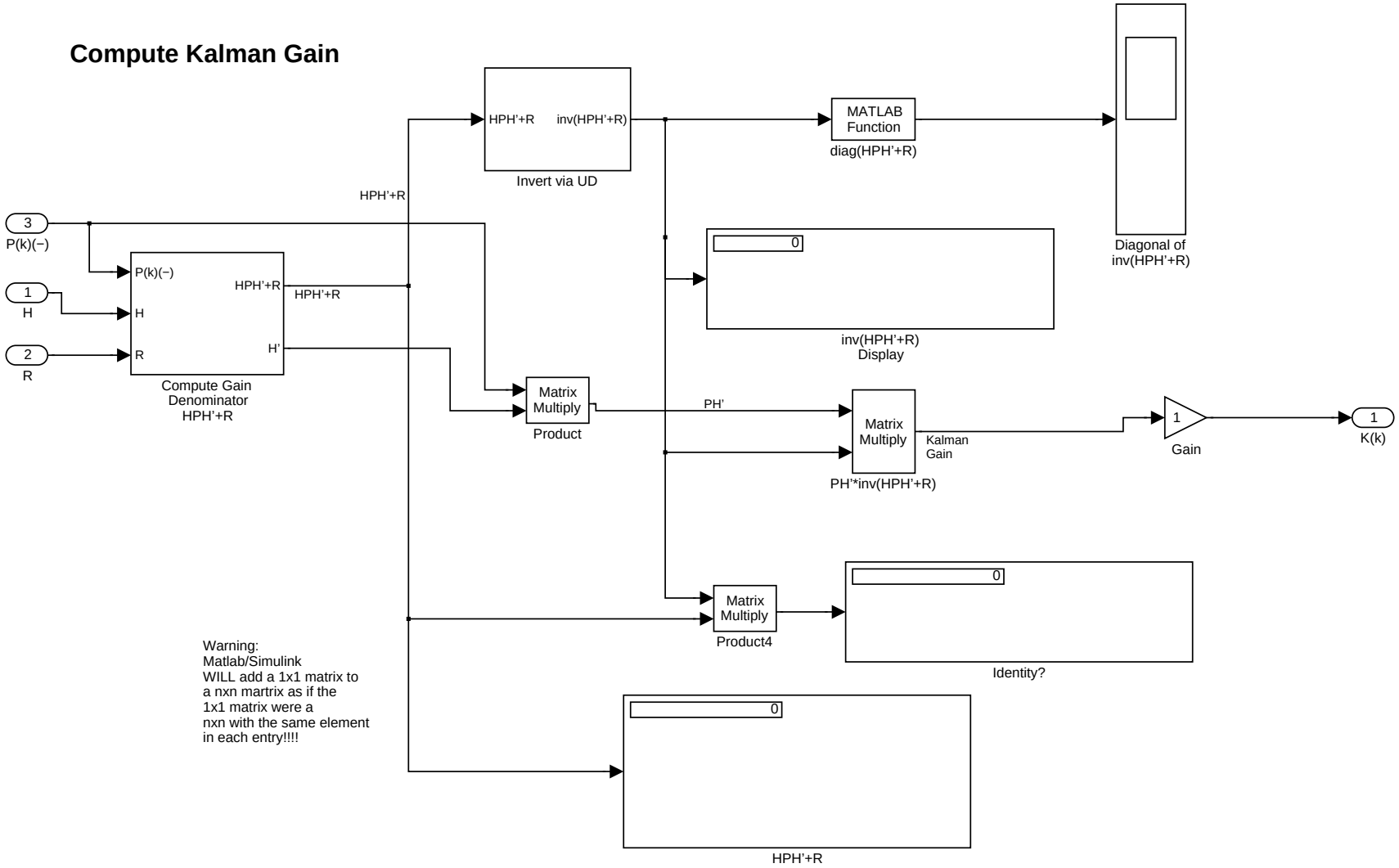
SIMLTIC
Comparing Performance of Two Models



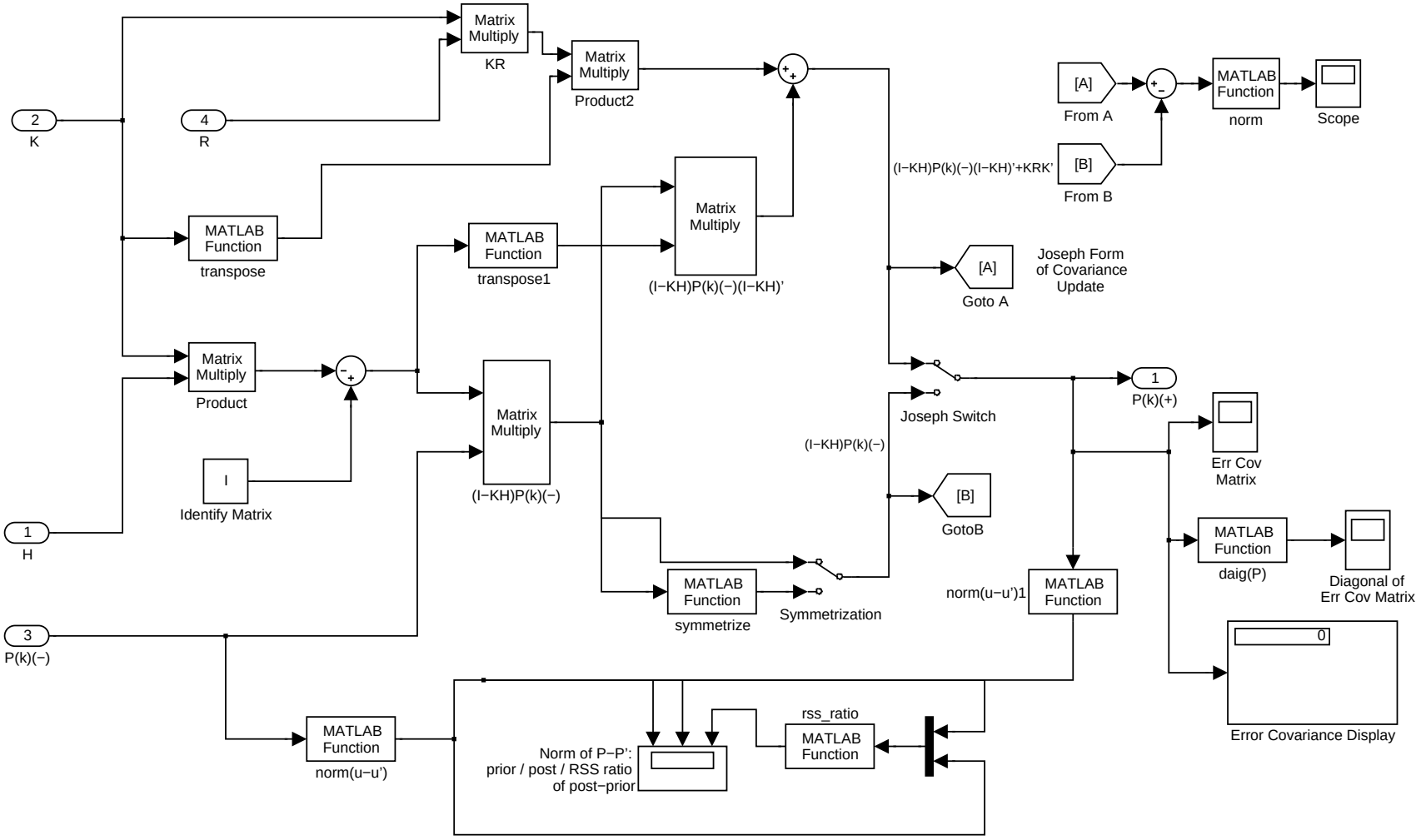
SIMLTIC
Subsystem 1



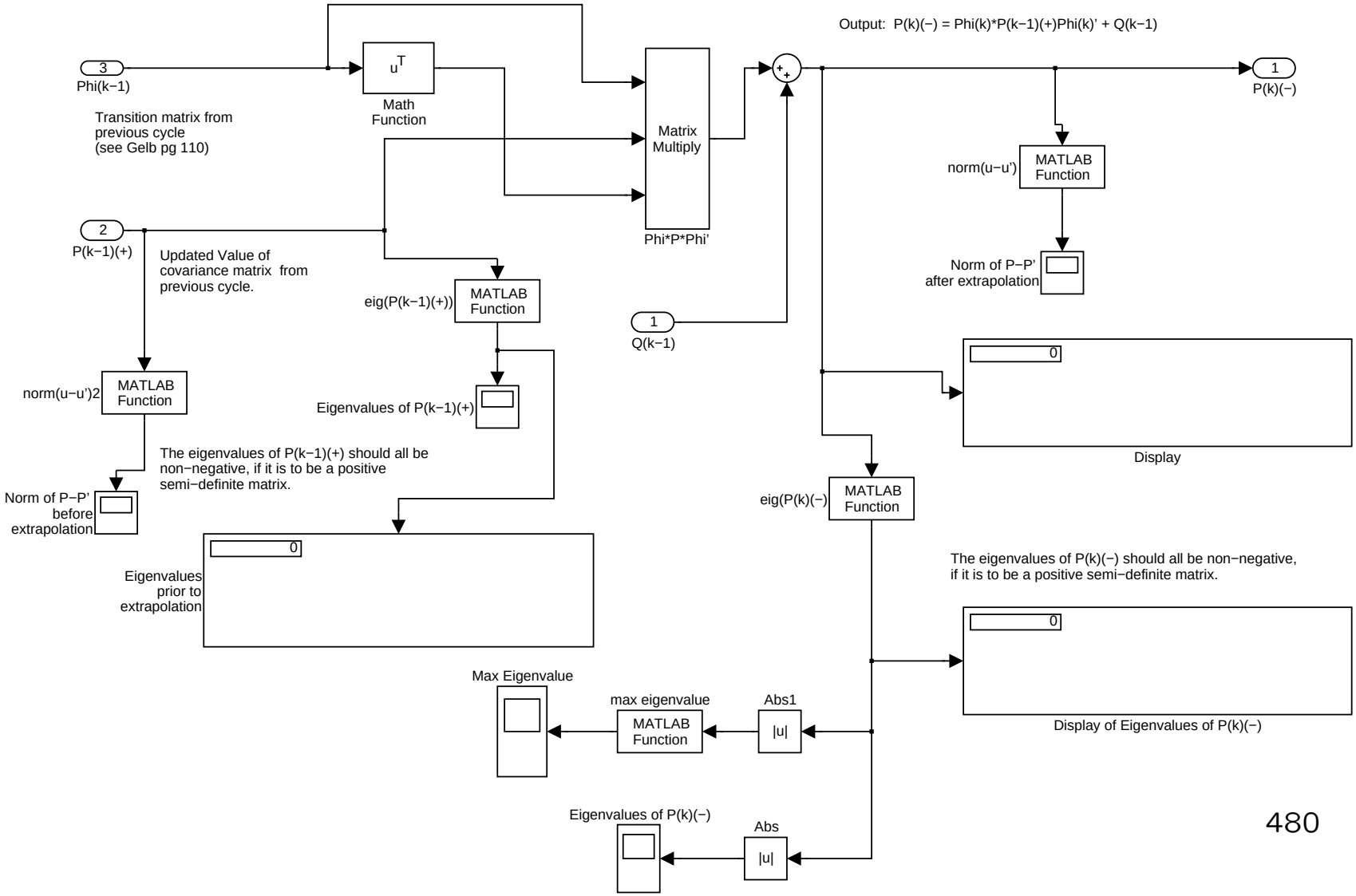
Compute Kalman Gain



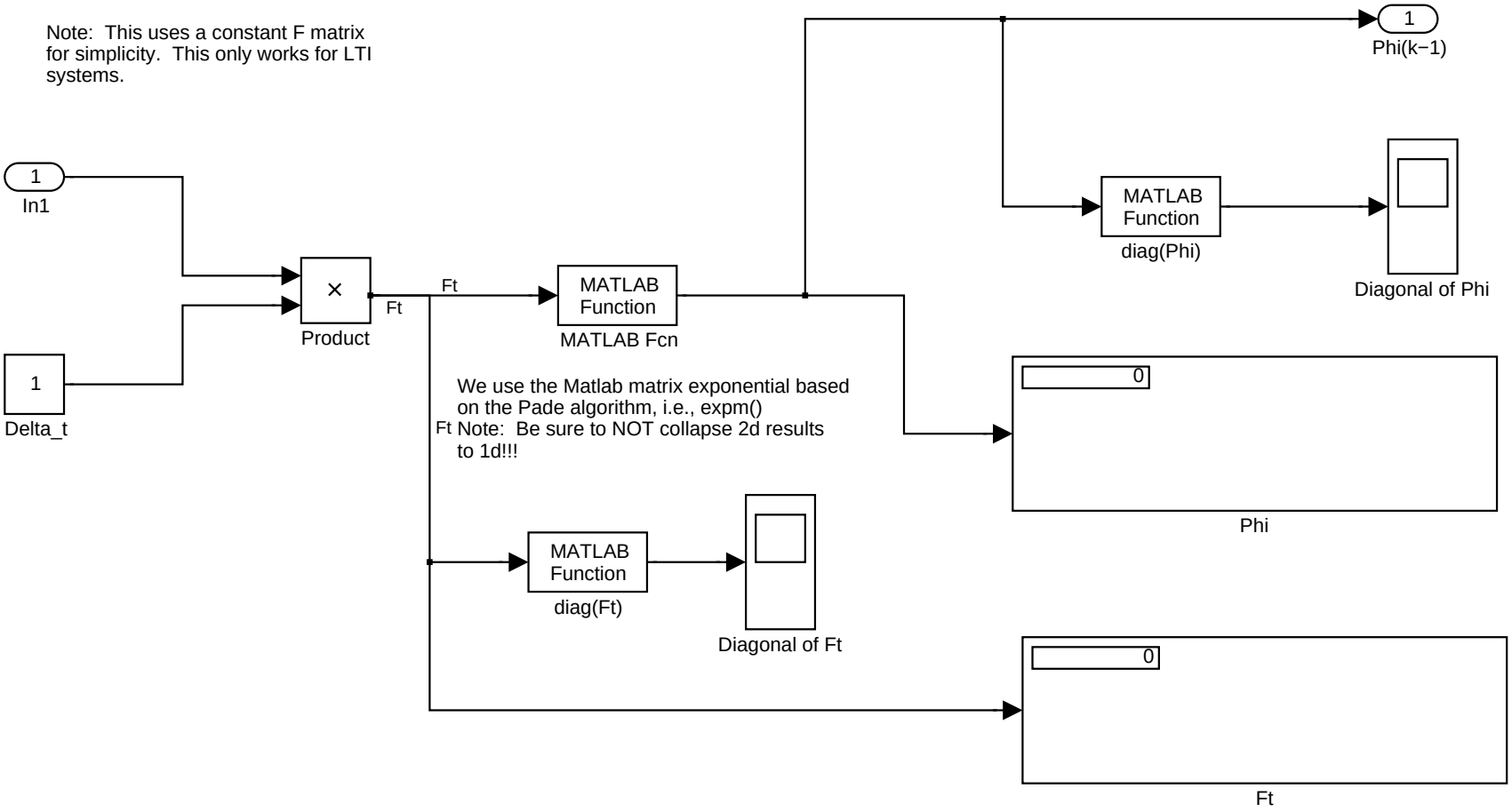
Update Error Covariance Matrix



Extrapolate Error Covariance Matrix



Compute Transition Matrix

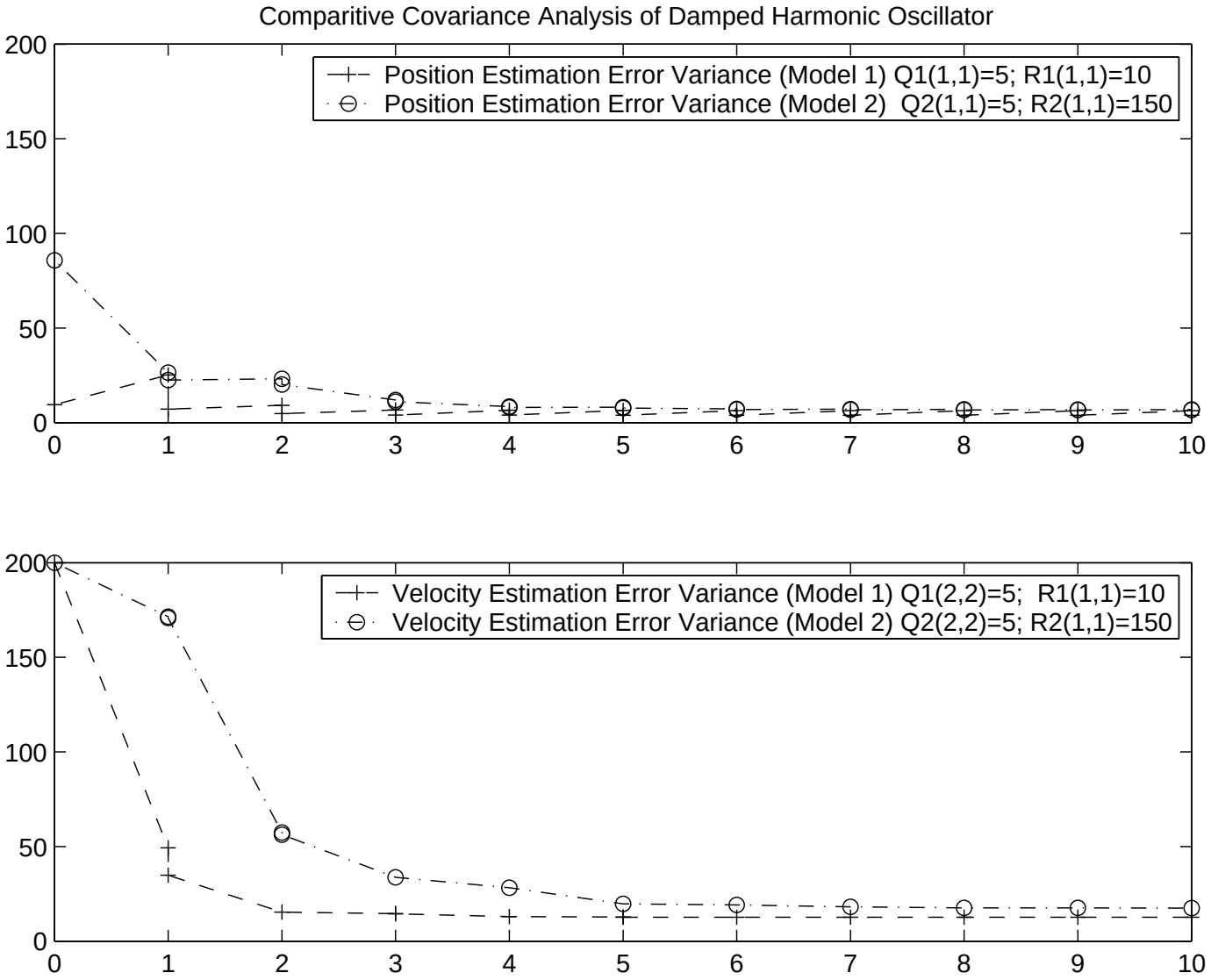


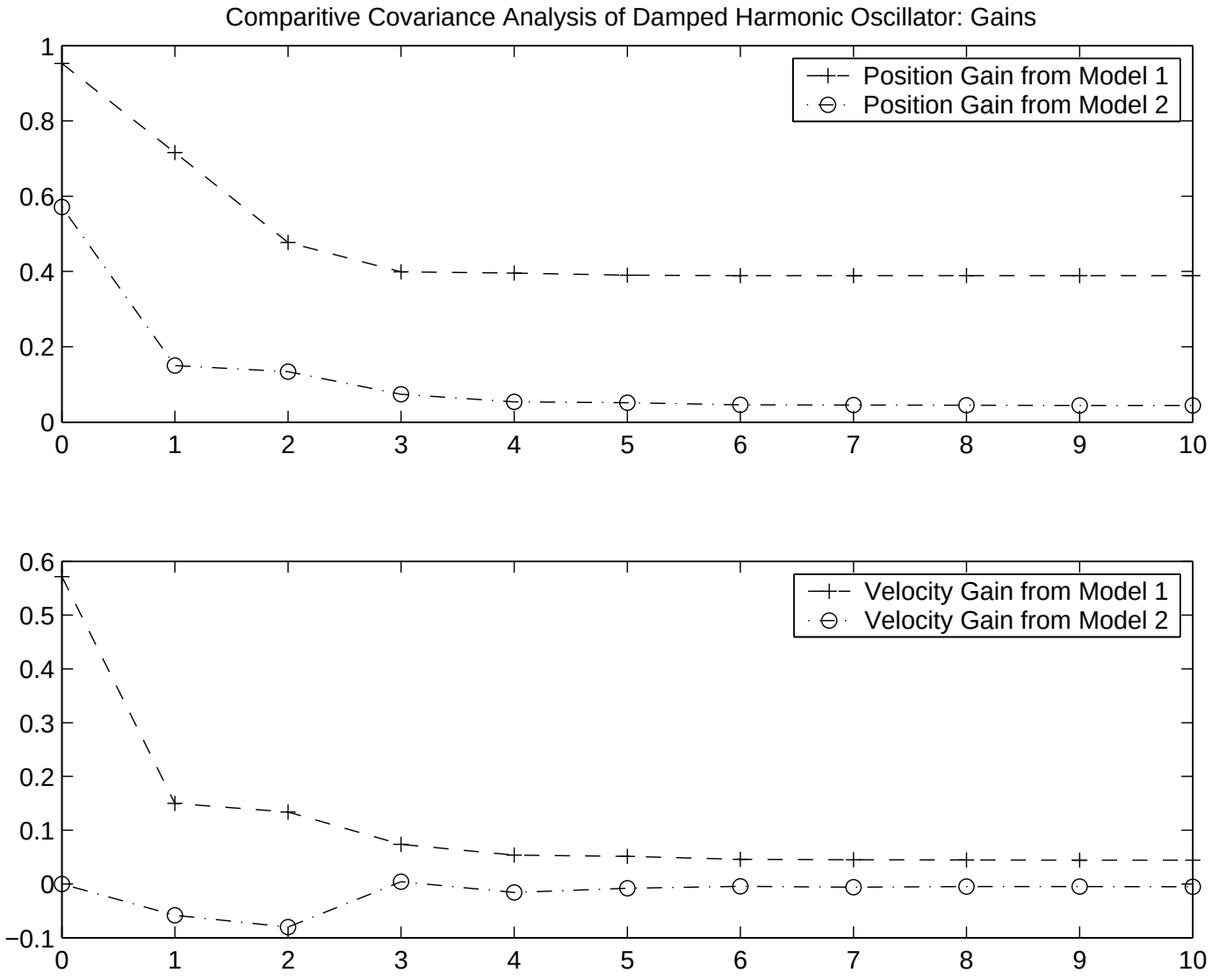
Transistion Matrix as Matrix Exponential

- If system dynamics matrix F is constant, i.e., not time-varying over the interval of interest, then $\Phi(\Delta t)$ can be expressed as matrix exponential:

$$\Phi(\Delta t) = I + F\Delta t + \frac{1}{2!}[F\Delta t]^2 + \frac{1}{3!}[F\Delta t]^3 + \cdots$$

- If Δt is small enough, the approximation $\Phi(\Delta t) \approx I + F\Delta t$ may be adequate


Comparitive Covariance Analysis of Damped Harmonic Oscillator
Position Estimation Error Variance (Model 1) Q1(1,1)=5; R1(1,1)=10
Position Estimation Error Variance (Model 2) Q2(1,1)=5; R2(1,1)=150
Velocity Estimation Error Variance (Model 1) Q1(2,2)=5; R1(1,1)=10
Velocity Estimation Error Variance (Model 2) Q2(2,2)=5; R2(1,1)=150
0
50
100
150
200
0
1
2
3
4
5
6
7
8
9
10


Comparitive Covariance Analysis of Damped Harmonic Oscillator: Gains
Position Gain from Model 1
Position Gain from Model 2
Velocity Gain from Model 1
Velocity Gain from Model 2

Trajectories

- In navigation applications, the system dynamics vary according to where you are on the earth
- Some quantities are dependent on your latitude and longintude.
- E.g., the vertical component of earth rate depends on your latitude

Trajectories

- Therefore, in order to perform a covariance analysis of a navigation system, a trajectory generator that computes truth value dynamic quantities such as body rates and accelerations is necessary

- This is the "truth model" that can then be corrupted by sensor error sources

Simulation vs. Real-Time

- Real-time time constraints
- Different processing rates
- Data Buffering
- Measurement Scheduling
- Keeping the Filter Awake
- Covariance Bumping

Time Constraints

- Simulation
  - offline, non-realtime
  - each calculation (transition matrix, Kalman gain, etc.) can run to completion
  - do not have to worry about rate group partitioning
- Real-Time
  - time is of the essence

Real-Time: Different Processing Rates

- In real-time apps, you cannot waste processing cycles
  - there are numerous other mission processing tasks to accomplish besides filtering
- Some, fast varying quanitites need to be updated often; others, less often

Real-Time: Different Processing Rates

- For example: body rates and quantities highly-dependent on them need to be updated as rapidly as possible
- Depends, of course, on platform dynamics
- Transport rates don't need to be updated as often

Rate Allocation Example: A Transport Aircraft Navigation Filter

- Kalman Gain Calcuation, State and Covariance Update: 0.1 Hz (was originally 0.05 Hz)

- State and Covariance Extrapolation: 4 Hz
  - dependent on some 16 Hz Fast Nav calcs: body rates and attitudes
  - dependent of some 8 Hz Slow Nav calcs: transport rates

Buffering

- Keep low rate data from being clobbered by higher rate data
- Keep high rate data from being clobbered by low rate data
- Buffering: Orderly passage and distribution of data up and down the rate group hierarchy

Buffering Example from an Aircraft Navigation Filter:

- Slow rate group exec buffers data to and from faster rate groups

- Faster rate groups do not push or pull data with respect to slower groups

  - REP8HZ buffers data to and from the 16 Hz rate group, but does not control buffering to and from 4 Hz

  - Hence REP16HZ does not call any BU' routines

Buffering Example:
Approach

- Buffer from faster at beginning of start of slow rate group
- Buffer to faster at end of slow rate group
- Within BU'* routine:
  1. Disable interrupts
  2. Perform buffering assignments
  3. Re-enable interrupts

```
117 DEF PROC BU'BUFFER'TO'05HZ'FROM'FASTER;
118 BEGIN
119
120   DISABLE THE INTERRUPTS
121 %
122       DISABLE'INTRUPT;
123
124   BUFFER SYSTEM LATITUDE AND LONGITUDE FOR G/S
125 %
126       SYSTEM'LATITUDE'05  = SYSTEM'LATITUDE'16;
127       SYSTEM'LONGITUDE'05 = SYSTEM'LONGITUDE'16;
128
129   ENABLE THE INTERRUPTS
130 %
131       ENABLE'INTRUPT;
```

Measurement Scheduling

- Art / Science unto itself
- Determining which measurements to process when
- Usually fixed schedule with potential operator override
  - E.g., Position Fix and Altitude Cal

Measurement Scheduling

- Sequential Processing of Measurements
  - Scalarize to avoid matrix inversion of $HPH^\top + R$
  - Or at least reduce dimension
  - feasible if R matrix can be block partitioned into uncorrelated blocks

Measurement Scheduling

- Examining eigenevalues of P
- The larger the eigenvalue, the less observable the associated state eigenvector
- State eigenvector is linear combination of states
- Measurements can be scheduled to improve observability

Keeping Filters Awake

- Filter goes to sleep means error covariance for certain states becomes so small that measurements no longer have an effect

- Can be the result of round-off problems

- Presence of non-random, determinisitc quantities can be contributors

  - Random constants are nearly deterministic

  - Watch out for states not driven by process noise!

Keeping Filters Awake

- Recall running average example
- That was a random constant
- Gain $1/N \to 0$ as $N \to \infty$
- One aircraft nav filter has 12 states modeled as random constants, i.e., the non-varying states

Covariance Bumping

- Covariance bumping refers to the addition of process noise to keep the filter from going to sleep

- Usually based on ad hoc rules

JOVIAL Samples of three different kinds of bumping:

```
  DEF  ITEM  POSITION'BUMP   STATIC REAL = 10.0;
  DEF  ITEM  V'BUMP          STATIC REAL = 10.0;
  DEF  ITEM  COVARIANCE'BUMP STATIC REAL = 50.0;
...
  OBS'COV'SCALER = POSITION'BUMP * OBS'COUNT'SCALER;
...
  NOISE'VECTOR(3) = (Y'KAL'OBS(INU,11) * OBS'COUNT'SCALER
                          * OBS'COV'SCALER * V'BUMP) **2;
...
   OBS'COV'SCALER = COVARIANCE'BUMP * OBS'COUNT'SCALER;
```

Exercises

1. For fixed $R = 3.0$, what will the the effect of increasing Q be in the model harmosc_riccati.mat?

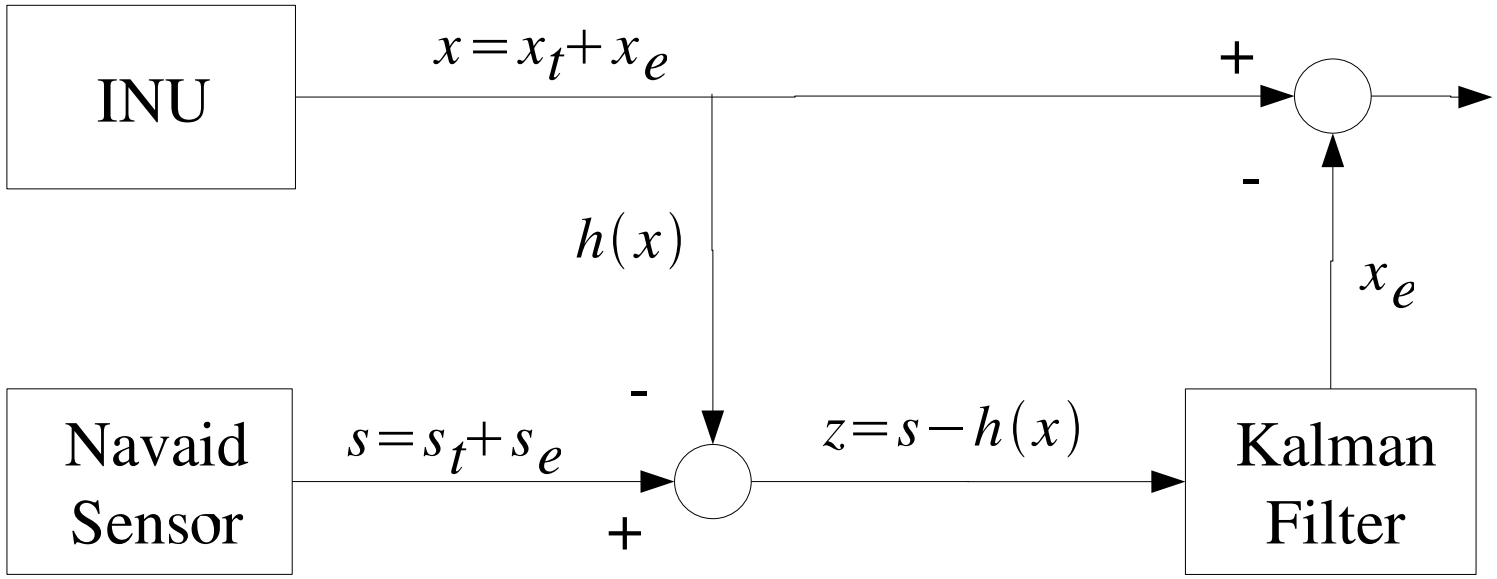
2. Run the simulation with various values of Q and check to see if the predicted behavior is achieved.

Day 4, Segment 2
Survey of Navaids
Topics

- GPS
- Position Fixes
- Radar
- Doppler Velocity Sensor (DVS)
- External Tracking Aids

- Digital Terrain Elevation Data (DTED)

INS Error State Kalman Filter
Feedforward Mechanization



Note: $z = s - h(x) = s_t - h(x_t) + s_e - h(x_e)$

Navigation Aids (Navaids)

- Measurement / Observation sources that aid in estimating INS error sources and dynamics

- Because we are interested in estimating INS errors, we use differencing to cancel out the whole state information

- INS outputs $x = x_t + x_e$, where x_t is the true whole state information generated by INS, i.e., position, velocity, attitude, etc.

- Navaid sensor outputs $s = x_t + x_e$, where x_t is once again the true quantities corrupted by sensor errors x_e.

Navigation Aids (Navaids)

- Note that the navaid sensor usually only outputs part of the whole state vector, e.g., position and velocity, but not attitude (in the case of GPS)

- That's one reason why we have the h function

- The INS signal is mapped by a linearized measurement function (or matrix) h such that $h(x) = h(x_t) + h(x_e)$ and $h(x_t) = s_t$

Navigation Aids (Navaids)

- Thus, we have

$$\begin{aligned} z &= s - h(x) \\ &= s_t + s_e - h(x_t) - h(x_e) \\ &= s_t - h(x_t) + s_e - h(x_e) \\ &= h(x_t) - h(x_t) + s_e - h(x_e) \\ &= s_e - h(x_e) \end{aligned}$$

- Our measurement, therefore, is the difference between the navaid sensor errors and the INS errors

Navigation Aids (Navaids)

- If s_e is more than just white measurement noise, then we would need to model this vector in the filter according to known or assumed dynamics

- Hence, sometimes adding a new navaid means adding new states to your filter

Navigation Aids (Navaids)

- It is crucial to examine the innovations sequence for any non-white time correlations!

$$z_k - h\left[\widehat{x}^-(k)\right] = s_e(k) - h\left[(x_e(k)\right] - h\left[\widehat{x}^-(k)\right]$$

  where we have added the index k for clarity

- We should have $\mathsf{E}\left[(z_k - h\left[\widehat{x}^-(k)\right])(z_j - h\left[\widehat{x}^-(j)\right])^\top\right] = 0$ when $k \neq j$, which will in general be true provided $\mathsf{E}\left[s_e(k)\right] = 0$

- If this is not the case, then it is probably s_e that is the culprit and you'd better start modeling!

- The condition

$$\mathsf{E}\left[(z_k - h\left[\widehat{x}^-(k)\right])(z_j - h\left[\widehat{x}^-(j)\right])^\top\right] = 0$$

  says that the innovations sequence is uncorrelated in time.

- Recall that the innovations sequence is $\nu_k = z_k - h\left[\widehat{x}^-(k)\right]$

- Thus, we should have $\mathsf{E}\left[\nu_k \nu_j^\top\right] = 0$

- This means that the optimal estimator $\widehat{x}^+(k)$ always makes optimal use of the new information provided by ν_k

GPS: The Global Positioning System

- 24+ satellites orbiting at 10,900 miles above surface
- Three Segments: Control, Space, User
- Time-tagged signals and known satellite position enable calculation of signal time of flight and hence range traversed from satellite to receiver
- Range to four satellites required to determine time plus three position coordinates

GPS: The Global Positioning System

- Satellites have precise atomic clocks
- User set receiver has less precise clock
- Time is of the essence: comparing satellite-generated time-tag with user clock time of arrival determines transit time
- Note that user clock can be a source of systematic error

GPS: The Global Positioning System

- Three types of navaid coupling:
  - Loose coupling
  - Tight coupling
  - Deep coupling

Loosely-Coupled GPS

- GPS receiver-generated position and velocity are differenced with INS position and velocity
- GPS position and velocity are generally pre-filtered (with a Kalman filter) before being used in inertial filter
- GPS position and velocity are also composite quantities from all satellites
- Least accurate approach to using GPS as a navaid

Loosely-Coupled GPS

- The GPS receiver does not have the advantage of using inertial information to completely separate process noise from measurement noise

- Simplifies Kalman integration: Don't necessarily have to model GPS receiver clock errors (assume receiver has already done so)

Tightly-Coupled GPS

- Pseudorange and delta pseudorange for each individual satellite are direct measurements presented to the filter
- Contains only measurement noise, not receiver filter's model of process noise
- Gives INS filter more fine-grained control over GPS measurements
- Higher accuracy

Deeply-Coupled

- Uses high-rate inertial data to aid in code and carrier phase tracking loops

- Most accurate solution when combined with tightly-coupled INS filter as well

GPS Errors

- Ephemeris data: Satellite's reported position can be in error
- Satellite clock: Negligible since SA (selective availability) has been turned off
- Ionosphere: Pseudorange errors caused by free electrons in the ionosphere

GPS Errors (cont)

- Troposphere: Pseudorange errors casued by temperature, pressure, humidity influences on the speed of light

- Multipath: Errors casued by locking on to reflected signals

- Receiver Noise: Errors caused by receiver's thermal noise and software inaccuracies

Error Source	**C/A-Code Error Budget (meters)**	**P-Code Error Budget (meters)**
Multipath	12.0	1.2
Receiver Noise / Software	11.1	1.1
Ionosphereic Delays	9.0	3.1
Satellite Clock and Ephemeris Errors	3.9	3.9
Tropspheric Delays	2.0	2.0

Using Loosely-Coupled GPS Measurements

- Position
  - Form primary GPS position observation by subtracting Kalman-corrected INS reference position from raw GPS position
  - Next, the innovation (or measurement residual) is formed by subtracting off the extrapolated state estimate of position error
  - Extrapolated position error estimate needs to compensate for accumulated velocity error during Kalman cycle (or some fraction thereof)

An example (JOVIAL) from a transport aircraft navigation filter:

```
165  COMPUTE RESIDUALS
166      RESIDUALS(1) = Y'KAL'OBS(INU,1) * OBS'COUNT'SCALER - SVVAR(1)
167  - 5.0 * SVVAR(4);
168      RESIDUALS(2) = Y'KAL'OBS(INU,2) * OBS'COUNT'SCALER - SVVAR(2)
169  - 5.0 * SVVAR(5);
170      RESIDUALS(3) = Y'KAL'OBS(INU,7) * OBS'COUNT'SCALER - SVVAR(3)
171          - 5.0 * SVVAR(6);
```

GPS Velocity Measurements

- Based on carrier phase measurements in receiver
- Generally independent of position measurements
- Observation is once again difference between Kalman-compensated INS reference velocity and GPS-indicated velocity

GPS Measurment Noise Covariance

- Based on EHE (estimated horizontal error) and EVE (estimated vertical error)

- These numbers highgly dependent on GDOPs

Geometric Dillution of Precision (GDOP)

- Satellites are not geostationary
- H matrix is time-varying
- Observability varies with time
- $GDOP = \sqrt{\text{tr}\left(H^\top H\right)^{-1}}$

GDOP Chimney

- Large DGOP is bad; normal values 3-5
- Singularity periods of time (about 1 hr) when GDOP values spike upward with a peak as much as 1000
- GDOP 50 $\approx$ 500 m position error
- Chimney's effects should be less acute if receiver has enough channels to track many satellites and to handle constellation changes

Position Fix

- Overfly Fix
- Known location
- Delta formed between overfly location and indicated, Kalman-compensated INS position
- Usually a one-shot opportunity, i.e., not periodic like GPS

Radar Fixes

- Instantaneous azimuth and elevation angles to a target of known latitude and longitude
- Or converted to local Cartesian coordinates (north, east, and altitude)
- Amount of cursor movement required to align curson with target yields position fix delta
- Otherwise, processed in filter like GPS or visual position fix

Doppler Velocity Sensor (DVS)

- DVS provides genuine velocity measurement with respect to ground

- DVS provides heading and drift velocities V_H and V_D

- Nonlinear measurement relationship to INS-indicated velocities V_x and V_y and platform azimuth β:

$$V_H = -V_x \sin\beta + V_y \cos\beta$$
$$V_D = V_x \cos\beta + V_y \sin\beta$$

DVS

- Relationship between (V_x, V_y) and (V_H, V_D) is non-linear h

- To generate a linearized H matrix we compute partial derivatives (Jacobians) to yield:

$$H = \begin{bmatrix} 0 & -\sin\beta & 0 & 0 & \cos\beta & 0 & 0 & 0 & -V_x\cos\beta - V_y\sin\beta \\ 0 & \cos\beta & 0 & 0 & \sin\beta & 0 & 0 & 0 & -V_x\sin\beta + V_y\cos\beta \end{bmatrix}$$

  for a canonical 9-state filter.

External Tracking Aids

- External platform tracks your vehicle and sends position / velocity info (JTIDS)

- Bearing and distance measurments
  - VOR, DME, TACAN

Digital Terrain Elevation Data (DTED)

- Published by National Geospatial Intelligence Agency (NGA)
  - formerly NIMA, formerly DMA
- 1 deg geocells of terrain height for each lat/long pair
- DTED Levels 0,1,2

DTED

- As navaid, a trail of radar altimeter measurements are stored
- Trail is correlated with DTED to determine offset
- SITAN, TERPROM

Day 4, Segment 3
Filter Maintenance
Topics

- Filter Performance and Integrity Monitoring
- Replacing Navaids
- Adding New Navaids
- Augmenting the State Vector

Filter Performance and Integrity Monitoring

- Covariance Monitoring
- Innovations Monitoring
- Corrective Action

Covariance Monitoring

- Ensuring Symmetry
- Managing Observability
- Ensuring Positive Definiteness

Covariance Monitoring: Ensuring Symmetry

- P is supposed to be positive definite and symmetric
- Symmetry means that $P^\top = P$, i.e., for each element p_{ij} of P we must have $p_{ij} = p_{ji}$
- Roundoff error can cause loss of symmetry
- The best way to ensure symmetry is to work only with items on or (say) above the diagonal and simply define elements below he diagonal by fiat.

Covariance Monitoring: Ensuring Symmetry

- Both $P(+)$ and $P(-)$ need to be symmetrized
  - After extrapolation: symmetrize
  - After update: symmetrize

Covariance Monitoring: Managing Observability

- Observability means there is a linear path from the measurements to states

- Structural observability determined by Φ and H:

If H is $1 \times n$, where n is the size of the state vector, then the state x_0 is said to be **observable** from n scalar measurements $\{z_0, \ldots, z_{n-1}\}$ if the block matrix

$$\begin{bmatrix} H^\top & \Phi^\top H^\top & \cdots & (\Phi^\top)^{n-1} H^\top \end{bmatrix}$$

is of rank n

Covariance Monitoring: Managing Observability

- If unobservable states are also unstable, estimation errors will be unstable and P will show symptoms
  - diag$(P) \to \infty$
  - I.e., one or more diagonal elements of P will grow without bound
- Not the same as divergence: "feature" of the design

Covariance Monitoring: Managing Observability

- Note also that the state estimation error variances (i.e., the diagonals of P) will naturally grow in the absence of measurements

- Going for a long time without incorporating a measurement is equivalent to lack of observability in the associated states

- Use covariance analysis simulation to detect non-observability

- The only real solution is to add new measurements

Covariance Monitoring: Ensuring Positive Definiteness

- P is supposed to be positive definite
- This means that $x'Px > 0$ whenever $x \neq 0$
- States modeled as random constants or other deterministic states can cause loss of positivity
- Add covariance bumping: Add small amounts of process noise to diagonal elements of Q for these states

Replacing Navaids

- Replacing a given sensor with another of the same type usually requires no changes to the state model
  - unless replacement sensor has significant errory dynamics that need to be modeled (see adding new sensors and state augmentation)
- Measurement noise covariance generally has to change however

Adding New Navaids

- What kinds of navaids are desirable?
- What are the steps involved in adding a new navaid?

Adding New Navaids
Desirable Characteristics:

- Navaids which render observable otherwise unobservable states
- Those having a simple relationship (preferably linear) to the state vector
- Having only white noise error dynamics

Augmenting the State Vector

- State Vector Augmentation: Expanding the size of the state vector
  - i.e., adding new states

- Why?
  - new navaids with their own error dynamics (e.g., GPS)
  - correlated (non-white) process noise
  - correlated (non-white) measurement noise

Augmenting the State Vector

- Why? (continued)
  - Correlation $\Rightarrow$ dynamics
    - No dynamics in white noise
  - If dynamics are present they must be modeled
  - Otherwise, dynamic effects will be allocated to other states

Augmenting the State Vector

- How?
  - model non-white noise as result of feeding white noise through a linear system
  - new states are outputs of new integrators
- State transition matrix Φ and error covariance matrix P grow as j^2, where j is number of new states
- Geometry matrix H grows as $m \cdot j$, where m is number of measurements and j is number of new states

Augmenting the State Vector
Example: Ship Inertial Navigation Systems (SINS)

- Long mission times
  - Especially submarines
- Position fixes insufficient to calibrate gyro bias drift w/o further modeling

Augmenting the State Vector
Example: Ship Inertial Navigation Systems (SINS)

- Gyro bias drift due to slowly changing bias
- Can be modeled as random walk
- Random walk modeled as integrated white noise
- Kalman filter needed to estimate drift

Augmenting the State Vector
Example: Ship Inertial Navigation Systems (SINS)

- Assume platform tilts are kept close to zero through velocity measurements damping the Schuler oscillations

- North-West-Up (NWU) coordinate frame (or West-South-Up)

Augmenting the State Vector
Example: Ship Inertial Navigation Systems (SINS)

- Two angular position errors in radians: ψ_N and ψ_W
  - Note ψ_N is a small angular rotation about the north axis and thus induces a west position error.
  - Similarly, ψ_W is a small rotation about the west axis and thus induces a south position error.

- $\theta_U = \psi_Z - \psi_N \tan(\lambda)$, where ψ_Z is platform azimuth error and λ is the latitude.

Augmenting the State Vector
Example: Ship Inertial Navigation Systems (SINS)

- Error Dynamics:

$$\begin{aligned} \dot{\psi}_N - \Omega_U \psi_W &= \delta g_N \\ \dot{\psi}_W + \Omega_U \psi_N - \Omega_N \theta_U &= \delta g_W \\ \dot{\theta}_U + \Omega_N \psi_W &= \delta g_U \end{aligned}$$

  where $\delta g_N, \delta g_W, \delta g_U$ are non-white gyro bias drift errors and $\Omega_N = \Omega \cos(\lambda)$ and $\Omega_U = \Omega \sin(\lambda)$ are components of earth rate

Augmenting the State Vector
Example: Ship Inertial Navigation Systems (SINS)

- Since $\delta g_N, \delta g_W, \delta g_U$ are non-white, the question arises as to what kind of model is appropriate

- Empirical data indicates that gyro biases exhibit random walk behavior over time

- Since random walk is the integral of white noise, white noise can be viewed (in some sense) as the derivative of random walk

Augmenting the State Vector
Example: Ship Inertial Navigation Systems (SINS)

- Hence, the following dynamics of random walk apply:

$$\dot{\delta} g_N = w_N$$
$$\dot{\delta} g_W = w_W$$
$$\dot{\delta} g_U = w_U$$

where w_N, w_W, w_U are white noise processes.

Augmenting the State Vector
Example: Ship Inertial Navigation Systems (SINS)

- We use the random walk equations to augment the state dynamics equations

- Thus we now have a six-state model:

$$\begin{bmatrix} \dot{\psi}_N \\ \dot{\psi}_W \\ \dot{\theta}_U \\ \dot{\delta}g_N \\ \dot{\delta}g_W \\ \dot{\delta}g_U \end{bmatrix} = \begin{bmatrix} 0 & \Omega_U & 0 & 1 & 0 & 0 \\ -\Omega_U & 0 & \Omega_N & 0 & 1 & 0 \\ 0 & -\Omega_N & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} \psi_N \\ \psi_W \\ \theta_U \\ \delta g_N \\ \delta g_W \\ \delta g_U \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ w_N \\ w_W \\ w_U \end{bmatrix}$$

Augmenting the State Vector: Summary

- Straightforward: When new, relevant dynamic relations found, simply add the states to the state vector and add the new dynamics to the F and/or Φ matrix

- Use basic models such as low-order systems driven by white noise to model new states

Augmenting the State Vector: Summary

- Some Popular Models
  - random walk
  - random constant
  - random ramp
  - Gauss-Markov (i.e., exponentially correlated)
  - combinations of the above!

Exercises

1. Write a simple 1-state model for a gyro bias modeled as a random constant

2. Now augment this model to a 2-state model by adding a random walk

Day 4, Segment 4
Filter Redundancy and Fault Tolerance
Topics

- Typical Weighting Scheme
- Detecting Faults
- Graceful Degradation

Typical Weighting Scheme

- Two separate INS error dynamics filters
- Use error covariance outputs from each filter to weight state estimates

Typical Weighting Scheme

- Weighting scheme:

$$w_i(x) = \frac{\sigma_j(x)}{\sigma_1(x) + \sigma_2(x)}$$

where $i \neq j \in \{1, 2\}$ and x is the parameter in question

Alternatives to Weighting: Decentralized Filtering

- Decentralized filtering without feedback
- Decentralized filtering with feedback

Decentralized Filtering without Feedback

- Assume two local filters

- Then for the global error covariance we have:

$$P^{-1} = P_1^{-1} + M_1^{-1} + P_2^{-1} + M_2^{-1} + M^{-1}$$

where P_1, P_2 are updated local error covariances for each filter, M_1, M_2 are extrapolated local error covariances prior to update, and M is the global filter's prior error covariance.

Decentralized Filtering without Feedback

- The global filter's estimate is

$$\widehat{x} = P\left[P_1^{-1}\widehat{x}_1 - M_1 m_1^{-1} + P_2^{-1}\widehat{x}_2 - M_2 m_2^{-1} + M^{-1}m\right]$$

  where m_1, m_2, m are the local a priori estimates for local filters 1 and 2 and the global filter, respectively.

Exercises

1. Let the x position error variance for filter 1 be 4 meters, that of filter 2 16 meters. Calculate the weighted average latitude given lat1=33.3 deg, lat2=33.21.

2. How would the decentralized filter's estimate differ from the weighted average?